EKNATHALABS · INTERVIEW PREP

DevSecOps / Cloud & Supply-Chain Security — EknathaLabs Interview Guide

Platform, SRE & DevOps interview guide

Questions	**155 questions · spoken answers included**
Online (always latest)	**interviewhub.eknathalabs.com/devsecops_security_interview_guide.html**
Printed	**2026-10-21**

Free to use and share for personal study. Answers are written for interview practice, so check command flags and versions against the official documentation before running anything in production.

certs.eknathalabs.com

Q1 What's wrong with storing secrets in environment variables, and what are the alternatives?

Secrets Mgmt

🎙 Spoken answer · ~88 s

Environment variables are convenient, but they leak easily. They're inherited by child processes, so any subprocess or plugin sees them. They show up in crash dumps, debug pages, process listings such as /proc/PID/environ, docker inspect, and Kubernetes pod specs, and they're often printed by logging or error reporting tools that dump the environment. They are static, so rotation requires a restart, and they're frequently committed in .env files or CI configuration. They also give no audit trail of who read them. Alternatives, in rough order of preference: eliminate the secret with workload identity, so the application uses short-lived tokens issued to its identity, with no stored credential at all; fetch secrets at runtime from a manager such as Vault, AWS Secrets Manager, or Google Secret Manager, using the application's identity; mount secrets as files in a memory-backed volume through the CSI driver or the External Secrets Operator, with restrictive file permissions, which are easier to rotate and don't leak through the environment; and use dynamic, short-lived credentials such as Vault database leases. If you must use environment variables, scope them to a single process, inject them at deploy time from a secret store, keep them out of logs and images, and rotate them. Add secret scanning and audit logging around it.

Likely follow-ups

- Why do files mounted from a secret store beat environment variables?
- What does workload identity remove?

```
# env vars leak via:
```

```
/proc/<pid>/environ      ← readable by same-uid processes
```

```
docker inspect, crash dumps, child processes (inherited)
```

```
CI logs that echo the environment
```

Env vars are better than hardcoding but still leak: they're inherited by every child process, visible in `/proc/<pid>/environ`, captured in crash dumps and `docker inspect`, and often accidentally printed in CI logs. They're acceptable for low-sensitivity config but weak for high-value secrets. Better: mount secrets as files from a secrets manager (tmpfs, mode 0400), or fetch them at runtime via a sidecar/agent so they never persist.

Pragmatic stance: Env vars injected by the platform from a secrets manager (not committed anywhere) are fine for most apps. Reserve file-mount/dynamic secrets for the crown jewels.

📚 Official docs

- OWASP: Secrets Management Cheat Sheet

certs.eknathalabs.com

Q2 How does HashiCorp Vault’s dynamic secrets model improve on static credentials?

Secrets Mgmt

🎙 Spoken answer · ~87 s

Static credentials are long-lived, widely shared, hard to rotate, and impossible to attribute: if a database password leaks, it works until someone notices and changes it everywhere. Vault's dynamic secrets generate credentials on demand, for a database, cloud IAM, SSH, or PKI. When an application requests access, Vault creates a unique user or key with a defined role and a lease, say one hour, and returns it. When the lease expires or is revoked, Vault deletes the credential from the target system. Benefits: every consumer has unique credentials, so audit logs show exactly who accessed what; the blast radius of a leak is small and time-limited because the credential expires quickly; there's no manual rotation, since credentials are continuously renewed or replaced; revocation is immediate, and you can revoke all credentials of a compromised application by lease prefix; and no one needs to know or store a long-lived password. Operationally, Vault needs a privileged connection to the target system to create users, applications must handle credential renewal and rotation gracefully, and connection pools must refresh. Vault itself becomes critical infrastructure, requiring high availability, auditing, and secure unsealing. The Vault agent or sidecar can handle renewal for applications. Dynamic secrets move security from protecting secrets to making them disposable.

Likely follow-ups

- What happens at lease expiry?
- What operational burden does Vault introduce?

```
# app requests a short-lived DB credential on demand
```

```
vault read database/creds/readonly
```

```
# → username/password valid for e.g. 1h, auto-revoked after
```

Static credentials live forever, get shared, leak, and are painful to rotate. Vault’s dynamic secrets are generated on demand, scoped to a single consumer, and have a short TTL — Vault creates a unique DB user per request and automatically revokes it when the lease expires. This shrinks the blast radius (a leaked credential is useless in an hour), gives per-app attribution, and makes rotation automatic rather than a manual fire drill.

Why SREs love it: "Rotate all DB passwords" becomes a non-event — credentials are ephemeral by design, so there’s nothing long-lived to rotate.

📚 Official docs

- Vault: Dynamic database secrets
- HashiCorp Vault documentation

certs.eknathalabs.com

Q3 Explain the difference between encryption at rest and envelope encryption with a KMS.

Secrets Mgmt

🎙 **Spoken answer · ~85 s**

Encryption at rest means stored data is encrypted on disk, so someone stealing the storage media or a snapshot can't read it. The question is how keys are handled. Envelope encryption is a layered scheme: a data encryption key, DEK, encrypts the actual data, and then a key encryption key, KEK, held in a KMS wraps, or encrypts, the DEK. The wrapped DEK is stored next to the encrypted data, while the KEK never leaves the KMS. To decrypt, the application sends the wrapped DEK to the KMS, which unwraps it, subject to IAM permissions and audit logging, and the application uses the plaintext DEK in memory to decrypt the data. Why this design: you don't send large data to the KMS, which is slow and limited in payload size; you can use a unique DEK per object, limiting exposure if one is compromised; rotating the KEK only requires re-wrapping DEKs, not re-encrypting all the data; the KMS gives centralised access control, audit trails, and optional hardware security module protection; and you can revoke access by disabling the KEK. Cloud services like S3 SSE-KMS, EBS, and GCS CMEK use this model, and Kubernetes can encrypt Secrets in etcd with a KMS provider the same way.

Likely follow-ups

- Why not send all data to the KMS directly?
- How does key rotation work with envelopes?

```
data → encrypted with a DEK (data encryption key)
```

```
DEK → encrypted with a KEK (key encryption key) held in KMS
```

```
store: ciphertext + wrapped DEK; KEK never leaves the KMS/HSM
```

Envelope encryption uses two layers: a local data key (DEK) encrypts the actual data, and a master key (KEK) in the KMS encrypts the DEK. You store the ciphertext alongside the wrapped DEK; to decrypt, you ask the KMS to unwrap the DEK. The KEK never leaves the KMS/HSM, so compromising your storage doesn't expose the master key, and you can rotate the KEK without re-encrypting all data — just re-wrap the DEKs.

Why two keys: Encrypting gigabytes directly with a KMS call is slow and rate-limited. The DEK does bulk crypto locally; the KMS only handles the tiny DEK.

📚 **Official docs**

- Kubernetes: Encrypting Secret data at rest
- AWS KMS

certs.eknathalabs.com

Q4 How do you manage secrets in Kubernetes — and why are native Secrets not enough?

Secrets Mgmt

🎙 Spoken answer · ~86 s

Native Kubernetes Secrets are only base64 encoded, not encrypted, by default. They're stored in etcd in plain form unless you enable encryption at rest, anyone with read access to the Secret API, or to etcd or its backups, can read them; RBAC mistakes with get or list on secrets are common; they appear in the environment or volumes of any pod that references them; and they are usually committed to Git as manifests, which exposes them. There's also no built-in rotation, versioning, fine-grained audit, or dynamic generation. Good practice layers on top. Enable etcd encryption with a KMS provider. Tighten RBAC so only the needed service accounts can read specific secrets, and restrict list and watch. Keep secret values out of Git by using an external store, with the External Secrets Operator or Secrets Store CSI driver syncing from Vault or a cloud secret manager, or Sealed Secrets or SOPS when GitOps requires encrypted values in the repository. Prefer mounted files over environment variables. Use workload identity so pods fetch secrets directly with short-lived credentials. Use short-lived or dynamic credentials where possible. Enable audit logging on secret access, use namespace isolation, and consider network policies for the secrets provider. Rotate regularly and scan for exposed secrets.

Likely follow-ups

- Why is base64 not encryption?
- What does enabling KMS encryption for etcd protect against?

```
# native Secrets are only base64 — NOT encrypted
```

```
echo "cGFzc3dvcmQ=" | base64 -d   → password
```

```
# harden: EncryptionConfiguration (etcd at-rest) + RBAC + ESO
```

Kubernetes Secrets are merely base64-encoded, stored in etcd, and readable by anyone with get access in the namespace — not real encryption. Hardening: enable `EncryptionConfiguration` so etcd encrypts secrets at rest (ideally KMS-backed), lock down RBAC so few principals can read them, and prefer the External Secrets Operator or Secrets Store CSI driver to sync from Vault/cloud secrets managers rather than committing Secret manifests.

Never commit Secret YAML to Git — base64 is encoding, not encryption. Use Sealed Secrets, SOPS, or an external store so Git never holds plaintext-equivalent secrets.

📚 Official docs

- Kubernetes: Secrets

certs.eknathalabs.com

Q5 What is the External Secrets Operator and how does it work?

Secrets Mgmt

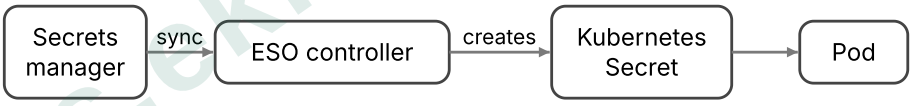
🎙 Spoken answer · ~86 s

The External Secrets Operator is a Kubernetes operator that synchronises secrets from external managers, such as AWS Secrets Manager, Google Secret Manager, Azure Key Vault, and HashiCorp Vault, into Kubernetes Secrets. You define a SecretStore, or ClusterSecretStore, that describes how to connect and authenticate to the provider, preferably via workload identity so no static credentials are needed. Then you create an ExternalSecret resource that names the remote keys, and how to map them to a target Kubernetes Secret, with templating options. The operator's controller watches these resources, fetches the values from the provider, creates or updates the Secret in the cluster, and re-syncs at a configured refresh interval, so rotations in the source propagate automatically. Benefits: the source of truth stays in a purpose-built secret manager with auditing, access control, and rotation; the Git repository contains only the ExternalSecret references, not values, which fits GitOps; applications consume ordinary Kubernetes Secrets without code changes. Considerations: the synced Secret still exists in etcd, so enable encryption at rest and restrict RBAC; pods mounting the Secret need a restart or reload to see updated values unless they watch the files; the operator's permissions to the provider should be least privilege; and the refresh interval trades freshness against API costs.

Likely follow-ups

- How does a rotated value reach a running pod?
- Why is the secret still in etcd?

🗺 External Secrets Operator




```
apiVersion: external-secrets.io/v1beta1
```

```
kind: ExternalSecret
```

```
spec:
```

```
  secretStoreRef: {name: vault-backend, kind: SecretStore}
```

```
  target: {name: db-creds}
```

```
  data: [{secretKey: password, remoteRef: {key: prod/db, property: pw}}]
```

The External Secrets Operator (ESO) syncs secrets from an external source of truth (Vault, AWS Secrets Manager, GCP Secret Manager) into native Kubernetes Secrets. You declare an `ExternalSecret` CR pointing at a backend; ESO fetches the value and materializes a K8s Secret, refreshing it on a schedule. This keeps

certs.eknathalabs.com

the authoritative secret outside the cluster and out of Git, while apps still consume a normal Secret — no app changes needed.

GitOps-friendly: You commit the ExternalSecret reference (no sensitive data) to Git; the actual value lives in Vault. Best of both worlds.

📚 **Official docs**

- External Secrets Operator

Q6 How do you detect secrets that have already been committed to a Git repo?

Secrets Mgmt

🎙 **Spoken answer · ~84 s**

Detecting committed secrets involves scanning both current files and the entire history, because deleting a secret in a later commit doesn't remove it from Git history. Tools: gitleaks, TruffleHog, detect-secrets, and GitHub's secret scanning, which use pattern matching for known formats, entropy detection for random-looking strings, and TruffleHog can verify whether a found credential is still active by testing it against the provider. Run a full-history scan on repositories initially, including all branches and forks. Then prevent new leaks with layered controls: pre-commit hooks that scan staged changes locally, CI checks on pull requests that fail on findings, and server-side push protection, as GitHub offers, which blocks pushes containing secrets. Scan beyond code: container images, build artefacts, logs, wikis, issue trackers, and chat. When you find one, treat it as compromised: revoke and rotate immediately, because removing it from history with git filter-repo or BFG does not undo exposure to anyone who cloned or to caches and forks. Investigate usage of the credential via audit logs. Then fix the root cause by moving to a secret manager, and track metrics like time to revoke. Manage false positives with allow-lists carefully, and keep the rulesets updated for your organisation's custom secret formats.

Likely follow-ups

- Why is rewriting Git history not enough?
- What does push protection do?

```
gitleaks detect --source . --redact      # scan working tree + history
```

```
trufflehog git file://. --only-verified   # verifies live creds
```

```
# pre-commit hook to PREVENT new ones:
```

```
gitleaks protect --staged
```

Tools like `gitleaks` and `trufflehog` scan the entire Git history (not just the current tree) for high-entropy strings and known credential patterns; trufflehog can even verify whether a found key is still live. Run them in CI to block PRs and as a pre-commit hook to stop secrets before they're ever committed. Scanning history

certs.eknathalabs.com

matters because a secret deleted in a later commit still lives in the Git objects forever.

> **Detection isn't remediation:** A committed secret must be *rotated*, not just deleted. Removing it from history (BFG/filter-repo) doesn't help if someone already cloned the repo — assume it's compromised.

> 📚 **Official docs**
>
> - gitleaks: detect secrets in git history

Q7 Explain secret rotation — what makes it hard and how do you automate it? Secrets Mgmt

> 🎙 **Spoken answer · ~88 s**
>
> Secret rotation means regularly replacing credentials so that a leaked one has a limited lifetime, and so you can replace it quickly in an emergency. It's hard because secrets are used in many places: applications, CI jobs, scripts, and other teams' systems, and a change must be coordinated, otherwise things break. Challenges: knowing everywhere a secret is used; downtime between changing the secret at the source and updating consumers; systems that read secrets only at startup; shared credentials owned by several teams; third-party systems with manual rotation; and fear of outages, which makes teams avoid rotating. Automation approaches: use a secret manager with built-in rotation, such as AWS Secrets Manager rotation Lambdas, or Vault's rotation features. Use a two-phase or dual-credential pattern: create the new credential, keep the old valid, let consumers pick up the new one, verify, then retire the old, which avoids downtime. Prefer dynamic short-lived credentials, which make rotation continuous and automatic. Use workload identity to eliminate secrets where possible. Have applications reload secrets without restarts, via file watching or sidecars, and the External Secrets Operator to propagate changes. Test rotation regularly in non-production, monitor for failures after rotation, set alerts on secret age, and keep an emergency rotation runbook. Rotation you've never practised will fail under pressure.
>
> **Likely follow-ups**
>
> - What is the dual-credential rotation pattern?
> - Why do rotation drills matter?

```
# the dual-credential pattern for zero-downtime rotation:
```

```
1. create new credential (v2) alongside old (v1)
```

```
2. roll apps to v2; verify all healthy
```

```
3. revoke v1 only after confirming nothing uses it
```

Rotation is hard because of the cutover window: revoke too early and live apps break; too late and a leaked credential stays valid. The safe approach is overlapping credentials — issue the new one, migrate consumers, verify, then revoke the old. Automate with Vault dynamic secrets (rotation is implicit via short

certs.EknathaLabs.com

TTLs) or cloud-managed rotation (AWS Secrets Manager rotation Lambdas). Track which services use which credential so you know when it's safe to revoke.

Best rotation is no rotation: Short-lived dynamic credentials make rotation continuous and automatic, eliminating the risky manual cutover entirely.

📚 **Official docs**

- Vault: Dynamic database secrets
- OWASP: Secrets Management Cheat Sheet

Q8 What is SOPS and when would you use it over a secrets manager?

Secrets Mgmt

🎙 **Spoken answer · ~84 s**

SOPS, Secrets OPerationS from Mozilla, now under the CNCF, encrypts the values in structured files such as YAML, JSON, or ENV, leaving the keys readable, using keys from a KMS like AWS KMS, GCP KMS, Azure Key Vault, or age or PGP. This produces files that can be safely committed to Git, with diffs showing which keys changed, and decrypted at deploy time by tools such as Flux's built-in SOPS support, the KSOPS plugin for Kustomize, or helm-secrets. When would I use it over a secrets manager? It's well suited to GitOps workflows where configuration, including secrets, should be versioned and reviewed together in the repository, and to small teams or setups without the cost and operational overhead of running Vault or paying for a managed service. It also works for bootstrapping, such as the initial credentials a secrets manager itself needs. Downsides: the decryption keys become the crown jewels and need strong control; there's no dynamic secrets, central rotation, fine-grained runtime audit, or per-secret access policy like a manager provides; rotating a secret means committing a new encrypted value and redeploying; and removing a person's access requires re-encrypting with new keys and acknowledging that they may hold old Git history.

Likely follow-ups

- Why does GitOps favour SOPS?
- What are SOPS's limitations versus Vault?

```
# encrypt only the values, keep keys readable, commit to Git
```

```
sops --encrypt --kms arn:aws:kms:... secrets.yaml > secrets.enc.yaml
```

```
sops --decrypt secrets.enc.yaml   # decrypts on demand
```

SOPS (Secrets OPerationS) encrypts the *values* in a YAML/JSON/env file while leaving keys in plaintext, so the file stays diff-friendly and commitable to Git. It encrypts with a KMS/age/PGP key, so only authorized principals can decrypt. Use it for GitOps secret-in-repo workflows where you want secrets versioned alongside config and don't want to run a full Vault — common in smaller teams or for bootstrapping config.

certs.eknathalabs.com

SOPS vs Vault: SOPS = encrypted files in Git, simple, static. Vault = central service, dynamic secrets, audit, leasing. SOPS for config-as-code; Vault for scale and dynamic needs.

📚 **Official docs**

- AWS Secrets Manager
- Mozilla SOPS

Q9 How do you prevent secrets from leaking into logs and observability tooling?

Secrets Mgmt

🎙 **Spoken answer · ~89 s**

Preventing secrets in logs and observability data takes a combination of design, filtering, and detection. At the source: never log request bodies, headers like Authorization, cookies, or full connection strings by default; use structured logging with explicit allowlisted fields instead of dumping objects; wrap secrets in types whose string representation is redacted; disable verbose or debug logging in production, and avoid printing environment variables. Don't put secrets in URLs or query strings, since they get logged by proxies and load balancers. Redaction in the pipeline: use log processors in Fluent Bit, Vector, or the OpenTelemetry Collector to mask patterns such as tokens, keys, and card numbers before data is stored; apply the same to traces and span attributes, and to error trackers like Sentry with scrubbing rules. Detection: run secret scanners over log storage and alert on matches, and test in CI for sensitive data in log output. Access and retention: restrict who can read logs, with RBAC, keep retention short for sensitive systems, and encrypt storage. For CI/CD, use the platform's secret masking, though it's not foolproof for transformed values. Educate developers with code review checks. If a secret does leak into logs, treat it as compromised, rotate it, and purge or restrict the affected logs. Metrics labels need the same care.

Likely follow-ups

- Why is secret masking in CI not enough?
- What do redaction processors in the collector do?

certs.eknathalabs.com

```
# scrub at multiple layers:
```

```
app: redact known fields before logging (password, token, authz)
```

```
pipeline: log-shipper regex filters (Fluent Bit/Logstash)
```

```
code review + linters that flag logging of secret vars
```

Secrets leak into logs via stack traces, request/response dumps, and debug logging of config objects. Defend in depth: redact sensitive fields in the app before they're logged (allowlist what you log, not denylist), add scrubbing filters in the log pipeline as a backstop, mask values in error reporters (Sentry), and never log

full request headers (Authorization). Treat logs as a potential exfiltration channel — anyone with log access effectively has those secrets.

Debug logging is the usual culprit: "log the whole config to debug startup" dumps every secret. Redact at the source; a downstream filter is only a safety net.

📚 **Official docs**

- OWASP: Secrets Management Cheat Sheet

Q10 What is workload identity (IRSA / GKE Workload Identity) and why is it better than keys?

Secrets Mgmt

🎙 **Spoken answer · ~90 s**

Workload identity lets a workload authenticate to cloud services as a cloud identity without holding a long-lived key. On AWS EKS, IRSA, IAM Roles for Service Accounts, uses the cluster's OIDC provider: the pod's Kubernetes service account token is a signed JWT, and the IAM role's trust policy trusts that issuer and a specific namespace and service account; the AWS SDK exchanges the token with STS for temporary credentials via AssumeRoleWithWebIdentity. EKS Pod Identity is a newer simplification. On GKE, Workload Identity maps a Kubernetes service account to a Google service account, or directly grants IAM roles to the Kubernetes principal, and the metadata server issues short-lived tokens. It's better than keys because there's no secret to store, distribute, leak, or rotate; credentials are short-lived, typically an hour, and auto-refresh; permissions are scoped per workload, enabling least privilege rather than node-wide roles; and audit logs show which workload made each call. The old alternative, giving the node's instance role to all pods, or mounting access keys as Secrets, gives every pod on a node the same broad permissions and creates leak risks. To harden: bind each service account to a narrowly scoped role, restrict the trust policy to specific namespace and service account names, block pod access to the node metadata endpoint, and monitor usage.

Likely follow-ups

- Why is the node's instance role a bad alternative?
- How does the trust policy limit which pods can assume a role?

```
# pod assumes a cloud IAM role via its K8s service account
```

```
# EKS (IRSA): SA annotated with an IAM role ARN
```

```
# GKE: SA bound to a GCP service account
```

```
# → SDK gets short-lived, auto-rotated creds. No stored keys.
```

certs.eknathalabs.com

Workload identity lets a pod's Kubernetes service account map to a cloud IAM identity, so the SDK receives short-lived, automatically-rotated credentials from the metadata/token service — there are no static access keys to store, leak, or rotate. It's the keyless ideal: credentials are ephemeral, scoped per-workload (least privilege), and tied to the pod's identity, giving clean audit trails. This eliminates the #1 cloud breach vector: long-lived keys in code/config.

The endgame: No long-lived cloud keys anywhere in your cluster. Every workload gets just-in-time, just-enough, auto-expiring credentials.

📚 **Official docs**

- Amazon EKS: IAM roles for service accounts (IRSA)
- Azure: Managed identities
- GKE: Workload Identity Federation for GKE

Q11 How do you handle secret sprawl across many services and environments?

Secrets Mgmt

🎙 **Spoken answer · ~85 s**

Secret sprawl is when credentials multiply across repositories, CI systems, clusters, and teams with no inventory or owner. To tame it: first, discover what exists, using secret scanners across code, images, and CI variables, and cloud audit logs for credential usage, then build an inventory with owners. Centralise in a small number of managers, ideally one authoritative system per environment, with a consistent path structure such as team, service, and environment, and access policies by path. Reduce the number of secrets: replace static keys with workload identity and OIDC federation, use dynamic short-lived credentials, and avoid sharing the same credential across services, giving each its own identity. Standardise how applications consume them, using the External Secrets Operator or CSI driver, and provide templates and libraries so teams don't invent their own. Automate lifecycle: rotation, expiry dates, and tracking secret age, with alerts for stale ones. Enforce policy: block new secrets in code with pre-commit and push protection, and use CI policies to prevent plaintext secrets in manifests. Separate environments so production secrets aren't accessible from development. Assign ownership via metadata and review access periodically. Measure: number of static credentials, age distribution, and percentage managed centrally. Make the secure path the easiest, so teams don't improvise.

Likely follow-ups

- How do you build an inventory of existing secrets?
- Why give each service its own credential?

```
central source of truth (Vault / cloud SM) + naming convention:
```

```
  <env>/<team>/<service>/<secret>   e.g. prod/payments/api/db-pw
```

```
+ per-path access policies + audit + automated inventory
```

certs.eknathalabs.com

Secret sprawl — the same credential copied into a dozen places — makes rotation impossible and breaches inevitable. Fix it with a single authoritative store, a consistent hierarchical naming scheme so access policies map cleanly to teams/envs, least-privilege path policies, and audit logging of every read. Periodically inventory which workloads read which secrets so you can answer "what's affected if this leaks?" instantly.

One source of truth: The moment a secret exists in two places, rotation breaks. Reference the central store everywhere; never copy values around.

📚 **Official docs**

- OWASP: Secrets Management Cheat Sheet

Q12 What is the principle of least privilege applied to secret access?

Secrets Mgmt

🎙 **Spoken answer · ~89 s**

Least privilege for secrets means each identity can read only the specific secrets it needs, only for as long as needed, and only for the operations needed. In practice: organise secrets in a hierarchy by team, service, and environment, and write policies granting read access on exact paths to the identity of that service, never wildcard access to everything. Separate environments: a development workload must never read production secrets. Distinguish read from write, list, and delete permissions; most applications need only read of specific secrets, and shouldn't be able to list all secrets, which reveals names and structure. Authenticate workloads by their identity, such as Kubernetes service account or cloud role, instead of shared tokens, so policies attach to the workload. Limit human access: engineers get access to non-production secrets by default, and production access via just-in-time, time-bound, approved elevation with auditing. Use short-lived credentials and dynamic secrets so access expires. Use conditions, such as source network, and require MFA for humans. Avoid shared service accounts and over-broad roles like secrets admin. Review access regularly with tools that show effective permissions and unused access, and alert on unusual reads or access outside normal patterns using audit logs. Test policies, and protect the ability to modify policies themselves, since that is effectively full access.

Likely follow-ups

- Why shouldn't applications be able to list secrets?
- How does just-in-time access work for humans?

```
# scope each policy tightly:
```

```
path "prod/payments/*" { capabilities = ["read"] }   # not "*"
```

```
# + deny by default, time-bound tokens, per-service identity
```

certs.eknathalabs.com

Least privilege means each identity can read only the specific secrets it needs, nothing more — the payments service can read `prod/payments/*`, not all of `prod/*`. Combine path-scoped read-only policies, deny-by-default, per-service identities (not shared accounts), and short-lived tokens. This bounds the blast radius: a compromised service exposes only its own secrets, and audit logs cleanly attribute every access to one identity.

Blast-radius thinking: Design so that "service X is compromised" leaks the minimum. Broad wildcard policies turn one breach into total compromise.

📚 **Official docs**

- AWS IAM: Least privilege (grant least privilege)

Q13 How do you securely pass secrets into a CI/CD pipeline?

Secrets Mgmt

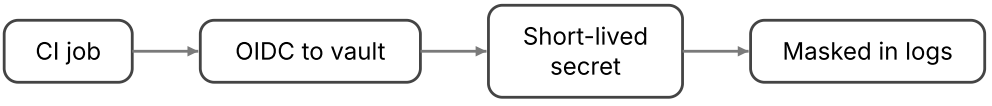
🎙 **Spoken answer · ~88 s**

Secrets in CI/CD pipelines are high-risk because pipelines run code and hold powerful credentials. The best approach is to avoid stored secrets by using OIDC federation: the CI provider issues a short-lived signed token for each job, and the cloud provider exchanges it for temporary credentials, with trust restricted to a specific repository, branch, and environment. Where secrets are needed, store them in the CI platform's encrypted secret store or fetch them at job time from Vault or a cloud secret manager using that OIDC identity. Scope narrowly: use per-environment secrets, with production secrets available only to protected branches or approved deployment jobs, and different credentials for build, test, and deploy. Expose them only to the steps that need them, not the whole job. Protect against leaks: mask secrets in logs, don't echo them or pass them on command lines, avoid debug tracing, and use build secret mounts for Docker rather than build arguments. Don't expose secrets to pull requests from forks, and be careful with pull_request_target patterns. Pin third-party actions or plugins by commit hash, since malicious ones can exfiltrate secrets. Use ephemeral runners, and restrict who can edit pipeline definitions. Rotate, audit access, and monitor for anomalous use. Use secret scanning on the repository to catch accidental commits.

Likely follow-ups

- How does OIDC federation work for CI?
- Why scope secrets to protected branches?

🗺 **Secrets in CI**




```
# use the platform’s secret store + OIDC, not plaintext vars
```

ceris.eknathalabs.com

```
# GitHub Actions → OIDC federation to cloud (no stored keys):
```

```
permissions: {id-token: write}
```

```
- uses: aws-actions/configure-aws-credentials@v4
```

```
  with: {role-to-assume: arn:..., aws-region: us-east-1}
```

The modern approach is OIDC federation: the CI runner presents a signed identity token to the cloud, which exchanges it for short-lived credentials — no long-lived keys stored in the CI system at all. For other secrets, use the platform's encrypted secret store (masked in logs), scope them to specific environments/branches, and require approvals for production. Never put secrets in plaintext workflow files or unmasked variables.

CI is a prime target: It has access to everything it deploys. Static cloud keys in CI are a top breach vector — OIDC eliminates them.

📚 **Official docs**

- Kubernetes: Good practices for Secrets

Q14 What goes in a secret-leak incident response runbook?

Secrets Mgmt

🎙 **Spoken answer · ~79 s**

A secret-leak runbook should be a concrete, practised checklist that starts with containment, as speed matters. Detection and triage: confirm the leak and identify what secret it is, where it was exposed, such as a public repository, a log, or chat, how long it has been exposed, and what it grants access to. Severity depends on privilege and exposure. Contain: revoke or disable the credential immediately; if revocation could break production, rotate it using the dual-credential method, but don't delay. Then rotate any related secrets. Investigate: use audit logs from the affected systems to determine whether the secret was used by anyone unauthorised, from which IP addresses, and what was accessed or changed, looking for new users, keys, resources, data exports, and persistence mechanisms. Scope the blast radius, including other places the same secret was reused. Remediate: remove the secret from the exposed location, including history with git filter-repo if relevant, though revocation matters more, and purge caches, forks, and logs where possible. Clean up any attacker footprint. Communicate: notify the security team, owners, and, if data was accessed, legal and compliance for disclosure obligations, with a single incident channel.

Likely follow-ups

- Why revoke before investigating?
- What do you check in audit logs after a leak?

```
1. REVOKE/rotate the credential immediately (assume compromised)
```

certs.eknathalabs.com

```
2. scope: where used? audit logs for anomalous use?
```

```
3. contain: disable affected identity, check for lateral movement
```

```
4. remediate: remove from history, fix the leak source
```

```
5. postmortem: how did it leak, how to prevent
```

When a secret leaks, treat it as compromised regardless of exposure duration. Rotate/revoke first — speed matters more than investigation. Then scope the damage (audit logs for the credential's usage), contain (disable the identity, look for lateral movement and new IAM users/keys an attacker may have created), remediate the leak source, and run a blameless postmortem. Removing the secret from Git history is necessary but never sufficient — rotation is what actually closes the exposure.

Rotate before you investigate: Every minute a leaked key stays valid is risk. Kill it first, analyze second.

📚 **Official docs**

- Google SRE Book: Managing Incidents
- Google SRE Workbook: Incident Response

certs.eknathalabs.com

Q15 What is a software supply-chain attack? Give real examples.

Supply Chain

🎙 **Spoken answer · ~87 s**

A software supply-chain attack compromises software by targeting something the victim trusts upstream, such as a dependency, build system, or update mechanism, instead of attacking the victim directly. Real examples. SolarWinds in 2020: attackers breached the build environment and inserted a backdoor into signed Orion updates, delivered to about eighteen thousand customers including government agencies. Codecov in 2021: attackers modified the Bash uploader script to exfiltrate environment variables and credentials from customers' CI environments. event-stream in 2018: a maintainer handed over a popular npm package to an attacker, who added code targeting a cryptocurrency wallet. The ua-parser-js and colors/faker incidents showed account takeover or maintainer sabotage. Dependency confusion in 2021, demonstrated by Alex Birsan, tricked build systems into pulling malicious public packages instead of internal ones with the same name. The Log4Shell vulnerability in 2021 showed how a flaw in a ubiquitous library cascades. The XZ Utils backdoor in 2024 involved a long social engineering campaign where an attacker became a maintainer and planted a backdoor in the compression library, caught almost by accident. The 3CX compromise was a supply chain attack chained from another supply chain compromise. Defences include lockfiles and pinning, SBOMs, signing and provenance, SLSA, scoped CI credentials, vetting dependencies, reproducible builds, and monitoring for anomalies.

Likely follow-ups

- How did dependency confusion work?
- What made the XZ backdoor notable?

```
SolarWinds (2020) — build system compromised, signed malware shipped
```

```
Codecov (2021)   — CI script altered, exfiltrated env secrets
```

```
xz/liblzma (2024)— malicious maintainer backdoored a core library
```

```
npm/PyPI typosquats — malicious lookalike packages
```

A supply-chain attack compromises software *before* it reaches you — via a dependency, build system, or distribution channel — so you ship the attacker's code while trusting your own pipeline. Because the malicious code arrives through a trusted path (a signed update, a popular library), traditional perimeter defenses miss it. The xz backdoor showed even a single trusted maintainer can poison a library used by millions; SolarWinds showed build-system compromise scales to thousands of victims.

Why it's the hot topic: Your code is increasingly other people's code — hundreds of transitive dependencies. The attack surface is everything you didn't write.

📚 **Official docs**

- SLSA specification

certs.eknathalabs.com

Q16 What is an SBOM and why does it matter?

Supply Chain

🎙 Spoken answer · ~88 s

A Software Bill of Materials is a machine-readable inventory of the components in a piece of software: packages, libraries, their versions, licences, and often hashes and supplier information, along with the relationships between them. Common formats are SPDX and CycloneDX; tools such as Syft, Trivy, and the Docker SBOM feature generate them from source or container images. It matters for several reasons. Vulnerability response: when a new critical CVE like Log4Shell appears, an SBOM lets you answer quickly which of your products and deployments contain the affected component, instead of scrambling to search. Risk management: you can continuously match components against vulnerability databases, and spot outdated or unmaintained ones. Licence compliance. Procurement and regulation: customers and regulators increasingly require SBOMs, for instance US executive order 14028 and the EU Cyber Resilience Act. Transparency and trust: it shows what is inside the artefact, including transitive dependencies. Practical points: generate the SBOM at build time for each artefact, store it with the artefact and attest it through signing, keep it in a searchable inventory, and update it per release. Limitations: an SBOM lists what's present, not whether a vulnerability is exploitable, which VEX addresses; accuracy depends on the tool and on capturing everything, including vendored code; and an SBOM alone does not prove integrity.

Likely follow-ups

- What formats are common for SBOMs?
- How does an SBOM help during a Log4Shell-style event?

```
# generate a Software Bill of Materials
```

```
syft packages dir:. -o spdx-json > sbom.json
```

```
trivy image --format cyclonedx myapp:1.2 > sbom.cdx.json
```

An SBOM is a complete, machine-readable inventory of every component and dependency (and version) in your software — like an ingredients label. It matters because when a new CVE drops (Log4Shell), the first question is "are we affected, and where?" Without an SBOM that takes days of manual hunting; with one it's a query. Standard formats are SPDX and CycloneDX, generated by tools like `syft` or `trivy`. Regulators and customers increasingly require them.

Log4Shell lesson: Teams with SBOMs answered "are we vulnerable?" in minutes. Teams without spent a frantic week grepping build manifests across hundreds of services.

📚 Official docs

- SPDX specification
- CycloneDX

certs.eknathalabs.com

Q17 Explain artifact signing with Sigstore/cosign and how verification works.

Supply Chain

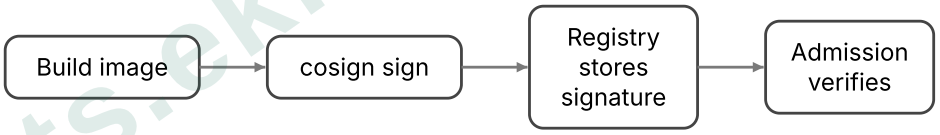
🎙 Spoken answer · ~89 s

Sigstore is an open-source project that makes signing and verifying software artefacts easy, and cosign is its tool for signing container images and other OCI artefacts. Traditional signing needed long-lived private keys that had to be protected and distributed. Sigstore's keyless flow removes that. The signer authenticates through OIDC with an identity such as a GitHub Actions workflow or an email; Fulcio, the certificate authority, issues a short-lived certificate, valid for minutes, binding the ephemeral public key to that identity. Cosign signs the artefact's digest with the ephemeral key, then the signing event and certificate are recorded in Rekor, a public append-only transparency log, and the private key is discarded. The signature and certificate are stored in the registry alongside the image. Verification: cosign verify checks that the signature is valid for the image digest, that the certificate chains to Fulcio's root, and that it was valid at the time recorded in the transparency log, and, importantly, that the identity and issuer match what you expect, for example a specific repository's release workflow. You can also sign with your own keys held in a KMS. In Kubernetes, admission controllers such as Kyverno, Connaisseur, or Sigstore policy-controller enforce that only images with valid, trusted signatures run. Always verify by digest and constrain the expected identity.

Likely follow-ups

- What does keyless signing remove?
- What identity should verification require?

🗺 Cosign signing




```
cosign sign myregistry/app@sha256:abc...   # keyless, OIDC identity
```

```
cosign verify --certificate-identity ci@org.com \
```

```
  --certificate-oidc-issuer https://token.actions... app@sha256:abc
```

Signing proves an artifact came from a trusted builder and wasn't tampered with. Sigstore/cosign enables *keyless* signing: the signer authenticates via OIDC (e.g. the CI's identity), gets a short-lived certificate, signs, and the signature + certificate are logged in a public transparency log (Rekor). Verification checks the signature, that the signer identity matches your expected builder, and that it's in the transparency log — no long-lived signing keys to manage or leak.

Keyless is the breakthrough: Traditional signing fails because the signing key leaks. Sigstore ties signatures to ephemeral OIDC identities + a public audit log instead.

certs.eknathalabs.com

📚 **Official docs**

- Sigstore documentation

Q18 What is SLSA and what do its levels mean?

Supply Chain

🎙 **Spoken answer · ~77 s**

SLSA, Supply-chain Levels for Software Artifacts, pronounced 'salsa', is a security framework from the OpenSSF that defines increasing levels of assurance about how software was built, and addresses tampering between source and consumer. In the current specification, the build track has levels. Build level 0 means no guarantees. Build level 1 requires that the build process is scripted and produces provenance, metadata describing how the artefact was built, so there's at least documentation, though it can be forged. Build level 2 requires a hosted build platform that generates and signs the provenance, so it's much harder to tamper with after the fact; builds run on a service rather than a developer's laptop. Build level 3 requires a hardened build platform with isolation between builds and protection of the signing material, so that even a malicious build step or project maintainer cannot forge provenance or influence other builds. Earlier versions had levels up to 4, including two-person review and hermetic, reproducible builds, which later moved to other tracks. Consumers verify provenance against expectations: that the artefact came from the expected source repository and commit and builder.

Likely follow-ups

- What does provenance contain?
- What distinguishes level 3 from level 2?

```
SLSA = Supply-chain Levels for Software Artifacts
```

```
L1: provenance exists (how it was built)
```

```
L2: signed provenance, hosted build service
```

```
L3: hardened, non-falsifiable provenance, isolated builds
```

SLSA is a framework of incremental maturity levels for build integrity. The core idea is **provenance** — verifiable metadata about how, where, and from what sources an artifact was built. Higher levels demand stronger guarantees: signed provenance, builds on a trusted hosted service, isolated/ephemeral build environments that can’t be tampered with, and provenance an attacker can’t forge. It gives you a roadmap to answer "can I trust that this binary was built from the source I think it was?"

Provenance is the heart: SLSA is about being able to trace an artifact back to its exact source and build process, with cryptographic proof that the chain wasn’t altered.

certs.eknathalabs.com

📚 **Official docs**

- GitHub: Artifact attestations (build provenance)
- SLSA specification

Q19 How do you scan container images for vulnerabilities and what are the gotchas?

Supply Chain

🎙 **Spoken answer · ~87 s**

Image scanning matches packages in an image against vulnerability databases to flag known CVEs. Tools like Trivy, Grype, Snyk, and registry-native scanners analyse OS packages and language dependencies, and can also find misconfigurations and secrets. Workflow: scan in CI on every build and fail on critical or high issues with fixes available, scan in the registry on push, and rescan continuously because new CVEs appear against images already built. Gotchas. Noise: many findings aren't exploitable or reachable, or have no fix yet, so use severity, fix availability, and context like exposure to prioritise, and use VEX statements to record not-affected findings. False negatives: scanners only know published CVEs, and miss vulnerabilities in vendored, statically compiled, or renamed components, binaries without package metadata, or custom code. Different scanners and databases give different results, and distribution maintainers may backport fixes without changing version numbers, causing mismatches. Base image drift: an image that's clean today can be vulnerable next week, so rebuild regularly. Scanning shows image contents, not runtime context. Gating too strictly blocks deployments for unfixable findings, so use expiry-based exceptions. Reduce findings by using minimal and distroless base images, removing unneeded packages, and multi-stage builds. Scan IaC and Dockerfiles too, and track time to remediate as the key metric.

Likely follow-ups

- Why rescan images that already passed?
- How does VEX reduce scanner noise?

```
trivy image --severity HIGH,CRITICAL --exit-code 1 myapp:1.2
```

```
grype myapp:1.2 --fail-on high
```

```
# scan at build (CI) AND continuously (registry/runtime)
```

Scanners (Trivy, Grype, Clair) compare the packages in an image against CVE databases. Scan in CI to block vulnerable builds *and* continuously re-scan images already in your registry — new CVEs are discovered daily for images that haven't changed. Gotchas: scanners produce false positives (a CVE in a package you don't actually invoke) and false negatives (vendored deps, language libs they miss), and CVSS severity ≠ your real risk. Use VEX statements to suppress non-exploitable findings rather than ignoring scans.

Scan continuously, not just at build: An image built clean today can be "critical" tomorrow when a new CVE lands. A one-time CI scan gives false comfort.

certs.eknathlabs.com

📚 **Official docs**

- Trivy

Q20 What is dependency confusion and how do you defend against it?

Supply Chain

🎙 **Spoken answer · ~88 s**

Dependency confusion exploits how package managers resolve names when both internal and public registries are configured. If your company has an internal package called acme-utils at version 1.0 in a private registry, an attacker can publish a package with the same name to the public registry with a higher version number, like 99.0. If the build system checks the public registry too and picks the highest version, it installs the attacker's package, which can run code through install scripts. Alex Birsan used this in 2021 to compromise many large companies. Defences. Reserve your internal names on the public registries or use scoped, namespaced packages, such as @acme/utils on npm, where the scope is owned by your organisation. Configure package managers so that internal package names or scopes resolve only from the private registry: npm scope registry mappings, pip with a single index-url rather than extra-index-url, Maven repository ordering and exclusions, and Artifactory or Nexus virtual repositories with routing rules that block public lookup of internal names. Use lockfiles with integrity hashes, and pip's require-hashes, so unexpected packages fail. Route all dependency fetching through a proxy registry that you control, with allowlists. Disable install scripts where possible. Monitor for suspicious public packages matching your internal names, and review build logs for unexpected downloads.

Likely follow-ups

- Why does a higher version number win?
- How do scoped packages prevent this?

```
# attacker publishes a PUBLIC package matching your INTERNAL name
```

```
# at a higher version → resolver pulls the malicious public one
```

```
# defend: scoped names, explicit registry, namespace reservation
```

Dependency confusion exploits package managers that check public registries alongside private ones: an attacker publishes a malicious public package with the same name as your internal package but a higher version, and the resolver fetches theirs. Defenses: use scoped/namespaced names tied to your org, configure the package manager to pull internal packages *only* from your private registry (no public fallback for those names), reserve your namespaces on public registries, and pin/verify with lockfiles and checksums.

Higher version wins: Many resolvers pick the highest available version across all configured registries. Scoping and registry pinning break that attack path.

certs.eknathalabs.com

📚 **Official docs**

- OpenSSF Scorecard

Q21 Why are lockfiles and pinned dependencies a security control, not just reproducibility?

Supply Chain

🎙 **Spoken answer · ~81 s**

Lockfiles and pinned dependencies are security controls because they define exactly what code your build installs, making it deterministic and verifiable. Without them, a build may resolve version ranges to whatever is newest at that moment, so a malicious or compromised release, published minutes ago, gets pulled automatically into your next build, as in attacks on popular npm packages. A lockfile pins every direct and transitive dependency to specific versions and, in many ecosystems, records integrity hashes, so the package manager rejects content that differs from what was reviewed, protecting against tampering at the registry or in transit, and some dependency confusion attempts. Pinning gives you the chance to review updates deliberately: tools like Dependabot or Renovate open pull requests for upgrades that go through tests, scanning, and code review, rather than changing silently. It also supports reproducible builds and accurate SBOMs, and makes incident response easier because you know precisely which versions are deployed. Apply it everywhere: application dependencies, base images pinned by digest, CI actions pinned to commit SHAs, and Terraform provider versions locked. Use commands that honour the lockfile in CI, such as npm ci, and fail if it's out of sync.

Likely follow-ups

- Why pin CI actions to commit SHAs?
- How do you keep pinned dependencies patched?

```
# pin to exact versions + integrity hashes
```

```
package-lock.json / poetry.lock / go.sum   ← hash-verified
```

```
# unpinned: "requests>=2.0" → silently pulls a poisoned 2.99
```

Lockfiles record exact versions *and* cryptographic hashes of every dependency, so a build always pulls the precise, verified bytes you tested. Without pinning, a range like `>=2.0` means a compromised maintainer can publish a malicious patch release that silently enters your build. The integrity hashes also detect registry tampering — if the downloaded package doesn't match the recorded hash, the build fails. So lockfiles are both reproducibility and a tamper-detection control.

Hashes catch swaps: Even if an attacker replaces a package in the registry, the lockfile's hash mismatch stops the build cold.

certs.eknathalabs.com

📚 **Start here (official docs)**

- Kubernetes security checklist

Q22 What is provenance/attestation and how does it differ from an SBOM?

Supply Chain

🎙 **Spoken answer · ~78 s**

Provenance and attestations are related to, but different from, an SBOM. An SBOM answers 'what's inside this artefact?': an inventory of components and versions. Provenance answers 'how, where, and by whom was this artefact built?': it's a signed statement describing the build, including the source repository and commit, the builder platform and its identity, the build instructions or workflow, the inputs, and the output artefact digest. SLSA defines a standard provenance format. An attestation is the general mechanism: a signed, verifiable statement about an artefact, such as in the in-toto format, with a subject being the artefact digest and a predicate being the claim; provenance and SBOMs can both be delivered as attestations, as can vulnerability scan results or test results. The difference in use: SBOMs support vulnerability and licence management; provenance supports integrity and trust, letting a consumer verify that the artefact really came from the expected repository and pipeline, and was not built on a developer laptop or tampered with. In practice, you use both: the build produces the image, an SBOM, and a provenance attestation, all signed with Sigstore and stored in the registry.

Likely follow-ups

- What is an in-toto attestation?
- What would you check in provenance at deploy time?

```
SBOM       → WHAT is in the artifact (the ingredients)
```

```
Provenance → HOW/WHERE it was built (the recipe + kitchen)
```

```
# in-toto attestation: signed statement about the build
```

An SBOM describes *what* components an artifact contains; provenance (attestation) describes *how it was produced* — which source commit, which builder, what steps, signed so it can't be forged. They're complementary: the SBOM lets you answer "am I affected by CVE-X?", provenance lets you answer "was this really built from our trusted pipeline?" The in-toto framework standardizes signed attestations linking source → build → artifact.

Together they close the loop: SBOM (contents) + provenance (origin) + signature (integrity) = you can trust both what's inside and where it came from.

certs.eknathalabs.com

📚 **Official docs**

- GitHub: Artifact attestations (build provenance)
- SPDX specification
- CycloneDX

Q23 How do you secure your CI/CD pipeline against being the attack vector?

Supply Chain

🎙 **Spoken answer · ~84 s**

CI/CD pipelines are attractive targets because they hold powerful credentials and run code, so compromising one can poison every release. My defences. Access and credentials: use OIDC federation for short-lived cloud credentials instead of stored keys, scope secrets per environment, limit production deploy credentials to protected branches, and apply least privilege to pipeline service accounts and tokens. Pipeline integrity: treat pipeline definitions as code requiring review, restrict who can modify workflows and protected branches, and require approvals for production deployments. Third-party code: pin actions, plugins, and orbs to commit SHAs or verified versions, review them, and prefer vendored or internal ones; the Codecov incident shows that a compromised helper script exfiltrates secrets. Untrusted input: don't run untrusted pull request code with secrets, handle fork builds carefully, avoid injecting untrusted values like branch names or issue titles into shell commands, which causes script injection. Runner security: use ephemeral, isolated runners, avoid sharing self-hosted runners between trust levels, and restrict their network egress. Supply chain controls: build from locked dependencies, generate SBOMs and provenance, sign artefacts, and verify at deployment. Monitoring: audit logs of pipeline changes and secret use, alerts on anomalies, and regular reviews. Apply SLSA practices to harden the build platform itself.

Likely follow-ups

- What is script injection in a workflow?
- Why should runners be ephemeral?

```
✓ ephemeral, isolated build runners (no shared state)
```

```
✓ pin action/orb versions by SHA, not floating tags
```

```
✓ least-privilege tokens, OIDC not static keys
```

```
✓ no secrets to PRs from forks; require review for workflow changes
```

CI is high-value: it can read your secrets and push to production. Harden it by running builds in ephemeral isolated environments (a poisoned build can't persist), pinning third-party actions/plugins to immutable SHAs (a floating tag can be repointed to malicious code), using short-lived OIDC credentials instead of stored keys, scoping tokens to least privilege, and never exposing secrets to pull requests from forks. Treat the pipeline config itself as security-critical code requiring review.

certs.eknathalabs.com

Floating tags are dangerous: `uses: some/action@v3` can be silently repointed. Pin to a full commit SHA so the code you reviewed is the code that runs.

📚 **Official docs**

- GitHub Actions: Security hardening (pipeline as attack vector)

Q24 What is admission control for image provenance (e.g. only run signed images)?

Supply Chain

🎙 **Spoken answer · ~86 s**

Admission control for image provenance means the cluster refuses to run any image that doesn't meet your policy, enforcing the supply-chain guarantees at the last gate before runtime. In Kubernetes, an admission controller intercepts pod creation, and a validating policy checks each image. Tools: Kyverno with its verifyImages rule, Sigstore policy-controller, Connaisseur, Ratify with Gatekeeper, or Binary Authorization on GKE and similar managed policies on other clouds. A typical policy requires that the image is referenced by digest, comes from an allowed registry, has a valid cosign signature from a trusted identity, such as your CI pipeline's OIDC identity or a KMS key, and carries required attestations like provenance from the expected builder or a clean vulnerability scan. The controller verifies the signature against the public key or keyless identity, and rejects non-compliant pods with a clear message. Practices: start in audit mode to find violations before enforcing, exempt system namespaces carefully, handle images in init containers and ephemeral containers, cache verification results for performance, and make sure the admission webhook is highly available with a sensible failure policy, since failing closed can block the cluster and failing open weakens security. Use mutation to rewrite tags to digests. Provide a documented break-glass process, with audit logging.

Likely follow-ups

- Why start in audit mode?
- What is the trade-off of failing open versus closed?

certs.eknathalabs.com

```
# policy: reject any image lacking a valid cosign signature
```

```
# enforced by Kyverno / Sigstore policy-controller at admission
```

```
verifyImages: [{imageReferences: ["*"], attestors: [...]}]
```

Admission control enforces supply-chain policy at the moment a workload is created: an admission controller (Kyverno, Sigstore policy-controller, OPA Gatekeeper) verifies each image's signature and provenance before the pod is allowed to run, rejecting unsigned or unattested images. This makes "only run images our pipeline built and signed" a hard, cluster-enforced guarantee rather than a hopeful convention — even a compromised deployer can't run an untrusted image.

Shift the trust boundary into the cluster: Signing is useless if nothing checks signatures. Admission verification is what turns signing into actual protection.

📚 **Official docs**

- Google SRE Book: Handling Overload
- GitHub: Artifact attestations (build provenance)
- Sigstore documentation

Q25 How do you handle a critical CVE in a widely-used dependency (Log4Shell-style)? Supply Chain

🎙 **Spoken answer · ~86 s**

For a Log4Shell-style event, speed and structure matter. Triage: confirm the CVE's severity, exploitability, and whether it's being exploited in the wild, using advisories and threat intelligence. Identify exposure: use SBOMs and dependency scanners to find which applications, images, and hosts include the affected library, direct or transitive, including in vendor software and shaded jars; and determine which are internet-facing or process untrusted input, to prioritise. Mitigate immediately where patching takes time: apply vendor workarounds, such as disabling the vulnerable feature or setting a configuration flag, use WAF rules to block exploit patterns, restrict egress to stop callbacks, and add network segmentation. Detect: search logs and telemetry for exploitation attempts and indicators of compromise, and treat anything exposed during the window as potentially compromised. Patch: update the dependency, test, and roll out, starting with the most exposed systems, using automated pull requests and pipelines; rebuild images and redeploy rather than patching in place. Verify by rescanning. Communicate with stakeholders through a single incident channel with regular updates, and notify customers if affected. Afterwards, hold a retrospective: how fast could we answer which systems were affected? Improve with an SBOM inventory, faster dependency updates, and runtime protections. Follow-up for lingering vulnerable instances, which often remain in forgotten systems.

Likely follow-ups

- How does an SBOM speed up exposure analysis?
- What mitigations help before a patch is ready?

```
1. SCOPE with SBOM: which services/images contain it?
```

```
2. MITIGATE fast: WAF rule / config flag / network egress block
```

```
3. PATCH: bump version, rebuild, redeploy by priority
```

```
4. VERIFY: re-scan; confirm no lingering instances
```

certs.eknathalabs.com

The response order is scope → mitigate → patch → verify. First use your SBOM/scanners to find every affected artifact (you can't fix what you can't find). Apply a fast mitigation that buys time — a WAF rule, a feature flag, blocking the exploit's egress — while you prepare real patches. Then patch by exposure priority (internet-facing first), rebuild and redeploy, and re-scan to confirm nothing was missed. Communicate status throughout.

Mitigate before you patch: Patching everything takes hours-to-days; a WAF rule or egress block can neutralize active exploitation in minutes. Do both in parallel.

📚 **Official docs**

- Trivy

Q26 What are the risks of base images and how do you minimize them?

Supply Chain

🎙 **Spoken answer · ~90 s**

Base images are the foundation of every container, and they bring risks. They contain operating system packages and libraries with their own vulnerabilities, often hundreds in a full distribution image, most irrelevant to your application but still flagged and exploitable if reachable. They may be outdated, unmaintained, or come from untrusted publishers; public images can be compromised or contain malware or cryptominers. Tags are mutable: python:3.12 today differs from next week, so builds aren't reproducible and may silently change. They can include unnecessary tools such as shells, package managers, and compilers, giving attackers a toolkit after compromise, and may run as root by default. Mitigations. Choose minimal bases: slim variants, Alpine with awareness of musl compatibility, or distroless and Chainguard images, which have few packages and fewer CVEs. Use multi-stage builds so build tools stay out of the final image. Use only official or verified publishers, or build your own hardened golden base images that are centrally maintained, scanned, signed, and approved, and require teams to use them via policy. Pin by digest, and use automation to update the digest regularly. Rebuild often to pick up patches. Scan on build and continuously. Run as non-root, drop capabilities, and use read-only filesystems. Mirror approved bases in your own registry to avoid rate limits and upstream changes.

Likely follow-ups

- Why does a smaller base image reduce risk?
- What is a golden base image programme?

```
# fewer packages = smaller attack surface + fewer CVEs
```

```
FROM ubuntu:22.04        # ~hundreds of packages, many CVEs
```

```
FROM gcr.io/distroless/base   # no shell, no pkg manager
```

```
FROM scratch             # nothing but your static binary
```

cors.eknathalabs.com

Every package in a base image is potential vulnerability surface and something to patch. Minimize by using slim/distroless/scratch images: distroless has no shell or package manager (an attacker who breaks in has no tools), and `scratch` ships only your static binary. Pin base images by digest (not `:latest`), rebuild regularly to pick up upstream security fixes, and scan them. Smaller images also deploy faster and have a dramatically smaller CVE count to triage.

Distroless defeats post-exploitation: No `sh`, `curl`, or `apt` means a compromised process can't pivot, download tools, or open a reverse shell easily.

📚 **Official docs**

- Distroless images

Q27 What is a transparency log (Rekor) and why is it important for signing?

Supply Chain

🎙 **Spoken answer · ~85 s**

A transparency log is a public, append-only, tamper-evident record. Rekor is Sigstore's transparency log for software supply chain metadata. Every time an artefact is signed, an entry with the signature, the certificate or public key, and the artefact digest is added to Rekor, which returns a signed timestamp and an inclusion proof; the log is based on a Merkle tree, so entries cannot be altered or removed without detection, and anyone can audit it. Why it matters. Keyless signing uses certificates valid for only minutes, so how do you verify later, after they've expired? The log's entry proves the signature was created during the certificate's validity period, so verification can succeed. Without it, short-lived certificates wouldn't be usable. Accountability and detection: because all signing events are public, a compromised identity that signs malicious artefacts leaves a record; maintainers and organisations can monitor the log for unexpected signatures made with their identity and detect compromise early. Non-repudiation: signers can't deny signing. Trust minimisation: it limits the damage of a compromised CA or key, since fraudulent signatures would need to appear in the public log. This is the same concept as Certificate Transparency for TLS. For private use, you can run a private Rekor instance.

Likely follow-ups

- Why do short-lived certificates need a transparency log?
- How can an organisation monitor the log?

```
# every signature recorded in a tamper-evident public log
```

```
rekor-cli search --artifact app   # find all signing events
```

```
# append-only, cryptographically verifiable (Merkle tree)
```

Rekor is an append-only, tamper-evident public ledger of signing events. When you sign with cosign, the signature and signer identity are recorded there, so anyone can later verify a signature existed at a point in time and detect if records were altered (it's a Merkle-tree structure). This solves keyless signing's trust

certs.eknathalabs.com

problem: even with short-lived certificates, the transparency log provides durable, auditable proof of who signed what and when — and lets you detect unexpected signing events that might indicate key/identity compromise.

Detect rogue signing: Monitoring the log for signatures under your identity that your pipeline didn't produce is an early warning of compromise.

📚 **Official docs**

- Sigstore documentation

Q28 How do you vet and govern third-party / open-source dependencies?

Supply Chain

🎙 **Spoken answer · ~87 s**

Governing third-party and open-source dependencies means deciding what you let into your software and keeping it healthy over time. Vetting before adoption: assess necessity, as the best dependency is the one you avoid; check the project's health, such as active maintenance, number of maintainers, release cadence, responsiveness to security issues, and community size; review its security posture using OpenSSF Scorecard, which checks practices like code review, branch protection, and signed releases; check known vulnerabilities and history, the licence compatibility with your use, and the size of its transitive tree. Look for red flags such as typosquatting names, recent ownership transfers, and install scripts. Governance: route all fetching through an internal proxy registry such as Artifactory or Nexus, where you can allowlist approved packages, block known-bad ones, and cache for availability. Maintain an approved list for sensitive areas and a lightweight review process for new additions. Enforce with lockfiles, pinned versions, and integrity hashes. Ongoing: generate SBOMs, scan continuously for vulnerabilities and licences, automate updates with Dependabot or Renovate, and track end-of-life components. Define SLAs for remediating vulnerabilities by severity. Monitor for abandonment and consider contributing to or forking critical dependencies. Assign ownership so someone is responsible for each. Balance control with developer productivity, so the process doesn't encourage workarounds.

Likely follow-ups

- What does OpenSSF Scorecard measure?
- Why route dependencies through a proxy registry?

```
✓ automated: Dependabot/Renovate, OSV scanning, license checks
```

```
✓ health signals: maintenance, maintainers, OpenSSF Scorecard
```

```
✓ policy: allowlist registries, block known-bad, review new deps
```

cors.eknathalabs.com

Governance combines automation and judgment. Automate vulnerability scanning (OSV, Dependabot), dependency updates, and license compliance in CI. Assess dependency health before adopting — is it maintained, how many maintainers, what's its OpenSSF Scorecard? Set policy: pull only from approved registries, block known-malicious packages, and require review when adding new dependencies (especially ones with few maintainers or recent ownership changes, the xz attack pattern). The goal is reducing how much unvetted code you implicitly trust.

Fewer deps = less risk: The most secure dependency is the one you don't add. Question whether a 5-line utility is worth a new transitive tree.

📚 **Official docs**

- OpenSSF Scorecard

Q29 What is VEX and how does it improve vulnerability management?

Supply Chain

🎙 **Spoken answer · ~90 s**

VEX, the Vulnerability Exploitability eXchange, is a machine-readable statement from a software supplier about whether specific vulnerabilities actually affect a product. A scanner reporting a CVE in a component only tells you the vulnerable code is present; many such findings aren't exploitable because the vulnerable function is never called, the condition can't occur in this configuration, or it's mitigated elsewhere. A VEX document states the status of each vulnerability for a product: not affected, with a justification such as vulnerable code not present or not in execute path; affected, with recommended remediation; fixed; or under investigation. Formats include CSAF, OpenVEX, and CycloneDX VEX. Benefits. It cuts noise: consumers can automatically suppress findings marked not affected, so teams focus on real risk. It improves communication between suppliers and customers: instead of answering questionnaires after each new CVE, the supplier publishes VEX alongside the SBOM. It supports triage at scale, speeds up response, and adds context to compliance. Practices: produce VEX statements as part of the release process, informed by reachability analysis and engineering review; sign them as attestations so they can be verified; keep them updated as understanding changes; and feed them into scanners like Grype and Trivy that can consume VEX. Be careful: a wrong not-affected claim creates false assurance, so require evidence and review.

Likely follow-ups

- What statuses can a VEX statement have?
- What risk does an incorrect not-affected claim carry?

```
VEX = Vulnerability Exploitability eXchange
```

```
# a signed statement: "CVE-X is present BUT not exploitable here"
```

```
status: not_affected   justification: vulnerable_code_not_in_execute_path
```

ceris.eknathalabs.com

Scanners flag every CVE in every included package, drowning teams in findings — most of which aren't actually exploitable in your context (the vulnerable function is never called, the component isn't reachable). VEX is a machine-readable, often signed statement asserting whether a given CVE actually affects your product and why. It lets you suppress non-exploitable findings *with documented justification* instead of blanket-ignoring scans, so real risks stay visible and auditors see your reasoning.

Signal over noise: VEX turns "5,000 CVEs" into "the 12 that actually matter," with an audit trail explaining the other 4,988.

📚 **Official docs**

- Trivy

Q30 How do you implement a secure build pipeline end-to-end (source to deploy)?

Supply Chain

🎙 **Spoken answer · ~89 s**

An end-to-end secure pipeline protects each stage from source to deployment. Source: protected branches, mandatory code review, signed commits, secret scanning with push protection, and dependency review. Build: run on ephemeral, isolated, hosted runners with least-privilege credentials via OIDC; build from locked, pinned dependencies fetched through a controlled proxy registry; use minimal, pinned base images; run tests, SAST, and dependency and licence checks; scan IaC and Dockerfiles. Artefact: build the image once, generate an SBOM and a provenance attestation at SLSA level 2 or 3, scan for vulnerabilities with a gate, sign the image digest with cosign, and push to a private registry with immutable tags and access controls. Deploy: promote the same digest through environments, using GitOps with reviewed changes; admission control verifies signatures and attestations and enforces policies on pods, like non-root and no privileged containers; secrets come from a manager at runtime. Runtime: network policies, workload identity, runtime detection with Falco, and continuous rescanning. Cross-cutting: audit logging across the pipeline, monitoring and alerting, an incident response plan, protecting pipeline configuration as code, and regularly reviewing and testing the controls, for instance with red-team exercises. Measure with metrics such as time to patch and percentage of signed deployments. The principle is to verify at each handoff, so no single compromise defeats everything.

Likely follow-ups

- Where does admission control fit in the pipeline?
- Why verify at each handoff?

```
source: signed commits, branch protection, code review
```

```
build:  isolated runner, pinned deps, SBOM + provenance generated
```

```
scan:   SAST + dependency + secret + image scan, gate on severity
```

coris.eknathalabs.com

```
sign:   cosign sign image + attestations
```

```
deploy: admission controller verifies signature/provenance
```

A secure pipeline chains trust from commit to runtime: enforce signed commits and reviewed PRs at the source; build in isolated runners with pinned dependencies, emitting an SBOM and signed provenance; run layered scanning (SAST, dependency, secret, image) with severity gates; sign the artifact and attestations with cosign; and enforce at deploy time via an admission controller that only admits signed, attested images. Each stage produces verifiable evidence the next stage checks — no single compromised step silently ships untrusted code.

Unbroken chain of custody: The power is in the linkage — every artifact traces back to a reviewed commit and a trusted build, cryptographically verified before it runs.

📚 **Official docs**

- GitHub: Artifact attestations (build provenance)

Q31 Why should containers run as non-root, and how do you enforce it?

K8s / Container

🎙 **Spoken answer · ~88 s**

Containers run as root by default, and root inside a container is UID 0 on the host kernel. Although namespaces provide isolation, a vulnerability, misconfiguration, or added capability can let a root process break out, and then it has root on the host. Even without escape, root in the container can modify files on mounted volumes and host paths, bind privileged ports, install packages, and use more kernel attack surface. Running as a non-root user limits the damage: a compromised process can't change the system files, can't use many privileged operations, and escape exploits that require root become harder. Enforcement. In the Dockerfile, create a user with a fixed UID and set USER. In Kubernetes, set securityContext with runAsNonRoot true, runAsUser to a non-zero UID, and runAsGroup and fsGroup for volume permissions, allowPrivilegeEscalation false, drop all capabilities, and a read-only root filesystem. Enforce cluster-wide with Pod Security Admission at the restricted level, or policies via Kyverno or OPA Gatekeeper that reject pods running as root. Applications need to be written to avoid privileged ports, listening on 8080 rather than 80, and to write only to designated writable directories or volumes. For additional isolation, user namespaces map container root to an unprivileged host user. Scan images and manifests in CI for root usage.

Likely follow-ups

- What does runAsNonRoot actually enforce?
- How do user namespaces help even for root processes?

```
securityContext:
```

```
  runAsNonRoot: true
```

cors.eknathalabs.com

```
  runAsUser: 10001
```

```
  allowPrivilegeEscalation: false
```

```
# enforce cluster-wide via Pod Security Admission "restricted"
```

A container running as root that gets compromised gives the attacker root in the container — and combined with a kernel or runtime escape, potentially root on the node. Running as an unprivileged UID limits what a breakout can do. Enforce it in the pod's `securityContext` (`runAsNonRoot`, a high UID, no privilege escalation), build images with a non-root `USER`, and apply Pod Security Admission "restricted" cluster-wide so non-compliant pods are rejected at admission.

Defense in depth: Non-root alone isn't a sandbox — pair it with dropped capabilities, read-only rootfs, and seccomp. Each layer makes a breakout harder.

📚 **Official docs**

- Kubernetes: Security context

Q32 What are Linux capabilities and which should containers drop?

K8s / Container

🎙 **Spoken answer · ~81 s**

Linux capabilities split the all-powerful root privilege into distinct units, such as NET_BIND_SERVICE for binding low ports, CHOWN for changing file ownership, SYS_ADMIN for a huge range of administrative operations, NET_RAW for raw sockets, and SYS_PTRACE for tracing processes. Container runtimes give containers a default subset of capabilities, which is still more than most applications need. The security best practice is to drop all capabilities and add back only the specific ones required. In Kubernetes, securityContext capabilities drop ALL, then add, for example, NET_BIND_SERVICE if the app must bind port 80. Which to definitely avoid: SYS_ADMIN, which is nearly equivalent to root and enables many container escapes; NET_ADMIN, which allows network configuration changes, and can enable ARP spoofing or traffic redirection; SYS_PTRACE, which can inspect other processes and is used in escapes; SYS_MODULE, loading kernel modules; DAC_OVERRIDE and DAC_READ_SEARCH, bypassing file permission checks; NET_RAW, enabling packet spoofing, which the restricted Pod Security profile drops; SETUID and SETGID, and MKNOD. Combine with non-root users, no privilege escalation, seccomp, and AppArmor or SELinux. Enforce through Pod Security Admission or policy engines, audit with scanners, and test applications with minimal capabilities by running them under dropped sets.

Likely follow-ups

- Why is SYS_ADMIN so dangerous?
- How do you find the minimal capabilities an app needs?

core.eknathalabs.com

```
securityContext:
  capabilities:
    drop: ["ALL"]
    add: ["NET_BIND_SERVICE"]   # only if binding <1024
```

Capabilities split root's monolithic power into granular units (e.g. `NET_ADMIN`, `SYS_ADMIN`, `NET_BIND_SERVICE`). Containers get a default set they rarely all need. Best practice: drop `ALL` and add back only the specific capabilities the workload requires. Especially avoid `SYS_ADMIN` (nearly equivalent to root) and `SYS_PTRACE`/`NET_RAW` unless essential. This shrinks what a compromised process can do even if it runs as UID 0.

Drop ALL, add back minimally: Most apps need zero capabilities. Starting from nothing and justifying each addition is the secure default.

📚 **Official docs**

- Kubernetes: Security context
- Linux man-pages: capabilities(7)

certs.eknathalabs.com

Q33 What is a read-only root filesystem and why use it?

K8s / Container

🎙 Spoken answer · ~87 s

A read-only root filesystem mounts the container's filesystem as read-only, so processes cannot write to it. In Kubernetes you set readOnlyRootFilesystem true in the container's securityContext, and in Docker use the --read-only flag. Why it helps: many attacks rely on writing to disk after compromise, such as dropping malware or backdoors, modifying binaries or configuration, installing tools, writing web shells, or tampering with libraries to persist. With a read-only root, those become impossible or much harder, so an attacker with code execution has limited ability to establish persistence. It also enforces immutability, since the running container always matches the image, which supports reproducibility and makes drift detection trivial, and it encourages cleaner design: applications must explicitly declare where they need to write. Practical implementation: identify which paths the application legitimately writes to, such as /tmp, caches, and PID files, and mount emptyDir volumes, optionally memory-backed, at those specific locations; use persistent volumes for real data; configure logging to stdout instead of files. Some applications need adjustments or configuration to avoid writing in unexpected places, so test thoroughly. It's one layer; combine it with non-root users, dropped capabilities, and no privilege escalation. The restricted Pod Security profile doesn't require it, so enforce it with policy engines such as Kyverno.

Likely follow-ups

- Where does an app write if the root filesystem is read-only?
- What attacks does this make harder?

```
securityContext:
```

```
  readOnlyRootFilesystem: true
```

```
# mount writable emptyDir only where needed:
```

```
volumeMounts: [{name: tmp, mountPath: /tmp}]
```

A read-only root filesystem prevents an attacker from writing malware, modifying binaries, or dropping a web shell into the container — a huge chunk of post-exploitation requires writing files. Mount specific writable volumes (emptyDir) only where the app genuinely needs to write (e.g. `/tmp`, a cache dir). This also catches apps that write where they shouldn't, surfacing hidden state. It's one of the highest-value, lowest-effort container hardening controls.

Immutability as security: If the filesystem can't change, the running container can't be persistently tampered with — a restart wipes any in-memory foothold.

📚 Official docs

- Kubernetes: Security context (read-only root filesystem)

certs.eknathalabs.com

Q34 How do Kubernetes NetworkPolicies provide zero-trust networking?

K8s / Container

🎙 Spoken answer · ~90 s

By default, every pod in a Kubernetes cluster can talk to every other pod, so a single compromised pod can reach databases, internal APIs, and other namespaces, enabling lateral movement. NetworkPolicies are namespaced resources that select pods and define allowed ingress and egress traffic by pod labels, namespace labels, IP blocks, and ports. They are enforced by the CNI plugin, such as Calico, Cilium, or Antrea, and are ineffective without one that supports them. Zero-trust means no implicit trust based on network location; every connection must be explicitly allowed. The approach: apply a default-deny policy in every namespace for both ingress and egress, then add narrowly scoped allow rules for each legitimate flow: frontend to API, API to database on a specific port, all pods to DNS, and ingress controller to the services it exposes. Restrict egress to required destinations to hinder data exfiltration and command-and-control callbacks, and block the cloud metadata endpoint. Use namespace selectors to isolate tenants and environments. Operational tips: label workloads consistently, store policies in Git, build them from observed traffic with tools like Hubble or flow logs before enforcing, and test them with connectivity checks. Standard policies operate at layers 3 and 4 only; for identity-based, encrypted, and layer 7 control, add a service mesh with mTLS or Cilium policies.

Likely follow-ups

- Why does an egress default-deny matter?
- What does a NetworkPolicy need in order to work?

```
# default-deny all ingress in a namespace, then allow explicitly
```

```
kind: NetworkPolicy
```

```
spec:
```

```
  podSelector: {}        # all pods
```

```
  policyTypes: [Ingress]  # with no rules = deny all ingress
```

By default, every pod can talk to every other pod — flat, trusting networking. NetworkPolicies let you enforce least-privilege segmentation: start with a default-deny policy, then explicitly allow only the flows you need (frontend→api, api→db). This contains lateral movement — a compromised pod can't freely scan and pivot across the cluster. Note they require a CNI that enforces them (Calico, Cilium); without an enforcing CNI the policies are silently ignored.

Default-deny first: Allow-lists only work if the baseline is deny. Without a default-deny policy, your "allow" rules add nothing — everything is already permitted.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: Network Policies
- NIST Zero Trust Architecture (SP 800-207)

Q35 What is RBAC in Kubernetes and what are common misconfigurations?

K8s / Container

🎙 **Spoken answer · ~86 s**

Kubernetes RBAC controls who can do what to which API resources. Roles and ClusterRoles define sets of permissions as verbs on resources; RoleBindings and ClusterRoleBindings grant those to users, groups, or service accounts, within a namespace or cluster-wide. Common misconfigurations. Granting cluster-admin broadly, or to service accounts and CI systems; wildcards in verbs or resources, such as resources star and verbs star. Giving access to secrets with get, list, or watch, where list on secrets reveals all values. Permissions that allow privilege escalation: create on pods lets someone run a pod with a powerful service account or privileged settings; rights to create rolebindings, bind, escalate, or impersonate let a user grant themselves more; access to pods/exec reaches inside workloads; node proxy access. ClusterRoleBindings where a namespaced RoleBinding would do. Using the default service account with added permissions, shared across workloads. Binding to the system:authenticated or system:unauthenticated groups. Stale bindings for departed users. Not auditing effective permissions. Good practice: least privilege with specific verbs and resource names; namespaced roles; a dedicated service account per workload with minimal rights; group-based human access tied to the identity provider with short-lived credentials; regular audits using kubectl auth can-i, rbac-tool, or KubiScan; avoid wildcard use; and monitor audit logs for sensitive actions.

Likely follow-ups

- Which RBAC permissions enable privilege escalation?
- Why is list on secrets dangerous?

```
# dangerous grants to audit for:
```

```
cluster-admin bound to a service account
```

```
verbs: ["*"] on resources: ["*"]
```

```
create/patch on pods + access to a privileged SA = escalation
```

RBAC controls who can do what to which resources. Common misconfigurations: binding `cluster-admin` to service accounts or users who don't need it, wildcard verbs/resources, and subtle escalation paths — e.g. the ability to create pods plus access to a privileged service account token lets a user escalate. Audit with `kubectl auth can-i` and tools like `kubectl-who-can`/rbac-tool. Apply least privilege: scope roles to namespaces and specific verbs, and avoid mounting SA tokens where not needed.

certs.eknathalabs.com

Escalation via creation: `create pod` + a powerful SA in the namespace = effective privilege escalation. RBAC review must consider these chains, not just direct grants.

📚 **Official docs**

- Kubernetes: RBAC

Q36 Explain Pod Security Admission and its three levels.

K8s / Container

🎙 **Spoken answer · ~86 s**

Pod Security Admission is the built-in Kubernetes admission controller, stable since version 1.25, that enforces the Pod Security Standards, replacing the deprecated PodSecurityPolicy. It applies at the namespace level through labels. The three standard levels. Privileged: unrestricted, allowing everything, including privileged containers and host namespaces; intended for system and infrastructure workloads such as CNI plugins. Baseline: a minimally restrictive policy that prevents known privilege escalations; it blocks privileged containers, host namespaces for network, PID, and IPC, hostPath volumes, hostPorts, adding dangerous capabilities, and unsafe sysctls, while allowing typical applications with defaults. Restricted: follows current hardening best practices; in addition to the baseline it requires running as non-root, disallowing privilege escalation, dropping all capabilities, with only NET_BIND_SERVICE allowed to be added, using a seccomp profile of RuntimeDefault or Localhost, and restricting volume types. Three modes per namespace: enforce rejects violating pods, audit allows them but records an audit annotation, and warn returns a warning to the user. Adoption approach: label namespaces in warn and audit mode first at the target level, review the violations, fix workloads, then enforce; set cluster-wide defaults with the admission configuration; exempt only necessary namespaces or users. PSA is coarse, with no custom rules, so for finer policies use Kyverno or Gatekeeper alongside it.

Likely follow-ups

- What do the enforce, audit, and warn modes do?
- Why adopt PSA with warn mode first?

certs.eknathalabs.com

```
# namespace label enforces a profile:
```

```
pod-security.kubernetes.io/enforce: restricted
```

```
privileged  → unrestricted (no controls)
```

```
baseline    → blocks known privilege escalations
```

```
restricted  → hardened: non-root, drop caps, seccomp, etc.
```

Pod Security Admission (PSA) is the built-in replacement for PodSecurityPolicies. It applies one of three profiles per namespace via labels: **privileged** (anything goes — for system/infra namespaces only), **baseline** (blocks the most dangerous settings like host namespaces and privileged containers), and **restricted** (the

hardened best-practice baseline: non-root, no privilege escalation, dropped capabilities, seccomp). You can set `enforce`, `audit`, and `warn` independently to roll it out gradually.

Roll out with warn/audit first: Set `warn` and `audit` to "restricted" before `enforce` — you'll see which existing workloads would break without actually blocking them.

📚 **Official docs**

- Kubernetes: Pod Security Standards

Q37 What is a container escape and how do you reduce the risk?

K8s / Container

🎙 **Spoken answer · ~84 s**

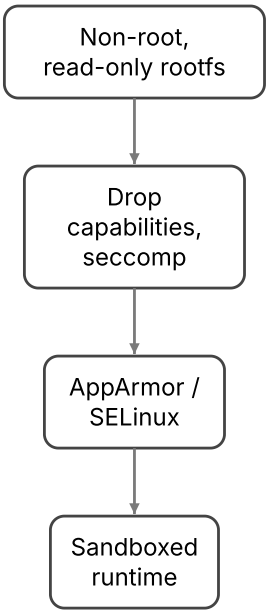
A container escape is when a process inside a container breaks isolation and gains access to the host or other containers, usually reaching root on the node and then the rest of the cluster. Routes: kernel vulnerabilities, since containers share the host kernel, such as Dirty COW and Dirty Pipe; container runtime flaws like the runc vulnerability CVE-2019-5736 and CVE-2024-21626; misconfigurations such as privileged containers, dangerous capabilities like SYS_ADMIN, mounting the Docker socket or sensitive host paths including /, /proc, or /var/run, hostPID or hostNetwork namespaces, writable cgroup release agents, and excessive permissions to the container runtime or kubelet API. Reducing risk follows defence in depth. Configuration: never run privileged, drop all capabilities, run as non-root with no privilege escalation, a read-only root filesystem, and avoid hostPath and host namespaces; enforce this with Pod Security Admission at restricted level or policy engines. Isolation layers: seccomp profiles to reduce accessible syscalls, AppArmor or SELinux, and user namespaces. Stronger sandboxes for untrusted workloads: gVisor or Kata Containers, or dedicated nodes and VMs for multi-tenant isolation. Patching: keep the kernel, container runtime, and node images updated, and use hardened, minimal host operating systems like Bottlerocket or Container-Optimized OS. Detect with runtime tools such as Falco.

Likely follow-ups

- Which misconfigurations most often enable escapes?
- When would you use gVisor or Kata?

certs.eknathalabs.com

🗺 **Container escape defenses**




```
# escape vectors to lock down:
```

```
privileged: true, hostPID/hostNetwork/hostIPC, hostPath mounts
```

```
docker.sock mounted in a pod, SYS_ADMIN cap, kernel CVEs
```

A container escape is when a process breaks out of its isolation to the host. Containers share the host kernel, so a kernel vulnerability or an over-privileged config can bridge the boundary. Reduce risk by never running privileged containers, not mounting the Docker socket or sensitive host paths, dropping capabilities, applying seccomp/AppArmor, keeping nodes patched, and — for hostile multi-tenant workloads — using stronger isolation like gVisor or Kata Containers (microVMs) that don't share the host kernel directly.

Mounting docker.sock = game over: A pod with the host's Docker socket can launch a privileged container and own the node. Never do it; use proper APIs instead.

📚 **Official docs**

- Docker: Engine security

certs.eknathalabs.com

Q38 What is seccomp and how do you apply a profile in Kubernetes?

K8s / Container

🎙 Spoken answer · ~88 s

Seccomp, secure computing mode, is a Linux kernel feature that filters the system calls a process can make. The kernel exposes hundreds of syscalls, and every one is potential attack surface for kernel vulnerabilities; most applications use a small fraction. A seccomp profile lists allowed syscalls, and anything else is blocked, returning an error or killing the process. This shrinks the kernel attack surface available to a compromised container and can block exploit techniques. Container runtimes ship a default profile that blocks around forty-odd dangerous syscalls, but Kubernetes historically ran pods as Unconfined unless you opted in. To apply a profile in Kubernetes, set securityContext seccompProfile type RuntimeDefault, either per pod or container, to use the runtime's default profile, which is the recommended baseline and required by the restricted Pod Security level. You can also set the kubelet's SeccompDefault feature to make RuntimeDefault the default for all pods. For stricter control, use type Localhost with a custom JSON profile stored on the node under the kubelet's seccomp directory, referenced by localhostProfile. Build custom profiles by recording the syscalls an application uses, via tools like the Security Profiles Operator, which can record and distribute profiles, or Inspektor Gadget, then test thoroughly because a missing syscall causes failures. Start with audit-like logging, then enforce.

Likely follow-ups

- What does RuntimeDefault give you?
- How would you build a custom profile?

```
securityContext:
```

```
  seccompProfile:
```

```
    type: RuntimeDefault   # blocks ~dangerous syscalls
```

```
# or type: Localhost, localhostProfile: my-profile.json
```

Seccomp (secure computing mode) filters which Linux syscalls a process may make, drastically shrinking the kernel attack surface available to a compromised container. The easy win is `RuntimeDefault`, which applies the container runtime's curated profile blocking dozens of dangerous/obscure syscalls most apps never use. For tighter control, generate a custom profile (e.g. with the Security Profiles Operator) allowing only the syscalls your app actually makes. Many kernel-exploit CVEs require syscalls that `RuntimeDefault` already blocks.

RuntimeDefault is nearly free: It blocks rarely-used syscalls (often the ones exploits need) with almost no compatibility risk — enable it everywhere.

📚 Official docs

- Kubernetes: Seccomp tutorial

certs.eknathalabs.com

Q39 How do you secure the Kubernetes API server and etcd?

K8s / Container

🎙 Spoken answer · ~90 s

The API server is the front door to the cluster, and etcd holds all cluster state, including Secrets, so both are critical. API server hardening: don't expose it to the internet; use private endpoints or restrict access to authorised networks and VPN; require TLS; disable anonymous authentication and the insecure port; use strong authentication, such as OIDC integration with your identity provider with short-lived tokens and MFA, instead of static tokens or shared client certificates; enforce RBAC with least privilege and enable the Node and RBAC authorisers; turn on admission controllers including Pod Security Admission and policy engines; enable audit logging with a sensible policy, shipped to a secure location; and set request limits and API priority and fairness against abuse. Restrict kubelet access with authentication and authorisation, not anonymous. Managed services handle some of this, but you still configure access. etcd hardening: allow only the API server to connect, with mutual TLS for client and peer communication; isolate on private networks with firewall rules; enable encryption at rest for Secrets with a KMS provider; encrypt and restrict access to backups; run on dedicated, hardened nodes; and monitor and alert on access. Keep versions patched, rotate certificates, and regularly benchmark with kube-bench against the CIS Kubernetes benchmark. Protect the control plane's credentials and admin kubeconfigs.

Likely follow-ups

- Why must etcd be isolated from workloads?
- What does enabling audit logging give you?

```
✓ etcd: encryption at rest (KMS), TLS, restricted to control plane
```

```
✓ API: strong authn (OIDC), RBAC, audit logging on
```

```
✓ disable anonymous auth, restrict who reaches the API endpoint
```

The API server is the cluster's front door and etcd holds all state (including Secrets). Harden etcd with TLS for client and peer traffic, encryption at rest (KMS-backed `EncryptionConfiguration`), and network isolation so only the control plane reaches it. For the API server: enforce strong authentication (OIDC, no anonymous access), least-privilege RBAC, comprehensive audit logging, and restrict network exposure (private endpoint, allowlisted CIDRs). Anyone who reaches etcd directly bypasses all RBAC — it's the ultimate prize.

etcd = everything: Direct etcd access means reading every secret and rewriting any object, bypassing RBAC entirely. Protect it like the keys to the kingdom.

📚 Official docs

- Kubernetes: Operating etcd clusters
- etcd documentation

certs.eknathalabs.com

Q40 What is runtime security and how do tools like Falco work?

K8s / Container

🎙 Spoken answer · ~87 s

Runtime security detects and responds to malicious or anomalous behaviour in running workloads, complementing build-time scanning, which can't catch zero-days, stolen credentials, or misconfigurations exploited after deployment. Falco, a CNCF project, is the best-known tool. It observes system calls and events from the Linux kernel, using an eBPF probe or kernel module, and enriches them with container and Kubernetes metadata. A rules engine evaluates events against rules written in YAML with conditions and outputs, with macros and lists for reuse; when a rule matches, Falco emits an alert. Examples of default rules: a shell spawned in a container, a sensitive file such as /etc/shadow read, a write below /etc or binary directories, an unexpected outbound network connection, a privileged container started, a process modifying its own binaries, the Docker socket mounted, or cryptominer indicators. Alerts go to stdout, Slack, SIEMs, or via Falcosidekick to many outputs, and can trigger automated response, such as killing a pod or applying a network policy through Falco Talon or a custom controller. Practicalities: tune the rules to your workloads to cut false positives, test in staging, manage rules as code, monitor the overhead, and make sure someone is on-call to respond to the alerts. Alternatives include Tetragon, Tracee, and commercial CNAPP agents.

Likely follow-ups

- How does Falco observe container activity?
- Why tune runtime rules to your workloads?

```
# Falco watches syscalls/eBPF for suspicious behavior:
```

```
- rule: Terminal shell in container
```

```
  condition: spawned_process and shell_procs and container
```

```
  output: "Shell spawned in %container.name"
```

Pre-deploy scanning catches known issues; runtime security catches what's happening *now*. Falco taps syscalls (via eBPF or a kernel module) and evaluates them against rules to detect suspicious behavior — a shell spawned in a container, an unexpected outbound connection, writes to sensitive paths, a process reading `/etc/shadow`. It alerts in real time so you can respond to active compromise that static scanning would never see (zero-days, misuse of legitimate tools, insider actions).

Detect, don't just prevent: No amount of scanning catches a zero-day or stolen-credential abuse. Runtime detection is your tripwire for the attacks you didn't anticipate.

📚 Official docs

- Falco documentation

certs.eknathalabs.com

Q41 How do you prevent a compromised pod from accessing the cloud metadata endpoint?

K8s / Container

🎙 Spoken answer · ~86 s

The cloud metadata endpoint, 169.254.169.254, gives instances information and, critically, temporary credentials for the node's IAM role. A compromised pod, or an SSRF vulnerability in an application, can query it and steal those credentials, gaining the node's cloud permissions; this was the route in the 2019 Capital One breach. Mitigations in layers. Don't depend on node credentials: use workload identity (IRSA, EKS Pod Identity, GKE Workload Identity), and give nodes minimal permissions. Block pod access to the endpoint: a NetworkPolicy egress rule denying 169.254.169.254, applied by default to all application namespaces, while allowing system components that need it; or cloud-specific controls. On AWS, require IMDSv2, which needs a session token obtained by a PUT request and defeats basic SSRF, and set the hop limit to one so containers on the node can't reach it through the extra network hop. On GKE, Workload Identity with the GKE metadata server intercepts metadata calls from pods and doesn't expose node credentials; also block legacy metadata endpoints. On Azure, restrict the Instance Metadata Service similarly or use workload identity. Protect applications against SSRF by validating outbound URLs, restricting egress, and blocking link-local addresses. Monitor audit logs for credentials used from unexpected locations, and test the controls from inside a pod.

Likely follow-ups

- Why does IMDSv2 defeat simple SSRF?
- What does the hop limit of one achieve?

```
# block the link-local metadata IP from pods
```

```
NetworkPolicy / Cilium: deny egress to 169.254.169.254
```

```
# + enforce IMDSv2 (AWS) and use workload identity instead
```

The cloud metadata endpoint (`169.254.169.254`) hands out the node's IAM credentials — a classic SSRF/escape target. A compromised pod that reaches it can steal the node role's permissions. Block egress to the metadata IP from workload pods via NetworkPolicy/Cilium, enforce IMDSv2 (which requires a session token and a hop-limit that defeats simple SSRF) on AWS, and give pods their own scoped identity via IRSA/Workload Identity so they never need the node's broad role.

SSRF → cloud takeover: Many cloud breaches chain "app SSRF" → "read metadata" → "assume node role." Blocking metadata access from pods breaks that chain.

📚 Official docs

- NIST: Computer Security Incident Handling Guide (SP 800-61)

certs.eknathalabs.com

Q42 What's the risk of the default ServiceAccount token and how do you mitigate it?

K8s / Container

🎙 Spoken answer · ~83 s

By default, Kubernetes mounts a service account token into every pod, in the default service account of its namespace, even when the application never talks to the API. That token is a credential for the API server: if an attacker compromises the pod, through remote code execution or SSRF, they can read the token from the filesystem and call the API with whatever permissions the service account holds. If the default account has been given broad RBAC rights, which happens when teams add bindings to it for convenience, the compromise could escalate to the whole namespace or cluster. Older tokens were also long-lived and not bound to the pod. Mitigations. Disable automatic mounting where the pod doesn't need API access: set automountServiceAccountToken false on the service account or on the pod spec. Create a dedicated service account for each workload and grant it only the exact RBAC permissions it needs, and never bind rights to the default account. Use the bound service account tokens that modern Kubernetes issues via the projected volume: they're short-lived, audience-bound, and invalidated when the pod is deleted. Restrict with network policies and egress controls so pods cannot reach the API server unless needed.

Likely follow-ups

- How do bound tokens differ from legacy ones?
- When should you disable automounting?

```
# stop auto-mounting the token where the app doesn’t need it
```

```
automountServiceAccountToken: false
```

```
# set on the SA or per-pod; use dedicated least-priv SAs
```

By default every pod mounts its ServiceAccount token at a known path, and many apps don’t need to talk to the API server at all. A compromised pod with a mounted token can use it to hit the API with whatever RBAC that SA has — and the default SA is often more privileged than realized. Mitigate by setting `automountServiceAccountToken: false` where the API isn’t needed, creating dedicated minimal-privilege SAs per workload, and using short-lived projected tokens with an audience.

No token, no API foothold: If a workload never calls the API, don’t mount its token. You’ve removed an entire post-exploitation avenue for free.

📚 Official docs

- Kubernetes: Service accounts

certs.eknathalabs.com

Q43 How do image pull policies and registry controls affect security?

K8s / Container

🎙 Spoken answer · ~88 s

Image pull policy and registry controls determine which image code the cluster actually runs, so they affect integrity and trust. imagePullPolicy values: Always checks the registry on every pod start and pulls if the digest differs; IfNotPresent uses a locally cached image if present; Never uses only local images. Security implications: with IfNotPresent and mutable tags like latest, a node may run a stale image or, importantly, a different image than expected, since the cache is keyed by name and tag; also, a pod can use an image that another tenant already pulled to the node, bypassing the registry authentication check, unless the AlwaysPullImages admission controller is enabled, which forces credentials to be checked each time. Best practices: reference images by immutable digest, not tags, so what runs is exactly what you scanned and signed; use private registries with authentication, giving nodes or namespaces scoped pull credentials, ideally via workload identity; restrict allowed registries through admission policy so only your approved registries are permitted; mirror and scan third-party images rather than pulling directly from public registries, avoiding rate limits and tampering; enable signature verification; use registry features such as immutable tags, vulnerability scanning, and access auditing; and manage pull secrets carefully, as they're credentials. Combine these with image retention policies.

Likely follow-ups

- Why does AlwaysPullImages matter in multi-tenant clusters?
- Why are digests preferable to tags?

```
✓ pull from trusted private registry only (block public)
```

```
✓ reference images by digest, not mutable :latest tag
```

```
✓ imagePullPolicy + registry allowlist via admission policy
```

Referencing `:latest` or other mutable tags means the image running today can differ from what you scanned and approved — a tag can be repointed to a malicious image. Pin by immutable digest (`@sha256:...`) so you run exactly the bytes you verified. Restrict pulls to your trusted registry (block arbitrary public registries via admission policy), require signature verification, and scan images in the registry. This ensures only known, scanned, signed images ever reach the cluster.

:latest is not a version: It's a mutable pointer. Digest-pinning is the difference between "the image I reviewed" and "whatever's there now."

📚 Official docs

- Kubernetes: Images (pull policy, registries)

certs.eknathalabs.com

Q44 What is multi-tenancy isolation in Kubernetes and how do you achieve it? K8s / Container

🎙 Spoken answer · ~87 s

Multi-tenancy means several teams or customers share a cluster, and the challenge is isolating them from each other for security, resource fairness, and blast radius. Kubernetes has no single tenancy switch, so you combine controls. Namespace per tenant as the basic boundary. Access: RBAC limiting each tenant to its namespaces, with identities via the identity provider. Network: default-deny NetworkPolicies, so tenants can't reach each other's pods, with explicit allows for shared services. Resources: ResourceQuotas and LimitRanges to prevent one tenant from consuming all capacity, plus priority classes. Workload security: Pod Security Admission at baseline or restricted, policy engines to restrict hostPath, privileged containers, and image sources; seccomp and AppArmor. Secrets and identity: separate service accounts and workload identity per tenant, with secrets scoped to the namespace. Node isolation: for stronger separation, dedicated node pools with taints and tolerations or node selectors, so tenants don't share a kernel, and sandboxed runtimes like gVisor or Kata for untrusted code. Control plane isolation: for the strongest guarantees, a separate cluster per tenant or virtual clusters with vcluster or the Capsule and Hierarchical Namespaces tools. Also, per-tenant observability, cost allocation, and audit logs. Distinguish soft multi-tenancy, between trusted internal teams, from hard multi-tenancy with untrusted tenants, where separate clusters are generally safest.

Likely follow-ups

- When would you choose separate clusters over namespaces?
- What do ResourceQuotas protect against?

```
soft: namespaces + RBAC + NetworkPolicy + ResourceQuota + PSA
```

```
hard: separate clusters, or vCluster, or node isolation + sandboxing
```

Multi-tenancy isolation prevents one tenant from affecting or accessing another. Soft multi-tenancy uses namespaces with RBAC (access), NetworkPolicies (network), ResourceQuotas/LimitRanges (noisy-neighbor), and Pod Security Admission (privilege) — fine for trusted internal teams. For untrusted or hostile workloads, soft isolation isn’t enough (shared kernel, shared control plane); use hard isolation: separate clusters, virtual clusters (vCluster), dedicated node pools with taints, and sandboxed runtimes (gVisor/Kata).

Trust level sets the bar: Internal teams → namespaces suffice. Running arbitrary customer code → you need kernel-level isolation, not just K8s primitives.

📚 Official docs

- Kubernetes: Multi-tenancy

certs.eknathalabs.com

Q45 How do you scan Infrastructure-as-Code (Terraform/manifests) for misconfigurations?

K8s / Container

🎙 Spoken answer · ~90 s

IaC scanning finds misconfigurations in Terraform, CloudFormation, Kubernetes manifests, Helm charts, and Dockerfiles before they are deployed, shifting security left. Tools: Checkov, Trivy config scanning, tfsec, KICS, Terrascan, and Kubernetes-focused ones like Kubesec, kube-linter, and Polaris. They parse the code and evaluate it against rule sets, which check for things like publicly accessible storage buckets, security groups open to the world, unencrypted volumes and databases, missing logging, overly permissive IAM policies, containers running as root or privileged, missing resource limits, and hardcoded secrets. Policy as code engines such as OPA with Conftest, Kyverno, and Sentinel let you write organisation-specific rules, for example required tags, allowed regions, or approved instance types. Implementation: run scanners locally via pre-commit hooks for fast feedback, in pull request CI so findings appear in review, and fail the build above a defined severity. For Terraform, scan the plan output as well as the source, because the plan reflects resolved values and modules. Add admission control in the cluster, which catches what slips through. Handle noise: tune rule sets, suppress with documented exceptions in code that include justification and expiry, and baseline existing issues, fixing new ones first. Also run drift detection and cloud posture management on live resources, since scanning code alone doesn't catch manual console changes. Track findings over time.

Likely follow-ups

- Why scan the Terraform plan and not only the source?
- How do you manage exceptions to scanner rules?

```
checkov -d .            # Terraform/K8s/CFN misconfig scanning
```

```
tfsec / trivy config .   # security rules for IaC
```

```
kube-linter lint ./manifests   # K8s best-practice checks
```

IaC scanners catch insecure configurations before they're ever provisioned — a public S3 bucket, a security group open to 0.0.0.0/0, an unencrypted volume, a privileged pod. Tools like Checkov, tfsec/Trivy, and kube-linter run in CI against your Terraform and manifests, failing the build on policy violations. This is "shift left": fixing a misconfiguration in a PR is trivial; fixing it after it's deployed (and possibly exploited) is an incident.

Cheapest place to fix: A misconfig caught in a PR costs a one-line change. The same misconfig found in prod by an attacker costs an incident. Scan IaC in CI.

📚 Official docs

- Checkov

certs.eknathalabs.com

Q46 What is the difference between authentication, authorization, and admission in K8s?

K8s / Container

🎙 Spoken answer · ~88 s

These are three distinct stages that every Kubernetes API request passes through. Authentication answers 'who are you?': the API server identifies the caller using client certificates, bearer tokens such as service account tokens, OIDC tokens from an identity provider, webhook authenticators, or a cloud provider's IAM integration. The result is a username, groups, and extra attributes; unauthenticated requests get 401, or are treated as anonymous if enabled. Authorisation answers 'are you allowed to do this?': given the identity and the request's verb, resource, namespace, and name, authorisers evaluate it. RBAC is the standard, alongside Node authorisation for kubelets, ABAC, and webhooks; failure gives 403 Forbidden. Admission answers 'is this particular request acceptable, and should it be modified?': after authorisation, admission controllers intercept create, update, and delete requests; mutating admission can change the object, such as injecting sidecars or setting defaults, and validating admission can reject it, as with Pod Security Admission, ResourceQuota, and policy engines like Kyverno and Gatekeeper, which can enforce rules RBAC cannot express, such as 'no privileged containers' or 'images must be signed'. The order is authentication, authorisation, mutating admission, schema validation, validating admission, then persisting to etcd. RBAC controls who can create pods; admission controls what those pods may contain. Both are needed for defence in depth.

Likely follow-ups

- What is the difference between mutating and validating admission?
- Can RBAC stop someone creating a privileged pod?

```
request → [AuthN: who are you?]
```

```
        → [AuthZ/RBAC: are you allowed?]
```

```
        → [Admission: should this specific object be allowed/mutated?]
```

```
        → persisted to etcd
```

Every API request passes three gates. **Authentication** establishes identity (certs, OIDC, SA tokens). **Authorization** (RBAC) decides whether that identity may perform the action on the resource. **Admission control** then inspects the actual object — validating it against policy (reject privileged pods, unsigned images) or mutating it (inject sidecars, defaults). Understanding the order matters: RBAC says "you may create pods," admission says "but not *this* privileged pod." They're complementary layers, not alternatives.

RBAC vs admission: RBAC is about verbs/resources ("can create pods"); admission is about object content ("this pod violates policy"). You need both.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: RBAC

Q47 How do you handle vulnerability scanning gates without blocking all deploys?

K8s / Container

🎙 **Spoken answer · ~86 s**

Vulnerability gates are needed, but a naive gate that fails on any finding blocks deployments for issues you can't fix and trains teams to bypass it. A balanced approach is risk-based gating. Define policy by severity and context: block on critical, and optionally high, vulnerabilities that have a fix available and are in internet-facing or sensitive workloads; allow findings with no available fix, or those marked not affected through VEX, but track them. Apply different thresholds by environment, strict for production releases, advisory for development branches. Use an exception process: time-boxed, documented exceptions with an owner and justification and an expiry date, approved by security, stored as code so they're visible. Shift left to make gates rarely fail: scan early on pull requests so developers fix issues before release, use minimal base images, and automate dependency updates with Renovate or Dependabot. Add an SLA-based model: instead of blocking all deploys, require remediation within defined windows, such as critical in seven days, high in thirty, and escalate when overdue. Separate the gate on new vulnerabilities introduced by this change, from the pre-existing backlog, so a team isn't blocked by old issues unrelated to their change. Provide emergency override with audit for urgent fixes, including security fixes themselves.

Likely follow-ups

- Why distinguish new findings from the existing backlog?
- What should an exception record contain?

```
✓ gate on CRITICAL/HIGH only; warn on MEDIUM/LOW
```

```
✓ require a fix to be available before failing (no unfixable blocks)
```

```
✓ time-boxed, approved exceptions with an owner + expiry
```

```
✓ use VEX to drop non-exploitable findings
```

An all-or-nothing "any CVE fails the build" gate gets disabled within a week because many CVEs are unfixable or irrelevant. Make gating pragmatic: block only on high-severity CVEs that have a fix available, use VEX to suppress non-exploitable findings with justification, and allow time-boxed exceptions that require an owner and auto-expire (so they don’t become permanent). This keeps the gate credible and enforced rather than bypassed.

A bypassed gate protects nothing: The goal is a gate teams respect. Over-strict gates get turned off; calibrated ones actually stop risky deploys.

certs.eknathalabs.com

📚 **Official docs**

- Trivy

Q48 What is mTLS in a service mesh and what threat does it address?

K8s / Container

🎙 **Spoken answer · ~72 s**

mTLS, mutual TLS, means both sides of a connection present certificates and verify each other, giving encryption and strong service identity. In a service mesh such as Istio, Linkerd, or Cilium service mesh, sidecar proxies, or per-node proxies in ambient modes, handle this transparently: the mesh's control plane acts as a certificate authority, issuing short-lived certificates to each workload based on its Kubernetes service account identity, in SPIFFE format, and rotating them automatically. When service A calls service B, the proxies establish an mTLS connection, each verifying the other's certificate chain and identity, so applications need no TLS code. Threats it addresses. Eavesdropping and tampering on the network inside the cluster, since traffic is encrypted. Spoofing and impersonation: a rogue pod can't pretend to be the payments service without the right identity certificate. Lateral movement: combined with authorisation policies, the mesh can enforce that only the checkout service may call payments, by identity rather than by IP addresses, which are unreliable in Kubernetes. It also gives an audit trail of service-to-service calls.

Likely follow-ups

- What is a SPIFFE identity?
- Why use identity-based policies instead of IP-based ones?

```
# mesh auto-issues certs; enforce strict mTLS:
```

```
kind: PeerAuthentication
```

```
spec: {mtls: {mode: STRICT}}
```

Mutual TLS encrypts and mutually authenticates service-to-service traffic: both sides present certificates, so each verifies the other’s identity and the traffic is encrypted in transit. It addresses the "flat trusted network" assumption — without mTLS, anyone who gains a network foothold can sniff traffic and impersonate services. A service mesh (Istio, Linkerd) automates certificate issuance and rotation transparently, so you get strong workload identity and encrypted east-west traffic without changing app code. Identity-based authorization policies then build on those verified identities.

Identity, not IP: mTLS lets you write policy like "only the checkout service may call payments" based on cryptographic identity, not spoofable IPs.

certs.eknathalabs.com

📚 **Official docs**

- Cloudflare Learning: mutual TLS
- Istio documentation

Q49 How do you audit a Kubernetes cluster for security posture?

K8s / Container

🎙 **Spoken answer · ~86 s**

Auditing a cluster's security posture means checking configuration against best practices, finding vulnerabilities, and testing controls. Benchmark checks: run kube-bench against the CIS Kubernetes Benchmark to examine control plane, etcd, kubelet, and worker node configuration; cloud providers' managed benchmarks and tools like Kubescape, which checks against NSA and CIS frameworks and MITRE ATT&CK mappings. Configuration scanning: Trivy operator, Polaris, or kube-score for workload manifests, finding privileged pods, root users, missing limits, hostPath mounts, and missing probes. Access review: audit RBAC with rbac-tool, KubiScan, or kubectl auth can-i for cluster-admin bindings, wildcards, secrets access, and service accounts with excess rights; review who has cloud-level access to the cluster. Network: check that default-deny policies exist, and what's exposed through services, ingresses, and the API endpoint. Image and supply chain: scan running images for vulnerabilities and check provenance and signatures. Secrets handling and etcd encryption. Control plane and node patch levels and versions. Audit logs: confirm enabled and shipped, and review for suspicious activity. Runtime detection coverage. Then go further with penetration testing or red-team exercises using tools like kube-hunter. Make it continuous, not one-off: run scanners on a schedule or as in-cluster operators, send results to dashboards, prioritise by risk, assign owners, track remediation, and re-audit after changes.

Likely follow-ups

- What does kube-bench check?
- Why make auditing continuous?

```
kube-bench          # CIS Kubernetes Benchmark checks
```

```
kube-hunter         # active attack-surface probing
```

```
trivy k8s cluster   # workload + config + RBAC scanning
```

```
+ review: RBAC, PSA, NetworkPolicy coverage, audit logs on
```

A posture audit combines automated benchmarking and manual review. `kube-bench` checks the cluster against the CIS Kubernetes Benchmark (control-plane flags, file permissions, etc.); `kube-hunter` probes for exploitable weaknesses; `trivy k8s` scans workloads, configs, and RBAC. Then review the things tools can't fully judge: is RBAC least-privilege, is there NetworkPolicy coverage, is Pod Security Admission enforced, is audit logging on and shipped somewhere. Track findings over time, not just a one-off scan.

certs.eknathalabs.com

CIS Benchmark as a baseline: kube-bench gives you a concrete, industry-standard checklist — a great starting scorecard for "how hardened is this cluster?"

📚 **Official docs**

- Kubernetes: Security checklist
- Linux audit documentation (auditd)

Q50 What is the supply-chain risk of Helm charts and how do you mitigate it?

K8s / Container

🎙 **Spoken answer · ~83 s**

Helm charts are templated packages that generate Kubernetes manifests, and they carry supply-chain risk because installing one can create powerful resources: ClusterRoles, privileged workloads, webhooks, and CRDs. Risks: a malicious or compromised chart, or one from an untrusted repository; typosquatted chart names; charts with overly broad RBAC, hostPath mounts, privileged containers, or insecure defaults; images referenced inside the chart that are unpinned, from unknown registries, or vulnerable; hooks that run arbitrary jobs at install time; template logic that behaves differently from what you reviewed; unmaintained charts that stay outdated; and updating a dependency chart can silently change behaviour. Mitigations. Source: prefer charts from trusted, maintained publishers, and pin exact versions, not ranges. Verify integrity: use chart signing with provenance files and helm verify, or OCI charts signed with cosign. Review before installing: run helm template and read the rendered output, diff against the previous version with the helm diff plugin, and scan the rendered manifests with Checkov, Trivy, or kube-linter, plus the images. Mirror approved charts and images into an internal registry, and prevent direct use of public repositories in CI. Enforce with admission policies: Pod Security, allowed registries, signature checks, so a bad chart can't create dangerous pods.

Likely follow-ups

- How would you review a third-party chart before use?
- Why mirror charts and images internally?

```
✓ pull charts from trusted/verified repos; pin chart versions
```

```
✓ helm template + review/scan rendered manifests before apply
```

```
✓ verify provenance: helm verify (signed charts)
```

A third-party Helm chart is executable templating that can deploy anything — over-privileged RBAC, a hidden sidecar, an image from an untrusted registry. Mitigate by sourcing charts only from trusted repositories, pinning exact chart versions (not floating), and — critically — rendering with `helm template` and reviewing/scanning the actual manifests it produces before applying. Verify signed-chart provenance with `helm verify` where available. Never `helm install` a random chart into prod without inspecting what it actually creates.

certs.eknathlabs.com

Render before you trust: A chart's values look innocent; the rendered output might grant cluster-admin. Always `helm template` and scan the real manifests.

📚 **Official docs**

- Helm: Chart best practices
- Helm documentation

Q51 How do you implement least-privilege egress (data exfiltration prevention)?

K8s / Container

🎙 **Spoken answer · ~88 s**

Least-privilege egress means workloads can only connect out to destinations they genuinely need, which blocks data exfiltration, command-and-control callbacks, and downloading of attacker tools after a compromise. Most environments leave egress wide open, which is a major gap. Layers. Kubernetes: a default-deny egress NetworkPolicy per namespace, then allow only DNS, specific internal services, and required external CIDRs. Plain policies work on IPs, so for domain-based control use Cilium or Calico FQDN policies, or route traffic through an egress gateway or proxy that enforces an allowlist of hostnames. Cloud network: private subnets without direct internet routes, with outbound traffic through NAT and a firewall such as AWS Network Firewall, Azure Firewall, or Cloud NGFW applying domain allowlists and inspection; security groups limiting egress ports; VPC endpoints or Private Google Access so access to cloud services stays on the provider network, with endpoint policies restricting which buckets or accounts can be reached; DNS filtering and logging, since DNS tunnelling is a common exfiltration route. Data-level controls: VPC Service Controls or equivalent perimeters, and bucket policies preventing copies to external accounts. Operations: build the allowlist by observing flow logs first, then enforce; monitor denied connections and unusual volumes; and treat exceptions as reviewed requests. Egress control is hard to retrofit, so design it in early.

Likely follow-ups

- Why is egress control often neglected?
- How do you build an allowlist safely?

```
# default-deny egress, allow only required destinations
```

```
kind: NetworkPolicy
```

```
spec: {podSelector: {}, policyTypes: [Egress], egress: [...]}
```

```
# + egress gateway / DNS allowlist (Cilium FQDN policy)
```

Most network controls focus on ingress, but egress control limits where a compromised pod can send data and which command-and-control servers it can reach. Apply default-deny egress and allow only the specific destinations a workload needs (its database, specific APIs). Cilium's FQDN-based policies let you allowlist by

certs.eknathlabs.com

domain; an egress gateway centralizes and logs outbound traffic. This both prevents data exfiltration and breaks malware that needs to phone home.

> **Egress control kills C2:** Malware that can't reach its command server is largely inert. Default-deny egress is an underused, high-impact control.

> 📚 **Official docs**
> - Google Cloud: VPC Service Controls (data exfiltration)

Q52 What security responsibilities are yours vs the cloud provider for managed K8s (EKS/GKE/AKS)?

K8s / Container

🎙 **Spoken answer · ~88 s**

In managed Kubernetes, security is shared, and the exact split depends on the service. The provider secures the control plane: API servers, etcd, schedulers, their patching, availability, and physical and infrastructure security; in EKS, GKE, and AKS this is managed for you. You are responsible for what runs on and around it. Data plane: with standard node pools, node operating system patching and upgrades, though some services such as GKE Autopilot and Fargate shift much of this to the provider; node hardening and configuration. Cluster configuration: choosing private endpoints and authorised networks, enabling audit logs and encryption of secrets, RBAC and identity integration, network policies, Pod Security settings, and admission control. Workloads: image security, vulnerability management, secrets handling, container hardening, and runtime protection. Identity: IAM roles, workload identity, and least privilege, including the cloud permissions for nodes. Networking: VPC design, firewall rules, ingress exposure, and egress control. Data: encryption choices and key management, backups. Upgrades: you must upgrade Kubernetes versions on time, though the provider offers tooling. Monitoring, logging, and incident response are yours. Common mistakes are assuming the provider handles node patching, leaving the API endpoint public, over-permissive node IAM roles, and not enabling audit logs. Read each provider's shared responsibility documentation, and use benchmarks to verify your side.

Likely follow-ups

- Which responsibilities shift on GKE Autopilot?
- What is the most common misunderstanding?

```
provider: control-plane patching, etcd encryption infra, HA
```

```
you: node OS patching*, RBAC, NetworkPolicy, workloads,
```

```
     image security, secrets, IAM, app code  (*managed nodes help)
```

cors.eknathalabs.com

Managed Kubernetes follows the shared responsibility model. The provider secures and patches the control plane (API server, etcd, scheduler) and underlying infrastructure. *You* remain responsible for everything above: node configuration and patching (eased by managed node groups/auto-upgrade), RBAC, network policies, workload hardening, image and supply-chain security, secrets, cloud IAM integration, and your application code. The common mistake is assuming "managed" means "secured" — the provider hands you a running control plane, not a hardened cluster.

Managed ≠ secure: Provider responsibility ends at the control plane. Misconfigured RBAC, open NetworkPolicies, and bad workloads are entirely on you.

📚 **Official docs**

- Amazon EKS: User guide
- Azure Kubernetes Service (AKS)
- GKE overview

Q53 What is policy-as-code and why adopt it?

Policy-as-Code

🎙 **Spoken answer · ~89 s**

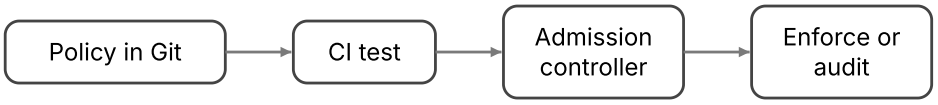
Policy-as-code means expressing security, compliance, and operational rules as machine-readable code, stored in version control, tested, and automatically enforced across the pipeline and runtime, instead of in documents and manual reviews. Examples: no public storage buckets, images only from approved registries, all resources tagged, containers non-root. Why adopt it. Consistency: the same rules apply everywhere, without depending on reviewers remembering them. Speed and scale: automated checks give developers immediate feedback and remove manual approval bottlenecks, which matters with hundreds of teams. Auditability: policies are versioned with history, reviews, and tests, and decisions are logged, which makes compliance evidence easier. Shift left: evaluate policy on infrastructure code and manifests in CI, before deployment, and again at admission or cloud control points, as defence in depth. Collaboration: security, platform, and development teams review the rules like any code. Reproducibility: policies can be unit tested and promoted across environments. Tools: OPA with Rego, Kyverno, Gatekeeper, Sentinel, Checkov custom policies, cloud-native options such as AWS SCPs, Azure Policy, and GCP Organisation Policies. Implementation tips: start with a small set of high-value rules in audit mode, involve developers, write clear failure messages with remediation guidance, manage exceptions explicitly, and monitor policy outcomes. The goal is guardrails that make the secure path the easy one, not gates that block people.

Likely follow-ups

- Where in the lifecycle should policies be evaluated?
- Why are clear failure messages important?

corp.eknathalabs.com

🗺 **Policy as code**




```
# policies are versioned, tested, reviewed code — not wiki pages
```

```
OPA/Rego, Kyverno, Sentinel, Cedar
```

```
# enforced automatically in CI and at admission
```

Policy-as-code expresses security and compliance rules as version-controlled, testable code that's enforced automatically — instead of PDFs and human review that don't scale and drift. "No public S3 buckets," "all images must be signed," "pods must run non-root" become machine-enforced policies in CI and at admission time. Benefits: consistency (the same rule everywhere), auditability (git history of policy changes), testing (validate policies before rollout), and speed (instant feedback to developers instead of a security review queue).

Guardrails over gates: Policy-as-code lets developers move fast within automatically-enforced boundaries, rather than waiting on manual security sign-off.

📚 **Official docs**

- Open Policy Agent documentation

certs.eknathalabs.com

Q54 What is OPA and how does Rego work at a high level?

Policy-as-Code

🎙 Spoken answer · ~88 s

Open Policy Agent is a general-purpose policy engine, a CNCF graduated project, that decouples policy decisions from the systems that enforce them. A service sends OPA a JSON query describing a request, such as the Kubernetes admission review or a Terraform plan, and OPA evaluates it against policies and returns a decision, such as allow or deny with messages. Policies are written in Rego, a declarative, query-oriented language inspired by Datalog. At a high level: you write rules that define values or sets based on the input and any loaded data. A rule like deny produces a message when a set of conditions all hold; the body of a rule is a list of expressions that are implicitly ANDed, and multiple rules with the same name are ORed. Rego has built-in functions, supports iteration over collections with comprehensions, and the data document can include external context, such as the list of approved registries. For example, a policy might iterate over the containers in a pod spec and deny when any container's image doesn't start with an approved registry prefix. Policies are tested with opa test, unit tests written in Rego, and evaluated in CI with Conftest. OPA integrates in Kubernetes through Gatekeeper, which adds constraint templates and constraints as custom resources.

Likely follow-ups

- How are multiple rules with the same name combined?
- How do you test Rego policies?

```
# Rego: deny if a pod runs as root
deny[msg] {
  input.kind == "Pod"
  not input.spec.securityContext.runAsNonRoot
  msg := "pods must set runAsNonRoot"
}
```

Open Policy Agent (OPA) is a general-purpose policy engine that evaluates JSON/YAML input against policies written in Rego, returning allow/deny decisions. It's decoupled from any one system — you can use it for Kubernetes admission (via Gatekeeper), Terraform plan checks, API authorization, and more. Rego is declarative: you write rules that produce decisions (e.g. `deny` messages) when conditions match the input. The engine handles evaluation; you describe the desired constraints.

One engine, many domains: OPA's value is a single policy language/engine across K8s, IaC, microservice authz, and CI — consistent governance everywhere.

certs.eknathalabs.com

📚 **Official docs**

- Open Policy Agent documentation

Q55 How does Kyverno differ from OPA Gatekeeper for Kubernetes policy?

Policy-as-Code

🎙 **Spoken answer · ~89 s**

Both enforce policies in Kubernetes through admission webhooks, but they differ in approach. OPA Gatekeeper uses OPA and Rego: you define a ConstraintTemplate containing the Rego logic, then create Constraint resources that parameterise and apply it to matching resources. It's powerful and expressive, the same Rego skills carry to other systems such as Terraform and API authorisation, and it has a mature audit feature to report existing violations. The downside is the learning curve of Rego and more verbose setup; mutation support exists but is less central. Kyverno is Kubernetes-native: policies are written as YAML Kubernetes resources, ClusterPolicy and Policy, using patterns, overlays, and CEL or JMESPath expressions, so there's no new language to learn, and it feels familiar to platform teams. It supports validate, mutate, generate, and image verification natively, including cosign signature verification, and background scanning of existing resources, plus policy reports. It's easier to adopt, but is focused on Kubernetes, and complex logic can get awkward in YAML. Choosing: Kyverno if you want fast adoption, mutation and generation, and simpler authoring, for Kubernetes-only needs; Gatekeeper and OPA if you want one policy language across multiple systems, or need very complex logic and already have Rego skills. Also consider Kubernetes' built-in ValidatingAdmissionPolicy with CEL, which handles simple cases without any webhook.

Likely follow-ups

- What are ValidatingAdmissionPolicies?
- When would you choose Gatekeeper over Kyverno?

```
Kyverno   → policies as native YAML; validate + MUTATE + generate
```

```
Gatekeeper→ OPA/Rego; powerful, general, steeper learning curve
```

Both are Kubernetes admission-policy engines, but differ in approach. Kyverno uses Kubernetes-native YAML policies (no new language), and beyond validation it can *mutate* resources (inject defaults, labels, securityContext) and *generate* resources (auto-create a NetworkPolicy per namespace) — lower barrier for K8s teams. Gatekeeper uses OPA/Rego, which is more expressive and reusable beyond Kubernetes but has a steeper learning curve. Choose Kyverno for K8s-focused simplicity and mutation; Gatekeeper/OPA when you want one policy language across many systems.

Kyverno's mutation is underrated: Auto-injecting secure defaults (drop caps, seccomp) means workloads become compliant without developers needing to know the rules.

certs.eknathalabs.com

📚 **Official docs**

- OPA Gatekeeper
- Open Policy Agent documentation
- Kyverno documentation

Q56 Write a policy concept: enforce that all images come from an approved registry.

Policy-as-Code

🎙 **Spoken answer · ~85 s**

Conceptually, the policy says: every container in every pod, including init and ephemeral containers, must pull its image from an approved registry, and anything else is rejected. The design pieces. Input: the admission request for pods, or for workload resources like Deployments if you want earlier feedback. Data: a list of allowed registry prefixes, such as the company's private registry and a mirror of trusted public images, kept as configuration so it can change without rewriting the policy. Logic: for each container in the spec, check whether the image string begins with one of the allowed prefixes; if none match, produce a violation naming the container and image and explaining which registries are permitted. In Kyverno, this is a validate rule with a pattern on spec.containers.image using wildcard alternatives; in Gatekeeper, a constraint template with Rego plus a constraint listing the repos; and Kubernetes ValidatingAdmissionPolicy offers a CEL expression. Considerations: normalise image references, since an image without a registry defaults to Docker Hub, and beware of tricks like a registry name embedded in a path; require digests or signatures as an additional layer; exclude system namespaces explicitly and carefully; start in audit or warn mode to find violators; and keep the failure message actionable.

Likely follow-ups

- Why must image references be normalised?
- How do you let teams use public images safely?

```
# Kyverno: reject images not from registry.corp.com
validationFailureAction: Enforce
rules:
- match: {resources: {kinds: [Pod]}}
  validate:
    pattern: {spec: {containers: [{image: "registry.corp.com/*"}]}}
```

certs.eknathalabs.com

This is a foundational supply-chain control: only run images from your trusted, scanned registry. The policy matches Pods and validates that every container image starts with your approved registry prefix, rejecting anything else at admission. It stops developers (or attackers) from pulling arbitrary public images that haven't been scanned or signed. Combine with signature verification for defense in depth — right source *and* proven integrity.

Choke point control: Forcing all images through one registry gives you a single place to scan, sign, and govern everything that runs.

📚 **Official docs**

- Kyverno: Policies (validate, mutate, generate)

Q57 How do you test and roll out policies safely without breaking workloads?

Policy-as-Code

🎙 **Spoken answer · ~88 s**

Policies can break workloads, so rollout needs to be gradual, observable, and reversible. Develop: write policies as code in Git, with unit tests using sample resources, both passing and failing, run in CI with tools like opa test, kyverno test, or Conftest. Run policies against the actual manifests in your repositories and clusters to preview the impact. Roll out in stages: first audit or dry-run mode, which logs violations without blocking; then warn mode, which shows developers warnings when they apply resources; then enforce on non-production namespaces; and finally enforce in production, namespace by namespace or by team. Throughout, review violation reports and dashboards, and contact owners of violating workloads with clear remediation guidance and a deadline. Plan for existing workloads: policies usually apply to new and updated resources at admission, but background scans find existing violations, so they can be fixed gradually. Provide exceptions through a formal, time-limited process. Safety features: exclude critical system namespaces, make sure the webhook is highly available with sensible timeouts, and choose the failure policy deliberately, since failing closed can block cluster operations if the policy engine goes down. Have a quick rollback: the ability to switch a policy back to audit mode with a single change. Communicate widely, publish documentation, and gather feedback.

Likely follow-ups

- Why run in audit mode before enforcing?
- What is the risk of a webhook failing closed?

```
1. write unit tests for the policy (opa test / kyverno test)
```

```
2. deploy in audit/warn mode → observe violations
```

```
3. fix or exempt existing offenders
```

```
4. switch to enforce; monitor admission denials
```

cors.eknathalabs.com

A policy that blocks legitimate deploys gets ripped out, so roll out gradually. Unit-test the policy logic against known-good and known-bad inputs. Deploy first in audit/warn mode to discover what *would* be blocked without actually blocking — surfacing existing non-compliant workloads. Remediate or grant scoped, documented exceptions for those, then flip to enforce. Monitor admission denials afterward so you catch false positives fast. This is the same warn→enforce pattern as Pod Security Admission.

Never go straight to enforce: Cluster-wide enforce on day one will block existing workloads and cause an outage. Audit first, always.

📚 **Official docs**

- Kyverno: Policies (validate, mutate, generate)

Q58 What is the difference between validating, mutating, and generating policies?

Policy-as-Code

🎙 **Spoken answer · ~78 s**

These are three kinds of actions a policy engine can take on resources. Validating policies check a resource against rules and either allow or reject it. They don't change anything; for example, reject pods that run as root or that lack resource limits. They're the safest and most common type, and useful in audit mode too. Mutating policies modify resources as they're admitted, adding or changing fields: injecting a default security context, adding labels or annotations, setting resource limits, adding a sidecar, or rewriting an image tag to a digest or a registry to a mirror. They make the secure choice the default and reduce developer effort, but they can surprise users because the deployed object differs from the submitted one, complicate GitOps, which may show drift, and need care with ordering since mutations run before validations. Generating policies create new resources in response to events: when a namespace is created, automatically generate a default-deny NetworkPolicy, ResourceQuota, LimitRange, and RoleBindings; they keep generated resources synchronised and can clone secrets or config into namespaces. Kyverno supports all three natively; Gatekeeper supports validation and mutation, with generation less central.

Likely follow-ups

- Why can mutation conflict with GitOps?
- What would you generate for each new namespace?

```
validate → allow/deny based on rules (reject privileged pods)
```

```
mutate   → modify the object (inject securityContext defaults)
```

```
generate → create new objects (a default-deny NetPol per namespace)
```

certs.eknathalabs.com

Validating policies make a pass/fail decision and reject non-compliant resources. Mutating policies transform resources as they're admitted — injecting secure defaults so workloads become compliant automatically (e.g. setting `runAsNonRoot`, adding seccomp, dropping capabilities). Generating policies create companion resources (auto-provision a NetworkPolicy, ResourceQuota, or default Secret for each new namespace). Used together, they let you both enforce rules and pave a secure-by-default path so teams don't have to remember every control.

Mutation = secure defaults: Instead of rejecting insecure pods and making devs fix them, mutating policies quietly make every pod secure. Less friction, same outcome.

📚 **Official docs**

- Kyverno: Policies (validate, mutate, generate)

Q59 How do you enforce compliance frameworks (CIS, PCI, SOC2) as code?

Policy-as-Code

🎙 **Spoken answer · ~84 s**

Enforcing frameworks like CIS, PCI DSS, and SOC 2 as code means translating their control requirements into automated, continuously evaluated checks, with evidence generated automatically. Start by mapping each control to technical implementations and owners; many controls are procedural and need human processes, but a large portion, such as encryption, logging, access controls, network segmentation, and patching, can be automated. Prevent: use policy-as-code at multiple points. In Terraform and Kubernetes CI, scan with Checkov, tfsec, and Conftest, using rule sets mapped to CIS benchmarks; at deployment, admission controllers enforce pod and image rules; at cloud level, use service control policies, Azure Policy, and Organisation Policies to forbid non-compliant configurations, such as unencrypted storage or public buckets. Detect: continuous posture tools, such as AWS Config conformance packs, Security Hub standards, Azure Defender and GCP Security Command Center, kube-bench for CIS Kubernetes, and Kubescape, evaluate live resources and report drift. Remediate: auto-remediation for safe cases, and ticketing for others. Evidence: collect logs, configuration snapshots, and policy reports automatically, and use compliance automation platforms such as Vanta or Drata, or OSCAL-based tools, to map checks to controls and keep audit-ready evidence. Treat policies as versioned code with reviews, record exceptions with justification, and review regularly.

Likely follow-ups

- Which controls can't be automated?
- How does continuous evidence collection help audits?

```
map controls → automated checks:
```

```
  "encrypt data at rest" → IaC scan rule + runtime check
```

```
  tools: Checkov packs, OPA bundles, cloud Config rules, Steampipe
```

ceris.eknathalabs.com

Compliance frameworks are sets of controls; policy-as-code translates each control into an automated, continuously-evaluated check. Instead of an annual manual audit, you map requirements (encryption at rest, access logging, MFA, network segmentation) to rules in Checkov, OPA, cloud-native config services (AWS Config, GCP SCC), or query tools like Steampipe — running continuously and producing evidence. This gives continuous compliance and auditor-ready evidence on demand, rather than a frantic point-in-time scramble.

Continuous > point-in-time: Automated control checks mean you're compliant every day and can prove it instantly — not just compliant the week before the audit.

📚 **Official docs**

- NIST Cybersecurity Framework

Q60 What is GitOps and how does it strengthen security posture?

Policy-as-Code

🎙 **Spoken answer · ~68 s**

GitOps is an operating model where Git is the single source of truth for the desired state of infrastructure and applications, and an automated agent, such as Argo CD or Flux, continuously reconciles the live environment to match what's in Git. Changes happen through pull requests, not by running commands against the cluster. Security benefits. Reviewed and auditable changes: every change has an author, reviewers, approvals, and history, which supports compliance and forensics. Reduced credentials: the agent runs inside the cluster and pulls changes, so CI systems and developers don't need cluster-admin credentials, and the attack surface of external access to the cluster shrinks. Drift detection and self-healing: manual changes, including malicious or accidental ones, are detected and reverted. Consistency and reproducibility: environments are rebuilt from Git, helping disaster recovery and making configurations uniform. Policy enforcement: branch protection, required reviews, signed commits, and CI policy checks apply to every change, with admission policies as a second layer. Easy rollback by reverting a commit.

Likely follow-ups

- How does GitOps reduce credential exposure?
- What are the risks of the GitOps agent?

```
# Git is the single source of truth; a controller reconciles
```

```
ArgoCD / Flux → cluster state == git state, auto-reverts drift
```

GitOps declares desired state in Git and uses a controller (ArgoCD, Flux) to continuously reconcile the live cluster to match. Security benefits: every change is a reviewed, signed, audited PR (no manual `kubectl` in prod); drift is auto-detected and reverted (an attacker's out-of-band change gets undone); the full change history is in Git; and you can disable direct cluster write access entirely, funneling all changes through the reviewed pipeline. The Git repo becomes the authoritative, auditable control plane.

certs.eknathalabs.com

Drift reversion is a security feature: If an attacker manually edits a Deployment, the GitOps controller reverts it within minutes — unauthorized changes don't persist.

📚 **Official docs**

- Kubernetes: Security checklist
- OpenGitOps principles

Q61 How do you manage policy exceptions without undermining the whole system?

Policy-as-Code

🎙 **Spoken answer · ~86 s**

Exceptions are inevitable, because rules will not fit every legitimate case, but poorly managed exceptions erode a policy until it means nothing. Principles. Make them explicit and visible: exceptions are declared in code, such as Kyverno PolicyExceptions, Gatekeeper config exclusions, or annotated suppressions in scanners, stored in Git, reviewed by security, and never granted through verbal agreements or silent bypasses. Require justification: who owns it, why it's needed, what compensating controls exist, the risk accepted, and a ticket reference. Scope narrowly: to a specific resource, namespace, or workload, not to a whole class of resources or a blanket wildcard. Time-box: every exception has an expiry date and triggers a review; automated alerts when they approach or pass expiry, and the exception is removed by default if not renewed. Approval: appropriate to the risk, with higher-risk exceptions requiring security leadership sign-off. Track: a register with metrics on count, age, and trends; a growing number of exceptions for one policy suggests that the policy needs improving or that a better solution should be provided, such as a safe alternative. Detect abuse: audit who creates exceptions and restrict that permission with RBAC. Review periodically and fix underlying causes, for instance by adding a supported path for the common use case.

Likely follow-ups

- Why must exceptions expire?
- What does a rising exception count tell you?

```
# exceptions must be: explicit, scoped, owned, time-bound, audited
```

```
PolicyException: {match: specific-workload, expiry: 2026-09-01}
```

Some exceptions are legitimate (a legacy app that can't yet run non-root). The danger is permanent, unscoped, forgotten exceptions that hollow out enforcement. Make every exception explicit and narrowly scoped to a specific workload, attached to an owner and a business justification, time-bound with an expiry that forces re-review, and audited/visible. Kyverno PolicyExceptions and similar mechanisms let you grant these as code. The key is that an exception is a tracked, expiring decision — not a quiet permanent hole.

Exceptions rot: A "temporary" exception with no expiry becomes permanent. Always set an expiry and an owner so it's revisited, not forgotten.

certs.ekhathalabs.com

📚 **Official docs**

- Kyverno: Policies (validate, mutate, generate)
- Kubernetes: Policies overview

Q62 What role does policy-as-code play in Terraform/cloud provisioning?

Policy-as-Code

🎙 **Spoken answer · ~85 s**

In Terraform and cloud provisioning, policy-as-code acts as guardrails over what infrastructure can be created. At the pre-deployment stage: scan the Terraform code and, better, the JSON plan output in CI with tools like Checkov, tfsec or Trivy, and Conftest with OPA, or HashiCorp Sentinel in Terraform Cloud or Enterprise, which evaluates policies between plan and apply and can hard-fail the run. Typical rules: storage encrypted, no public buckets or open security groups, required tags, allowed regions and instance types, no overly permissive IAM, required logging, and cost limits. Developers see failures in the pull request, with remediation hints. At the cloud control plane level, as a backstop that can't be bypassed by skipping CI: AWS Service Control Policies and permission boundaries, Azure Policy, and GCP Organisation Policies prevent forbidden configurations regardless of how resources are created, including console changes. Detective controls: AWS Config rules, Security Hub, and Cloud Asset Inventory continuously evaluate deployed resources and detect drift. Good practice: layer these so that the early checks give quick feedback and the cloud-level controls provide enforcement; version and test policies; roll them out in advisory mode first; use shared Terraform modules that already comply, making the secure option the easy one; and manage exceptions formally.

Likely follow-ups

- Why check the plan output instead of only the source?
- Why have cloud-level policies in addition to CI checks?

```
# gate the plan before apply
terraform plan -out tfplan && terraform show -json tfplan \
  | opa eval -d policy/ -I "data.terraform.deny"
# or Sentinel (TFE/TFC), Checkov on the plan
```

Policy-as-code can evaluate a Terraform *plan* before it's applied, blocking insecure infrastructure at provisioning time — no public databases, mandatory encryption, required tags, allowed regions/instance types. OPA/Conftest, HashiCorp Sentinel, and Checkov can all gate on the JSON plan. This catches issues the moment infrastructure is defined, before any resource exists, and enforces organizational standards consistently across every team's Terraform without relying on reviewers to remember every rule.

Gate the plan, not the apply: Evaluating the plan means you reject bad infra before a single resource is created — no cleanup, no exposure window.

certs.eknathalabs.com

📚 **Official docs**

- Conftest
- Open Policy Agent documentation

Q63 How do you detect and prevent configuration drift?

Policy-as-Code

🎙 **Spoken answer · ~88 s**

Configuration drift is when the actual state of infrastructure diverges from the declared state, usually through manual console changes, emergency fixes, scripts, or other tooling. It causes security gaps, such as a security group opened for debugging and never closed, unreproducible environments, and surprises during the next apply. Prevention. Make infrastructure as code the only path: remove or restrict write access in the console and CLI through IAM, giving humans read-only access plus break-glass elevation with audit; apply changes only through pipelines. Use GitOps for Kubernetes, where Argo CD or Flux continuously reconcile and self-heal. Immutable infrastructure: replace instances from new images instead of patching in place. Policy guardrails and organisation-level controls blocking non-compliant changes. Detection. Run terraform plan on a schedule, or use drift detection features such as Terraform Cloud health assessments, driftctl or AWS CloudFormation drift detection, and alert if the plan shows differences. Use CloudTrail, Azure Activity Logs, or audit logs to detect out-of-band changes by non-pipeline identities, with alerts. Use AWS Config or equivalent for continuous compliance. For servers, configuration management tools or file integrity monitoring. Response: decide whether to revert the drift to the declared state or codify the change if it was legitimate, record why, and fix the process gap that led to the manual change.

Likely follow-ups

- How do you detect out-of-band changes through audit logs?
- When is codifying the drift the right response?

```
detect:  GitOps controller diff, terraform plan (shows drift),
```

```
         cloud config rules, periodic scans
```

```
prevent: GitOps auto-reconcile, restrict manual access, IaC-only
```

Drift is when live state diverges from declared state — from manual hotfixes, console clicks, or attacker changes. Detect it by continuously comparing actual vs desired: GitOps controllers flag/revert drift automatically, `terraform plan` reveals out-of-band changes, and cloud config services alert on non-compliant resources. Prevent it by making the declarative pipeline the *only* write path — restrict manual production access so changes must go through reviewed IaC/GitOps. Drift is both an operational and a security problem: it can hide unauthorized changes.

Drift can be an attack signature: An unexplained config change might be a breach, not a teammate's hotfix.

certs.eknathalabs.com

Auto-detection turns drift into a signal.

📚 **Official docs**

- Infrastructure drift (Terraform: plan -refresh-only)

Q64 What is least privilege in cloud IAM and how do you achieve it at scale?

Policy-as-Code

🎙 **Spoken answer · ~89 s**

Least privilege in cloud IAM means identities have only the permissions needed for their tasks, for only the time needed. At scale, you can't hand-craft every policy, so you need systems. Start with structure: separate accounts, projects, or subscriptions by environment and workload, with organisation-level guardrails, such as service control policies and permission boundaries, that cap the maximum privileges anywhere. Use roles instead of users; federate human access through SSO with groups, and give workloads roles via workload identity, not static keys. Define standard, reviewed role templates for common job functions, delivered through Terraform modules, so teams request and use pre-approved patterns instead of writing arbitrary policies. Avoid wildcards and admin-level managed policies; scope to specific resources and conditions. Provide just-in-time elevation for sensitive access with approval, time limits, and logging. Right-size continuously: use tools such as IAM Access Analyzer, GCP IAM Recommender, and Azure Access Reviews to compare granted versus used permissions from access logs, generate least-privilege policies from actual activity, and flag unused roles and keys. Review access regularly, remove stale identities, and detect risky combinations that enable privilege escalation. Monitor high-risk actions with alerts. Treat IAM as code with peer review. Measure the percentage of identities with admin rights, unused permissions, and age of credentials, and aim to reduce them.

Likely follow-ups

- How do permission boundaries cap privileges?
- How do tools generate least-privilege policies from usage?

```
✓ start from zero; grant specific actions on specific resources
```

```
✓ use access analyzers to right-size from actual usage
```

```
✓ avoid wildcards (Action:"*", Resource:"*"), no long-lived keys
```

```
✓ permission boundaries / SCPs as guardrails
```

Least-privilege IAM grants each identity only the actions it needs on the resources it needs. At scale, achieve it by starting from deny and adding specific permissions, using access analyzers (AWS IAM Access Analyzer, GCP Recommender) that observe actual usage and suggest tighter policies, avoiding wildcard

certs.eknathlabs.com

actions/resources, and layering organization-wide guardrails (SCPs, permission boundaries) that cap what any policy can grant. Prefer roles assumed via federation over long-lived keys. Review and prune unused permissions continuously — grants accumulate.

Right-size from real usage: Access analyzers turn "what might it need?" into "what did it actually use in 90 days?" — the data-driven path to least privilege.

📚 **Official docs**

- AWS IAM: Least privilege (grant least privilege)

Q65 How do you secure Terraform state, which often contains secrets?

Policy-as-Code

🎙 **Spoken answer · ~87 s**

Terraform state records every managed resource and its attributes, and that often includes secrets in plaintext: database passwords, generated keys, and connection strings, even if marked sensitive in the output. So state must be treated like a secret. Protect storage: use a remote backend, such as S3 with DynamoDB locking, GCS, Azure Blob, or Terraform Cloud, never local files committed to Git; enable encryption at rest with a KMS key, and encrypt in transit; enable versioning for recovery and restrict deletion. Control access tightly with least-privilege IAM: only the pipeline's deployment role and a few break-glass administrators can read or write state, and separate state files by environment and component, so access to a development state doesn't expose production secrets; block public access and enable access logging with alerts. Reduce what lands in state: avoid having Terraform generate secrets; reference secrets from a secrets manager at runtime through data sources or have applications fetch them, use ephemeral resources and write-only attributes in newer Terraform versions, which keep values out of state; use dynamic credentials and workload identity instead of static provider keys. Never print state or plans with sensitive data in CI logs. Protect backups, rotate secrets that may have been exposed, and scan state with secret scanners.

Likely follow-ups

- Why does state contain secrets even when marked sensitive?
- How do ephemeral values keep secrets out of state?

```
✓ remote backend with encryption at rest (S3+KMS, TFC)
```

```
✓ state locking (DynamoDB) to prevent corruption
```

```
✓ strict access control; NEVER commit state to Git
```

```
✓ minimize secrets in state (use dynamic/external where possible)
```

certs.eknathalabs.com

Terraform state stores resource attributes — often including sensitive values like generated passwords and keys — in plaintext within the state file. Secure it: use a remote backend encrypted at rest and in transit (S3 + KMS, Terraform Cloud), enable state locking to prevent concurrent corruption, tightly restrict who can read the state (it's effectively a secret), and never commit state files to Git. Where possible, avoid putting secrets in state at all by sourcing them dynamically. Treat state access with the same care as the secrets it contains.

State is a secret: Anyone who can read your state file may read database passwords and keys in plaintext. Lock down state access like a credential store.

📚 **Official docs**

- Kubernetes: Good practices for Secrets

Q66 What is shift-left security and what are its limits?

Policy-as-Code

🎙 **Spoken answer · ~88 s**

Shift-left security means moving security activities earlier in the development lifecycle, to design, coding, and CI, where issues are cheaper and faster to fix, using threat modelling, IDE plugins, pre-commit hooks, SAST, dependency and secret scanning, IaC scanning, and policy checks in pull requests. Benefits: faster feedback, lower remediation costs, fewer vulnerabilities reaching production, and developers learning secure habits. Its limits. It can't catch everything: runtime issues, misconfigurations that only exist in deployed environments, business logic flaws, zero-days in dependencies, and attacks on running systems need runtime controls, DAST, testing, monitoring, and incident response, so security must also shift right. Tool noise: false positives and a flood of findings overwhelm developers, who then ignore or bypass the tools; success depends on tuning, prioritisation, and actionable output. Pushing responsibility to developers without training, time, or support leads to burnout and shadow bypasses; developers aren't security experts, so security teams should provide enablement, guardrails, and secure defaults. Slower pipelines hurt productivity if scans are heavy. Coverage depends on teams actually adopting the tools. It can create a false sense of security: passing scans doesn't mean the system is secure. A balanced strategy combines shift-left prevention with runtime detection, threat modelling, paved roads, and measuring outcomes like time to remediate rather than counting scans.

Likely follow-ups

- What does shifting right mean?
- Why does tool noise undermine shift-left?

```
shift-left: catch issues early — IDE, pre-commit, CI, PR review
```

```
but ALSO shield-right: runtime detection, WAF, monitoring
```

certs.eknathalabs.com

Shift-left means moving security earlier in the lifecycle — IDE plugins, pre-commit hooks, CI scanning, IaC checks — so issues are caught when they're cheapest to fix and developers get fast feedback. It's highly effective and should be the default. But its limit is that it only catches *known* issues in *your* code/config before deploy; it can't stop zero-days, stolen credentials, insider misuse, or novel runtime attacks. So pair it with "shield-right": runtime detection (Falco), WAFs, anomaly monitoring, and incident response. Defense in depth spans the whole lifecycle, not just the left side.

Left and right, not either/or: Shift-left prevents known problems early; runtime defense catches the unknowns. Mature programs do both.

📚 **Official docs**

- OWASP DevSecOps Guideline

Q67 What is the cloud shared responsibility model and where do teams get it wrong?

Cloud Security

🎙 **Spoken answer · ~87 s**

The shared responsibility model divides security duties between the cloud provider and the customer. The provider is responsible for security of the cloud: physical data centres, hardware, the network infrastructure, the virtualisation layer, and the managed service internals. The customer is responsible for security in the cloud: their data, identity and access management, configuration of services, operating systems and applications they run, network controls, and encryption choices. The split moves with the service model: for infrastructure as a service you manage the OS and above; for platform services you manage the application and data; for software as a service you mainly manage identities, data, and configuration. Where teams get it wrong. Assuming the provider secures everything, for example leaving storage buckets public, or databases open to the internet, because it's a managed service. Weak identity hygiene: overprivileged roles, long-lived keys, missing MFA on root and admin accounts; identity is the most common cause of cloud breaches. Not patching the operating systems and applications on virtual machines. Not enabling logging, such as audit trails, so incidents can't be investigated. Misunderstanding managed Kubernetes responsibilities, like node patching and RBAC. Neglecting data classification, encryption, and backups. Misreading which defaults are secure. Assuming compliance certifications of the provider automatically make your workloads compliant.

Likely follow-ups

- How does the split change between IaaS and SaaS?
- What is the most common cause of cloud breaches?

```
provider: security OF the cloud (hardware, hypervisor, managed svc internals)
```

```
you: security IN the cloud (IAM, data, config, network, app)
```

certs.eknathalabs.com

The provider secures the underlying infrastructure; you secure what you put on it — identity, data, configuration, network rules, and application code. Teams get it wrong by assuming "the cloud is secure" and neglecting their half: leaving storage buckets public, over-permissive IAM, unencrypted data, open security groups. The line shifts by service type (more is yours for IaaS, less for SaaS), so you must know exactly where it falls for each service you use. Nearly all cloud breaches are customer-side misconfigurations, not provider failures.

Breaches live on your side: The provider rarely gets breached; customers misconfigure constantly. The model exists to clarify that config and IAM are your job.

📚 **Official docs**

- Kubernetes: Security overview (4C)
- AWS Shared Responsibility Model

Q68 How do you prevent and detect public storage bucket exposure?

Cloud Security

🎙 **Spoken answer · ~89 s**

Public storage exposure has caused many breaches, so I layer prevention, detection, and response. Prevention: enable account-level and organisation-level block-public-access settings, such as S3 Block Public Access, Azure's disabling of anonymous access, and GCS public access prevention with the organisation policy, so individual buckets can't be made public by mistake; use service control policies or organisation policies to enforce them; require bucket policies with least privilege, and prefer IAM over ACLs, with uniform bucket-level access; use presigned or signed URLs, or a CDN with origin access control, for content that must be shared, instead of public buckets; enforce through infrastructure-as-code modules with secure defaults and scan IaC in CI for public ACLs and policies. Detection: continuous posture monitoring with AWS Config rules and Access Analyzer for S3, Azure Policy and Defender, GCP Security Command Center, and CSPM tools that alert on any public or cross-account access; classify buckets holding sensitive data with tools like Macie or DLP; monitor data access logs for unusual downloads. Also scan external attack surface for exposed buckets named after your organisation. Response: automatically remediate by removing the public access and notifying the owner, investigate access logs to understand what was retrieved, and rotate any exposed credentials in the data. Maintain an inventory of buckets with owners and sensitivity.

Likely follow-ups

- What does block public access do at the account level?
- How do you share files without making a bucket public?

```
prevent: account-level public-access block (S3) / org policy
```

```
         IaC scan rules; deny public ACLs via SCP
```

```
detect:  Access Analyzer, Config rules, continuous CSPM scans
```

cors.EknathaLabs.com

Public buckets are a top breach cause. Prevent exposure with account/organization-level controls that block public access regardless of individual bucket settings (S3 Block Public Access, org policies/SCPs that deny making things public), plus IaC scanning that fails any Terraform creating a public bucket. Detect existing exposure with access analyzers and CSPM tools that continuously flag publicly-readable storage. The strongest control is the org-wide guardrail — it makes "accidentally public" structurally impossible, not just discouraged.

Make it impossible, not just discouraged: An account-level public-access block overrides any per-bucket mistake — far stronger than hoping each bucket is configured right.

📚 **Official docs**

- Amazon S3: Block Public Access

Q69 What is CSPM and what problems does it solve?

Cloud Security

🎙 **Spoken answer · ~88 s**

Cloud Security Posture Management continuously assesses cloud accounts and subscriptions for misconfigurations and compliance violations, by reading configuration data through provider APIs without agents. Tools include Wiz, Prisma Cloud, Orca, Microsoft Defender for Cloud, AWS Security Hub and Config, and GCP Security Command Center. Problems it solves. Visibility: the cloud changes quickly, and teams create resources faster than security can review; CSPM gives an inventory of assets across accounts and clouds. Misconfiguration detection: public storage, open security groups, unencrypted databases, disabled logging, missing MFA, overly permissive IAM, and exposed management ports. Compliance: mapping findings to CIS, PCI, SOC 2, HIPAA, and NIST frameworks, with reports and evidence. Drift: spotting changes that deviate from your baseline. Prioritisation: modern tools combine misconfigurations with vulnerabilities, identity permissions, network exposure, and data sensitivity to show attack paths and toxic combinations, such as an internet-exposed workload with admin rights and access to sensitive data, so teams fix what matters first. Remediation: guidance, tickets, and sometimes auto-fix. Limits: it's detective, not preventive, so combine with policy-as-code to prevent issues; it generates many findings requiring triage and ownership mapping; it sees configuration, not runtime behaviour, so pair with CWPP and detection tools; and it only helps if findings are routed to owners and fixed. Measure time to remediate.

Likely follow-ups

- How does attack path analysis improve prioritisation?
- Why isn't CSPM enough on its own?

certs.eknathalabs.com

```
CSPM = Cloud Security Posture Management
```

```
# continuously scans cloud config for misconfigs & compliance gaps
```

```
tools: Wiz, Prisma, native (Security Hub, SCC, Defender)
```

CSPM tools continuously inventory your cloud resources and evaluate their configuration against security best practices and compliance frameworks — flagging public buckets, over-permissive IAM, unencrypted databases, open security groups, missing logging. They solve the visibility problem at cloud scale: with thousands of resources across accounts changing constantly, no human can track posture manually. CSPM gives a prioritized, continuous view of "what's misconfigured and how risky is it," often with auto-remediation for common issues.

Visibility first: You can't secure what you can't see. CSPM's core value is continuous, account-wide inventory + misconfiguration detection at a scale humans can't match.

📚 **Official docs**

- Microsoft Defender for Cloud

Q70 How do you secure cloud network architecture (VPC design)?

Cloud Security

🎙 **Spoken answer · ~89 s**

Securing cloud network architecture starts with segmentation and minimal exposure. Design the VPC with separate subnets by tier: public subnets only for load balancers and gateways, private subnets for application workloads, and isolated subnets for databases with no internet route. Keep as little as possible public; place workloads behind load balancers and WAFs. Control traffic at multiple layers: security groups as stateful, least-privilege allow rules referencing other security groups rather than broad CIDRs; network ACLs as a coarse stateless backstop; and no 0.0.0.0/0 on management ports. Use separate VPCs or accounts for environments and sensitivity levels, connected through controlled hubs such as transit gateways with routing and firewall inspection, and avoid flat, fully meshed networks. Private connectivity to cloud services through VPC endpoints and Private Link, with endpoint policies, so traffic doesn't traverse the internet. Egress control through NAT and firewalls with allowlists, and DNS filtering. Administrative access via SSM Session Manager, IAP, or bastions with MFA instead of open SSH. Encrypt traffic in transit, with mTLS internally where needed. Observability: VPC flow logs, firewall logs, and DNS logs sent to a central place for detection. Add DDoS protection and WAF at the edge. Define the network as code with reviewed modules and policy checks, and test with regular reviews and penetration tests.

Likely follow-ups

- Why reference security groups instead of CIDR ranges?
- How do VPC endpoints improve security?

certs.eknathalabs.com

```
✓ private subnets for workloads; public only for LBs/bastions
```

```
✓ least-privilege security groups (no 0.0.0.0/0 on admin ports)
```

```
✓ egress filtering, VPC endpoints (keep traffic off public net)
```

```
✓ flow logs for visibility
```

Secure VPC design applies least privilege to the network. Place workloads in private subnets with no direct internet route; expose only load balancers in public subnets. Scope security groups tightly — never open SSH/RDP/database ports to 0.0.0.0/0. Use VPC endpoints so traffic to cloud services stays on the private backbone instead of the internet. Filter egress to limit exfiltration. Enable VPC flow logs for traffic visibility and incident investigation. Segment by trust level so a compromise in one tier can't freely reach others.

0.0.0.0/0 on port 22/3389 is a beacon: Admin ports open to the world get scanned and brute-forced within minutes. Use a bastion, SSM, or VPN — never world-open.

📚 **Start here (official docs)**

- Kubernetes security checklist

certs.eknathalabs.com

Q71 What is privilege escalation in cloud IAM and how do you find such paths? Cloud Security

🎙 Spoken answer · ~85 s

Privilege escalation in cloud IAM is when an identity with limited permissions uses them to gain more, for example through a permission that lets it modify IAM itself or act as a more powerful role. Common paths on AWS: iam:CreatePolicyVersion or iam:AttachUserPolicy to grant oneself admin; iam:PassRole combined with the ability to create a Lambda function or EC2 instance with a powerful role, then using that compute to act with the role's rights; sts:AssumeRole into roles with permissive trust policies; iam:UpdateAssumeRolePolicy; creating access keys for other users; and reading secrets or user data containing credentials. On GCP: iam.serviceAccounts.actAs, serviceAccounts.getAccessToken or implicit delegation, setIamPolicy, creating service account keys, and cloudfunctions deployment with a privileged service account. On Azure: role assignment write permissions, managed identity abuse, and Automation accounts. Finding such paths: use tools that analyse the permission graph, like PMapper and Cloudsplaining for AWS, Pacu for testing, ScoutSuite, Prowler, CloudFox, GCP's IAM policy analysis and Gcp-IAM-Privilege-Escalation research tools, and BloodHound-style graph tools such as AzureHound and Cartography, plus commercial CSPM attack path features. Mitigation: avoid wildcard permissions, restrict iam:PassRole to specific roles, use permission boundaries and SCPs, require conditions, alert on IAM changes in audit logs, and review regularly. Treat IAM-modifying permissions as admin-equivalent.

Likely follow-ups

- Why is iam:PassRole risky?
- What tools map escalation paths?

```
# e.g. iam:PassRole + lambda:CreateFunction → assume any role
```

```
# or iam:CreatePolicyVersion → rewrite own permissions
```

```
tools: PMapper, Cloudsplaining, Pacu, IAM Access Analyzer
```

Cloud IAM escalation happens when a seemingly-limited identity holds a permission combination that lets it grant itself more — e.g. `iam:PassRole` plus a compute-creation permission lets it run code as a more privileged role; `iam:CreatePolicyVersion` lets it rewrite its own policy. These chains aren't obvious from individual permissions. Tools like PMapper and Cloudsplaining map the IAM graph to surface escalation paths. Defense: avoid the dangerous combinations, use permission boundaries, and audit for known escalation patterns.

Combinations, not single grants: Each permission looks fine alone; the risk is the chain. Graph-based analysis (PMapper) is how you find what manual review misses.

📚 Official docs

- Kubernetes: RBAC good practices

certs.eknathalabs.com

Q72 How do you handle cloud audit logging for security (CloudTrail/equivalent)?

Cloud Security

🎙 Spoken answer · ~86 s

Cloud audit logging records who did what, when, and from where for control plane and data plane actions, and it's foundational for detection, investigation, and compliance. On AWS, CloudTrail records API calls; on Azure, the Activity Log and diagnostic logs; on GCP, Cloud Audit Logs with Admin Activity, Data Access, and System Event logs. Setup. Enable everywhere: organisation-level trails covering all accounts and regions, including management events and, for sensitive resources, data events such as S3 object access, with the cost trade-off considered, since data access logs can be large and are off by default in some clouds. Centralise: send logs to a dedicated, locked-down log archive account or project owned by the security team, so that attackers who compromise a workload account can't delete evidence. Protect integrity: write-once storage such as S3 Object Lock, log file validation, encryption with KMS, and restricted access, with separate permissions for readers and administrators. Retain for the period required by compliance and investigations, with hot and cold tiers. Analyse and alert: stream to a SIEM or analytics platform, and create detections for high-risk events like root account use, IAM policy changes, disabling logging, console logins without MFA, new access keys, security group changes, and unusual API calls or geographies.

Likely follow-ups

- Why centralise logs in a separate account?
- Which events would you alert on first?

```
✓ enable across ALL regions + management AND data events
```

```
✓ ship to a separate, locked-down logging account (immutable)
```

```
✓ detection rules + alerting on anomalous API calls
```

Audit logs (CloudTrail, GCP Audit Logs, Azure Activity Log) record every API call — the forensic backbone for detecting and investigating incidents. Best practice: enable in all regions (attackers use unused ones), capture both management and sensitive data events, and forward logs to a separate, tightly-controlled logging account with immutability/object-lock so an attacker who compromises the main account can't delete the evidence. Then build detections on top (unusual region, root usage, mass deletion, new IAM users) — logs no one watches don't stop anything.

Protect the logs from the attacker: The first thing a smart intruder does is disable/delete logging. A separate immutable logging account preserves the trail even if the main account falls.

📚 Official docs

- Linux audit documentation (auditd)
- AWS CloudTrail

certs.eknathalabs.com

Q73 What is the risk of long-lived access keys and what replaces them?

Cloud Security

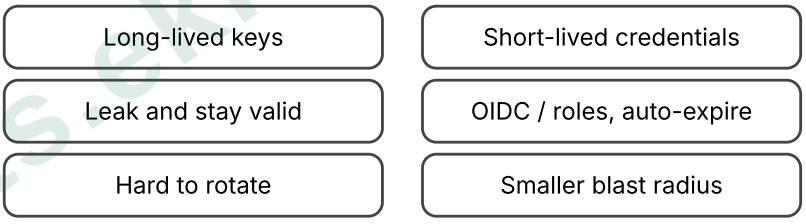
🎙 Spoken answer · ~85 s

Long-lived access keys are static credentials that don't expire unless someone rotates them. Risks: they get leaked in code repositories, logs, container images, laptops, or chat; they're copied and shared, so ownership and usage are unclear; they're rarely rotated; they work from anywhere, so a stolen key is immediately usable by an attacker; and they're a favourite target for scanners and attackers, who find exposed keys within minutes, leading to cryptomining, data theft, and large bills. Many major breaches began with a leaked key. Replacements. For humans: federated single sign-on with short-lived sessions, such as AWS IAM Identity Center, Azure Entra ID, and Google Cloud Identity, with MFA; engineers authenticate and get temporary credentials via the CLI. For workloads on cloud compute: instance profiles, IAM roles for service accounts and pod identity, managed identities, and attached service accounts, which provide automatically rotated temporary credentials. For CI/CD and external systems: OIDC federation or workload identity federation, so pipelines exchange their platform identity token for short-lived cloud credentials; for on-premises workloads, IAM Roles Anywhere or similar. When static keys are unavoidable, restrict their permissions and source IPs, store them in a secrets manager, rotate them automatically, monitor usage, and alert on age and unused keys.

Likely follow-ups

- How do workloads get credentials without keys?
- How can you stop teams creating long-lived keys?

🗺 Access keys




```
replace static keys with:
  workload identity / IRSA / instance roles (compute)
  OIDC federation (CI/CD, external systems)
  SSO + temporary role assumption (humans)
```

Long-lived access keys are the single most common cloud breach vector — they leak via Git, logs, laptops, and screenshots, never expire on their own, and are hard to attribute and rotate. Replace them with short-lived, automatically-rotated credentials: instance/pod roles for compute, OIDC federation for CI and external systems, and SSO with temporary role assumption for humans. Where static keys are unavoidable, rotate frequently, scope minimally, and monitor usage. The goal is zero long-lived keys anywhere.

certs.eknathalabs.com

Hunt and kill static keys: Inventory every long-lived key, replace with federation/roles, and alert on any new ones created. Each one removed closes a major exposure.

📚 **Official docs**

- AWS IAM: Temporary credentials (STS)

Q74 How do you secure serverless functions (Lambda/Cloud Functions)?

Cloud Security

🎙 **Spoken answer · ~86 s**

Serverless functions shift infrastructure security to the provider, but application, identity, and configuration risks remain, and the attack surface changes. Key practices. Least-privilege execution roles: give each function its own role with only the permissions it needs, because a shared, broad role is the biggest risk; avoid wildcards. Input validation: functions are triggered by many event sources, like HTTP, queues, storage events, and streams, and each can carry attacker-controlled data, so validate and sanitise all event input to prevent injection. Secrets: don't store them in environment variables or code; fetch from a secrets manager at cold start with caching, using the function's role. Dependencies: scan and minimise packages, pin versions, and use lockfiles, since third-party libraries are a large part of the code; keep runtimes updated. Network: put functions in a VPC only when they need private resources, use egress controls, and secure API gateways with authentication, authorisation, rate limiting, and WAF. Limit concurrency and timeouts to reduce denial-of-wallet and abuse. Logging and monitoring: enable structured logs and tracing, detect anomalous invocations, and avoid logging sensitive data. Protect event sources, for example restrict who can invoke the function through resource policies and who can publish to the triggering queue. Use code signing and controlled deployment pipelines.

Likely follow-ups

- What does denial-of-wallet mean?
- Why is a per-function role important?

certs.eknathalabs.com

```
✓ minimal per-function IAM role (not a shared broad role)
```

```
✓ scan dependencies (functions are dependency-heavy)
```

```
✓ secrets from a secrets manager, not env vars
```

```
✓ validate all input/event sources; set timeouts/concurrency caps
```

Serverless shifts but doesn’t remove security work. Each function should have its own least-privilege execution role (a shared over-broad role means one compromised function exposes everything). Functions pull in many dependencies, so dependency scanning matters as much as anywhere. Pull secrets from a

secrets manager at runtime rather than embedding in env vars. Validate every event/input — event sources are attack surface. Set timeouts and concurrency limits to bound abuse and cost (denial-of-wallet). The provider patches the runtime; the code, deps, IAM, and config are yours.

Per-function roles: The convenient shared role is the trap — it gives every function (and any attacker who lands in one) maximum blast radius.

📚 **Official docs**

- AWS Lambda Developer Guide

Q75 What is data exfiltration risk in the cloud and how do you mitigate it?

Cloud Security

🎙 **Spoken answer · ~88 s**

Data exfiltration is the unauthorised transfer of data out of the organisation. In the cloud, it can happen through compromised credentials copying data to an attacker's storage, public bucket exposure, overly permissive sharing of snapshots or images with external accounts, malicious insiders, compromised workloads sending data out over the network, DNS tunnelling, and misuse of legitimate services like the attacker's own cloud account. Mitigations in layers. Identity: least privilege, MFA, short-lived credentials, and conditional access limiting access to expected networks and devices. Data perimeters: use VPC Service Controls on GCP, AWS data perimeter controls with SCPs and resource policies conditioned on the organisation ID so data can only be accessed by principals and networks within your organisation, and private endpoints with endpoint policies to prevent copying to external accounts. Prevent public or external sharing of buckets, snapshots, and images with organisation policies. Network: egress filtering with allowlists, DNS filtering, and proxy inspection. Encryption with customer-managed keys, so that revoking key access renders copied data unreadable, and tight key policies. Data protection: classify sensitive data, use DLP scanning and tokenisation or masking, and minimise stored data. Detection: monitor data access logs for unusual volumes and access patterns, alert on large downloads, cross-account copies, and unusual egress with flow logs and UEBA tools.

Likely follow-ups

- What is a data perimeter?
- How does customer-managed encryption limit exfiltration?

```
✓ egress controls + VPC endpoints (no path to attacker buckets)
```

```
✓ DLP scanning, sensitive-data discovery (Macie/DLP API)
```

```
✓ alert on large/anomalous data transfers; scope read access
```

Once inside, attackers want your data out. Mitigate by limiting outbound paths — egress filtering and VPC endpoints so resources can't freely reach arbitrary internet destinations or external buckets; least-privilege so few identities can read sensitive datasets in bulk; DLP/data-discovery tools (Macie, DLP API) to know

ceris.eknathalabs.com

where sensitive data lives and detect exposure; and monitoring/alerting on anomalous transfer volumes or unusual destinations. Combine prevention (limit access and egress) with detection (spot the bulk read/transfer).

Bound both ends: Limit who can read data *and* where data can go. Either control alone leaves an exfil path; together they close it.

📚 **Official docs**

- Google Cloud: VPC Service Controls (data exfiltration)

Q76 How do you implement multi-account / landing-zone strategy for security?

Cloud Security

🎙 **Spoken answer · ~88 s**

A multi-account landing-zone strategy uses separate accounts, subscriptions, or projects as security boundaries, organised under an organisation hierarchy with centralised governance. Benefits: blast radius containment, since a compromise in one account doesn't automatically spread; separate permissions and billing per workload or environment; and guardrails applied by hierarchy. Typical structure on AWS using Organizations and Control Tower: a management account kept nearly empty; a security organisational unit with a log archive account for immutable centralised logs and a security tooling account for GuardDuty, Security Hub, and the audit role; an infrastructure OU with shared networking and shared services accounts; workload OUs separated by environment, with production accounts distinct from non-production, and often per application or team; and a sandbox OU with strict budget limits. Equivalent concepts are management groups and subscriptions on Azure and folders and projects on GCP. Security controls: SCPs or organisation policies as preventive guardrails, such as denying disabling of CloudTrail, restricting regions, and preventing public access; organisation-wide logging and detection services enabled automatically in each new account; centralised identity through SSO with permission sets assigned to groups, instead of local users; account vending automation through infrastructure as code, so new accounts arrive with baselines for networking, logging, tagging, and security tooling; a hub-and-spoke network design; and break-glass procedures.

Likely follow-ups

- What goes in a log archive account?
- How does an account vending process enforce baselines?

```
separate accounts by env/team/blast-radius + central guardrails:
```

```
  org SCPs (hard caps), centralized logging account,
```

```
  shared security tooling account, automated account vending
```

cors.eknathalabs.com

A multi-account landing zone uses account boundaries as strong isolation: separate prod from dev, separate teams, so a compromise or mistake in one account can't reach others (the account is a hard blast-radius boundary). Layer organization-wide guardrails: Service Control Policies that cap what any account can do (deny disabling logging, restrict regions), a centralized immutable logging account, and a shared security-tooling account. Automate account creation with baked-in baselines (Control Tower, landing-zone frameworks) so every new account starts secure.

Accounts as blast radius: The account boundary is one of the cloud's strongest isolation primitives — use it to contain failures and breaches, not just for billing.

📚 **Official docs**

- AWS Organizations: What is AWS Organizations?

Q77 What is SSRF and why is it especially dangerous in the cloud?

Cloud Security

🎙 **Spoken answer · ~88 s**

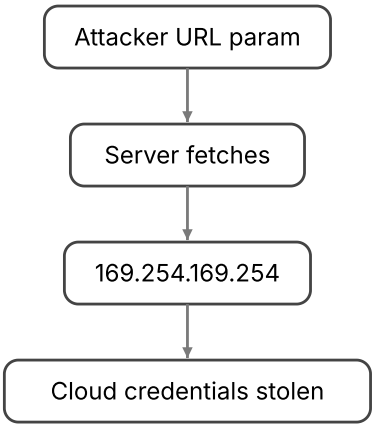
Server-Side Request Forgery is when an attacker tricks a server-side application into making requests to a destination of the attacker's choosing, for example through a URL parameter that the application fetches, such as an image importer, webhook, or PDF generator. The request originates from the server, so it can reach internal resources the attacker can't access directly. It's especially dangerous in the cloud because of the metadata service: at 169.254.169.254, instances can fetch temporary credentials for their attached role. An SSRF bug can be used to query that endpoint and exfiltrate the credentials, giving the attacker the cloud permissions of the instance, as in the 2019 Capital One breach. It can also reach internal admin panels, databases, Kubernetes API servers, and other services that trust internal traffic. Defences. Cloud-level: enforce IMDSv2 on AWS, which requires a session token obtained with a PUT request and so blocks simple GET-based SSRF, with a hop limit of one for containers; use workload identity on Kubernetes and block metadata access from pods with network policies; give instance roles minimal permissions so stolen credentials do little. Application-level: validate and allowlist destination URLs and domains, resolve DNS and check the resulting IP isn't private, loopback, or link-local, guard against redirects and DNS rebinding, and don't return raw responses.

Likely follow-ups

- Why is the metadata service the prime SSRF target?
- How does DNS rebinding bypass URL validation?

cers.eknathalabs.com

🗺 **SSRF to metadata**




```
# attacker tricks server into requesting internal URLs
```

```
GET /fetch?url=http://169.254.169.254/latest/meta-data/iam/...
```

```
# → steals instance role credentials
```

Server-Side Request Forgery makes a server fetch a URL of the attacker's choosing. In the cloud it's catastrophic because the metadata endpoint (`169.254.169.254`) returns the instance's IAM credentials — so an SSRF often escalates straight to cloud account compromise (this was the Capital One breach pattern). Defend by enforcing IMDSv2 (token + hop limit defeats simple SSRF), blocking pod/instance access to metadata where not needed, validating/allowlisting outbound URLs in apps, and using least-privilege roles so stolen credentials are limited.

SSRF + metadata = breach: The combination turned a web-app bug into the Capital One incident. IMDSv2 and metadata access controls specifically break this chain.

📚 **Official docs**

- OWASP Top Ten

certs.eknathalabs.com

Q78 How do you encrypt data in transit and at rest across cloud services?

Cloud Security

🎙 **Spoken answer · ~83 s**

Encryption in transit protects data moving across networks; encryption at rest protects stored data. In transit: enforce TLS 1.2 or higher on all public endpoints, with load balancers terminating or passing through TLS with strong cipher suites, and managed certificates from ACM, Google-managed certs, or Let's Encrypt, with automatic renewal. Internally, encrypt service-to-service traffic with mTLS through a service mesh or application libraries, enforce TLS on database connections, and use HTTPS-only policies on storage and APIs, for example secure transfer required on Azure Storage and bucket policies denying non-TLS requests on S3. Use VPN or private links with encryption for hybrid connectivity, and consider MACsec or IPSec over Direct Connect. At rest: use provider-managed encryption, which is on by default for many services, and for greater control customer-managed keys in KMS, Cloud KMS, or Key Vault, with key policies limiting who can use and administer keys, automatic rotation, and audit logging of key use. Cover everything: block volumes, object storage, databases, snapshots, backups, queues, and logs. Use envelope encryption and consider field-level or application-layer encryption for highly sensitive data. Enforce through policy: SCPs or organisation policies and config rules requiring encryption, and IaC scans that flag unencrypted resources.

Likely follow-ups

- What does a customer-managed key add over the default?
- How do you enforce TLS-only access to storage?

```
at rest:    KMS-backed encryption on every store (default-on)
```

```
in transit: TLS everywhere, incl. internal service traffic
```

```
keys:       customer-managed keys (CMK) for control + rotation
```

Encrypt at rest on every data store (databases, object storage, volumes, backups, snapshots) using KMS — ideally enforced as a default via org policy so nothing unencrypted can be created. Encrypt in transit with TLS for all traffic, including internal/east-west (don’t trust the private network). Use customer-managed keys when you need control over rotation, access policy, and the ability to revoke. Encryption protects against storage-media theft, snapshot leaks, and misconfigured access — but it’s only as strong as your key management.

Enforce, don’t hope: An org policy that denies creating unencrypted resources beats relying on every engineer to tick the encryption box.

📚 **Official docs**

- Kubernetes: Encrypting Secret data at rest

Certs.eknathalabs.com

Q79 What is a security group vs NACL and how do they layer?

Cloud Security

🎙 Spoken answer · ~82 s

Security groups and network ACLs are both firewalls in AWS-style VPCs, but they act at different levels. A security group is attached to an elastic network interface, or instance, and is stateful: if you allow inbound traffic, the response is automatically allowed, and vice versa. It supports only allow rules, with everything else denied by default, and rules can reference other security groups, which makes tiered designs easy, such as allowing the database group to accept connections only from the app group. All rules are evaluated together. A network ACL is attached to a subnet, and is stateless: you must explicitly allow return traffic in both directions, including ephemeral port ranges. It supports both allow and deny rules, evaluated in numbered order, first match wins, and applies to all traffic entering or leaving the subnet. They layer: traffic must pass the NACL at the subnet boundary and the security group at the instance. In practice, security groups carry the main, fine-grained policy because they're simpler and stateful; NACLs act as a coarse, secondary defence, for instance explicitly denying known bad IP ranges, or enforcing broad subnet-level restrictions, and are more difficult to manage because of the statelessness.

Likely follow-ups

- Why do NACLs need ephemeral port rules?
- When would you use a deny rule in a NACL?

```
Security Group: stateful, instance/ENI level, allow-only
```

```
NACL:          stateless, subnet level, allow AND deny rules
```

Security groups are stateful firewalls at the instance/ENI level — return traffic is automatically allowed, and they support only allow rules. Network ACLs are stateless firewalls at the subnet level — you must allow both directions explicitly, and they support deny rules. They layer: NACLs give coarse subnet-wide controls (including explicit denies, e.g. blocking a malicious IP range), security groups give fine-grained per-resource allow-listing. Use security groups as the primary control and NACLs for broad subnet guardrails.

Stateful vs stateless is the key distinction: SGs "remember" connections (allow the reply automatically); NACLs don’t, so you handle both directions.

📚 Official docs

- Amazon VPC: Security groups

certs.eknathalabs.com

Q80 How do you detect cryptomining / compromised cloud resources?

Cloud Security

🎙 **Spoken answer · ~89 s**

Cryptomining is a common outcome of compromised cloud credentials or workloads, because attackers monetise compute quickly. Detection signals. Cost and usage: sudden spikes in compute spend, new instances or GPU instances in unusual regions, or large autoscaling groups, caught by billing alerts, budgets, and anomaly detection. Resource metrics: sustained high CPU or GPU usage on workloads that normally idle. Network: outbound connections to mining pools and known malicious IPs or domains, Stratum protocol traffic, seen in VPC flow logs, DNS logs, and firewall logs. Provider threat detection: GuardDuty, Microsoft Defender for Cloud, and GCP Security Command Center's Event Threat Detection include findings for crypto-related activity. Audit logs: unusual API activity such as RunInstances in new regions, creation of new access keys or users, IAM changes, or disabled logging. Runtime tools like Falco detect suspicious processes such as xmrig in containers. Image and workload scanning find mining malware in images. Prevention: least-privilege IAM, no long-lived keys, MFA, SCPs restricting regions and instance types, service quotas set low, and egress filtering. Response: isolate the resource while preserving evidence, revoke exposed credentials, terminate unauthorised resources across all regions, check for persistence like new IAM users, roles, or Lambda backdoors, find the initial access vector, rotate credentials, and review the bill with the provider. Practise through runbooks.

Likely follow-ups

- Which billing signals suggest compromise?
- What persistence mechanisms do you look for afterwards?

```
signals: CPU spikes, unusual egress, new instances in odd regions,
         calls to mining pools, sudden cost increases
tools: GuardDuty / SCC / Defender threat detection + billing alerts
```

Cryptomining is a common goal after cloud compromise (free compute on your bill). Detect it through behavioral signals: sustained high CPU on unexpected instances, outbound connections to known mining pools, instances spun up in regions you never use, and — often the first alarm — a sudden spike in the cloud bill. Cloud-native threat detection (GuardDuty, Security Command Center, Defender) flags these patterns, including credential abuse and reconnaissance. Pair with billing/cost anomaly alerts, which frequently catch compromise before security tooling does.

The bill is a security sensor: A surprise cost spike is often the loudest indicator of compromise. Wire cost anomaly alerts into your security response.

📚 **Official docs**

- NIST: Computer Security Incident Handling Guide (SP 800-61)

certs.eknathalabs.com

Q81 What is confidential computing and when is it relevant?

Cloud Security

🎙 Spoken answer · ~80 s

Confidential computing protects data in use, closing the gap left by encryption at rest and in transit. Data is normally decrypted in memory while being processed, where a privileged attacker, such as a malicious administrator, a compromised hypervisor, or other tenants exploiting a host flaw, could read it. Confidential computing runs workloads in a hardware-based trusted execution environment, using technologies like AMD SEV-SNP, Intel TDX and SGX, and Arm CCA, where memory is encrypted with keys managed by the CPU and inaccessible to the host operating system, hypervisor, and even the cloud provider. A key feature is remote attestation: the hardware produces signed evidence of what code and configuration is running in the enclave, letting a relying party verify it before releasing secrets or data. Cloud offerings include AWS Nitro Enclaves, Azure confidential VMs and containers, and GCP Confidential VMs and Confidential GKE nodes. Relevant use cases: highly regulated data in finance, healthcare, and government that must stay protected from the cloud operator; multi-party analytics and collaboration where parties share data without revealing it to each other; protecting proprietary models and keys; processing sensitive data on untrusted infrastructure; and sovereignty requirements.

Likely follow-ups

- What does remote attestation prove?
- What data states does confidential computing protect?

```
# encrypt data IN USE (in memory), not just at rest/in transit
```

```
hardware enclaves: Intel SGX, AMD SEV, confidential VMs
```

Confidential computing protects data *while it's being processed* — the third state beyond at-rest and in-transit. Using hardware-based trusted execution environments (enclaves, confidential VMs), data stays encrypted in memory and is processed in an isolated region the host OS and hypervisor can't inspect. It's relevant for highly sensitive workloads where you must protect data even from the infrastructure operator or a compromised host — regulated data, multi-party computation, or processing secrets you can't expose to the cloud provider's plane. It's a specialized control, not a general default.

The third state: Most encryption covers rest and transit; confidential computing closes the "in use" gap for the highest-sensitivity workloads.

📚 Official docs

- Confidential Computing Consortium

certs.eknathalabs.com

Q82 How do you secure CI/CD access to cloud (deployment credentials)?

Cloud Security

🎙 Spoken answer · ~81 s

CI/CD access to cloud is a high-value target, so the goal is no standing credentials and minimal scope. Preferred approach: OIDC federation. The CI platform issues a signed identity token per job containing claims like repository, branch, and environment; the cloud provider is configured to trust that issuer and exchange the token for short-lived credentials, with a trust policy that restricts to the specific repository, branch or environment, and workflow. Examples: GitHub Actions with AWS IAM roles via sts:AssumeRoleWithWebIdentity, Azure federated credentials, and GCP Workload Identity Federation. This removes stored keys entirely. Scope tightly: separate roles for build, test, and deploy and per environment; production deploy roles only assumable from protected branches or approved environments; permissions limited to what deployment requires, for example updating a specific service, not administrator access; consider a permission boundary. Add approvals and separation of duties for production. Protect the pipeline: restrict who can modify workflow files, pin third-party actions, use ephemeral runners, avoid untrusted code running with deploy roles, and never print credentials. Prefer deployment mechanisms that reduce CI's power: GitOps, where CI only updates a repository and an in-cluster agent pulls changes, so CI needs no cluster credentials.

Likely follow-ups

- What claims does the trust policy check?
- How does GitOps reduce CI's cloud privileges?

```
✓ OIDC federation → short-lived creds, no stored keys
```

```
✓ scope the deploy role to exactly what it deploys
```

```
✓ separate roles per environment; require approval for prod
```

```
✓ restrict which branches/repos can assume the role
```

CI/CD holds the keys to production, making it a prime target. Use OIDC federation so the pipeline exchanges a signed identity token for short-lived cloud credentials — no long-lived keys stored in the CI system. Scope the deployment role tightly to the resources it actually manages, use separate roles per environment (dev pipeline can’t touch prod), restrict the trust policy to specific repos/branches, and require human approval for production deploys. Audit every deployment. A compromised pipeline with a broad, key-based cloud role is a worst-case scenario.

OIDC + scoped trust: Tie the cloud role to a specific repo and branch via the OIDC trust policy — even a stolen workflow can’t deploy from elsewhere.

📚 Official docs

- GitHub Actions: Security hardening

certs.eknathalabs.com

Q83 What is SAST and where does it fit in the pipeline?

AppSec

🎙 Spoken answer · ~88 s

Static Application Security Testing analyses source code, bytecode, or binaries without running the application, looking for vulnerability patterns such as SQL injection, cross-site scripting, hardcoded secrets, insecure cryptography, and unsafe deserialisation. Tools: Semgrep, CodeQL, SonarQube, Checkmarx, Fortify, and language-specific linters. It works by parsing code into abstract syntax trees and tracking data flow from sources of untrusted input to dangerous sinks, applying rules. Where it fits: as far left as possible. In the IDE, for immediate feedback; in pre-commit hooks for fast, lightweight rules; in pull request CI as a required check with results annotated on the changed lines, which is the sweet spot because developers have context; and in scheduled full scans of the main branch. Strengths: finds issues early, covers all code paths including untested ones, pinpoints exact lines, and scales across repositories. Limits: false positives and noise, which need tuning and prioritisation; false negatives, since it misses runtime, configuration, and business-logic flaws and issues from dependencies and deployed environments; language and framework coverage varies; and scans can be slow on large codebases, so use incremental scanning. To make it effective: start with a small set of high-confidence rules, fail builds only on high-severity new findings, provide fix guidance, allow suppressions with justification, and track findings over time.

Likely follow-ups

- What is data flow analysis in SAST?
- Why annotate findings on pull requests?

```
SAST = Static Application Security Testing (analyzes source code)
```

```
tools: Semgrep, CodeQL, SonarQube; runs in CI / on PR
```

SAST analyzes source code (without running it) for security flaws — SQL injection, hardcoded secrets, unsafe deserialization, path traversal patterns. It runs early, ideally on every PR, giving developers fast feedback at the cheapest point to fix. Strengths: broad coverage, finds issues before deploy, integrates into CI. Limits: false positives (flags code that isn't actually exploitable) and it can't see runtime/configuration issues. Tune the rules to your stack and triage findings so noise doesn't train developers to ignore it.

Tune to keep trust: Untuned SAST drowns devs in false positives and gets ignored. Curate rules for your stack so signal stays high.

📚 Official docs

- GitHub: About code scanning
- Semgrep documentation

certs.eknathalabs.com

Q84 What is DAST and how does it complement SAST?

AppSec

🎙 Spoken answer · ~88 s

Dynamic Application Security Testing tests a running application from the outside, as an attacker would, sending crafted requests and analysing responses to find vulnerabilities such as injection, cross-site scripting, authentication and session weaknesses, misconfigured headers, exposed endpoints, and server misconfigurations. Tools include OWASP ZAP, Burp Suite, and commercial scanners. It works with the compiled, deployed application in a test or staging environment, so it needs a running instance, authentication configured for the scanner, and an API specification or crawler to discover endpoints. How it complements SAST: SAST sees the source and finds code-level issues early but can't see runtime behaviour, configuration, or how components interact; DAST sees the deployed system, including the web server, framework, deployment configuration, and third-party integrations, and finds issues SAST misses, with lower false positives since it demonstrates actual exploitability, and it's language-agnostic. Limits: it runs later in the cycle, is slower, can't point to the exact line of code, has limited coverage of paths it doesn't discover, and may disrupt the target or create junk data, so run it against non-production. Pipeline fit: run quick baseline scans on every deploy to staging, deeper authenticated scans nightly or before release, and API scans from the OpenAPI definition. Triage results, suppress false positives, and feed findings back to developers.

Likely follow-ups

- Why does DAST have fewer false positives?
- Why run DAST against staging rather than production?

```
DAST = Dynamic Application Security Testing (tests running app)
```

```
tools: OWASP ZAP, Burp; probes the live app like an attacker
```

DAST tests a *running* application from the outside, sending malicious inputs and observing behavior — finding issues only visible at runtime: authentication flaws, misconfigurations, injection that actually executes, exposed endpoints. It complements SAST: SAST sees the code but not runtime behavior; DAST sees behavior but not the code. SAST finds the vulnerable line; DAST confirms it's actually exploitable through the real app. Run DAST against staging in the pipeline. Its limits: slower, needs a deployed app, and may miss code paths it doesn't exercise.

Inside-out vs outside-in: SAST reads the code, DAST attacks the app. Together they cover both what the code *says* and what the app *does*.

📚 Official docs

- OWASP ZAP
- GitHub: About code scanning
- Semgrep documentation

certs.eknathalabs.com

Q85 Walk through the OWASP Top 10 categories at a high level.

AppSec

🎙 Spoken answer · ~83 s

The OWASP Top 10 is a regularly updated awareness list of the most critical web application security risks. The 2021 edition's categories. A01 Broken Access Control: users acting outside intended permissions, such as accessing other users' records through insecure direct object references. A02 Cryptographic Failures: weak or missing encryption of sensitive data. A03 Injection: untrusted data interpreted as commands, including SQL, OS command, and cross-site scripting. A04 Insecure Design: missing or ineffective security controls at the design level, addressed by threat modelling. A05 Security Misconfiguration: default accounts, verbose errors, open cloud storage, unhardened servers. A06 Vulnerable and Outdated Components: known-vulnerable libraries and frameworks. A07 Identification and Authentication Failures: weak passwords, broken session management, missing MFA. A08 Software and Data Integrity Failures: insecure deserialisation, unsigned updates, and compromised CI/CD pipelines. A09 Security Logging and Monitoring Failures: not detecting or responding to attacks. A10 Server-Side Request Forgery. A newer 2025 edition updates the list, with supply chain failures and mishandling of exceptional conditions emphasised, so check the current version. How I use it: as a shared vocabulary for training developers, a checklist for threat modelling and code review, a basis for choosing SAST and DAST rules, and a way to prioritise.

Likely follow-ups

- Which categories relate most to the supply chain?
- What is OWASP ASVS for?

```
Broken Access Control, Cryptographic Failures, Injection,
```

```
Insecure Design, Security Misconfiguration, Vulnerable Components,
```

```
Auth Failures, Software/Data Integrity, Logging Failures, SSRF
```

The OWASP Top 10 is the consensus list of the most critical web app risks. Highlights: **Broken Access Control** (the #1 risk — users accessing what they shouldn't), **Cryptographic Failures** (weak/missing encryption), **Injection** (SQL/command injection from untrusted input), **Security Misconfiguration**, **Vulnerable & Outdated Components** (the supply-chain link), **Identification & Auth Failures**, **Software & Data Integrity Failures** (insecure CI/CD, unsigned updates), and **SSRF**. It's a prioritization framework, not exhaustive — a shared vocabulary for the risks that matter most.

Access control leads the list: Broken access control is consistently the most prevalent real-world web vulnerability — authorization bugs, not exotic exploits.

📚 Official docs

- OWASP Top Ten

certs.eknathalabs.com

Q86 How do you prevent injection attacks (SQL, command, etc.)?

AppSec

🎙 Spoken answer · ~87 s

Injection happens when untrusted input is mixed into a command or query so that the interpreter treats data as code, giving attackers control. Types: SQL, NoSQL, OS command, LDAP, XPath, template injection, and cross-site scripting, which is injection into the browser. The core defence is to keep data separate from code. For SQL: use parameterised queries or prepared statements, where the query structure is fixed and values are bound separately; use ORMs properly, avoiding raw string concatenation; stored procedures only if they don't build dynamic SQL; and apply least-privilege database accounts, so a successful injection has limited impact. For OS commands: avoid invoking shells with user input; use language APIs that take argument arrays instead of command strings, and allowlist permitted values. For XSS: contextual output encoding, a content security policy, and frameworks that escape by default. Input validation as a secondary layer: allowlist expected formats, types, lengths, and ranges server-side, rather than trying to blacklist bad characters. For NoSQL and others: use safe query builders and validate types, since objects can be injected into JSON queries. Additional layers: a WAF as a stopgap, error messages that don't leak details, static analysis and DAST to find injection flaws, code review focused on sinks, and security testing in CI.

Likely follow-ups

- Why are parameterised queries safe?
- Why is input validation only a secondary defence?

```
# NEVER concatenate untrusted input into commands/queries
```

```
cur.execute("SELECT * FROM u WHERE id=%s", (uid,))  # parameterized
```

```
subprocess.run(["ping","-c","1", host])             # arg list
```

Injection happens when untrusted input is interpreted as code/commands. The universal fix is separating code from data: parameterized queries (the database treats parameters as pure data, never SQL), argument lists instead of shell strings for commands, and safe APIs over string concatenation. Add input validation (allowlists) as defense in depth, and least-privilege so a successful injection has limited reach (the DB user can’t drop tables). The root cause is always the same — building executable strings from input — so the fix is always: don’t.

Validation is not enough alone: Allowlisting input helps, but parameterization is the actual fix — it makes injection structurally impossible, not just filtered.

📚 Official docs

- OWASP Top Ten

certs.eknathalabs.com

Q87 What is secure-by-default and threat modeling in design?

AppSec

🎙 Spoken answer · ~82 s

Secure by default means a system's out-of-the-box configuration is the secure one: users must deliberately opt out of protections, rather than opt in. Examples: encryption enabled, least-privilege roles, no default passwords, TLS required, private by default, and secure templates and golden paths where hardened settings are preset. Secure by design goes further, building security into architecture. Threat modelling is the structured practice of identifying what can go wrong before building. Steps. Define what you're protecting and scope: draw a data flow diagram with components, data stores, trust boundaries, and entry points. Identify threats systematically using a framework like STRIDE: spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege, applied to each element and flow; other methods include PASTA and attack trees. Assess risk: likelihood and impact, to prioritise. Decide mitigations: design changes, controls, or accepted risks, and record them with owners. Validate: turn mitigations into requirements, tests, and monitoring, and revisit as the design changes. Make it practical: run short sessions with developers, architects, and security together early in design and for significant changes, keep diagrams in the repository, and use tools like OWASP Threat Dragon. Embed in the process through design review checklists.

Likely follow-ups

- What does STRIDE stand for?
- What is a trust boundary?

```
# threat modeling: STRIDE
```

```
Spoofing, Tampering, Repudiation, Info disclosure,
```

```
Denial of service, Elevation of privilege
```

Secure-by-default means the safe configuration is the out-of-the-box one — deny by default, encryption on, minimal permissions — so doing nothing is safe and insecurity requires a conscious override. Threat modeling is the design-time practice of systematically asking "how could this be attacked?" before building. STRIDE is a common framework: walk through each category (Spoofing, Tampering, etc.) against your design's components and data flows, identify threats, and decide mitigations. Doing this early (Insecure Design is an OWASP category) catches whole classes of flaws that scanning never would.

Design beats detection: Threat modeling prevents architectural flaws that no scanner can find later — the cheapest vulnerability is the one you designed out.

📚 Official docs

- MITRE ATT&CK for Containers

certs.eknathalabs.com

Q88 How do you secure APIs (authn, authz, rate limiting, validation)?

AppSec

🎙 Spoken answer · ~89 s

Securing APIs layers several controls. Authentication: verify identity using standards like OAuth 2.0 and OpenID Connect with short-lived access tokens, validate the signature, issuer, audience, and expiry of JWTs, use mTLS or signed requests for service-to-service calls, and avoid API keys as the only credential for sensitive operations. Authorisation: enforce on every request, server-side, at both the function level and the object level, to prevent broken object-level authorisation, the top API risk, where changing an ID gives access to someone else's data; apply least-privilege scopes and role or attribute-based checks, and don't rely on the client. Input validation: validate against a schema, such as OpenAPI, for type, size, format, and allowed values, rejecting unexpected fields to prevent mass assignment; guard against injection. Rate limiting and quotas: protect against abuse, brute-force, scraping, and denial of service, with per-client limits and sensible payload size limits. Transport: TLS everywhere. Output: return only needed fields, avoid exposing excessive data and verbose errors. Inventory and governance: keep a catalogue of all APIs, retire old versions, and avoid shadow or zombie endpoints. Gateway and WAF: centralise authentication, throttling, and logging. Logging and monitoring: record who accessed what, detect anomalies. Test with DAST and API fuzzing, and review against the OWASP API Security Top 10. Version carefully and keep dependencies updated.

Likely follow-ups

- What is broken object-level authorisation?
- Why reject unexpected fields in requests?

```
✓ strong authn (OAuth2/OIDC, short-lived tokens)
```

```
✓ authorize EVERY request at the object level (no IDOR)
```

```
✓ validate/schema-check all input; rate limit + quotas
```

```
✓ no sensitive data in URLs/logs; TLS only
```

API security layers several controls. Authenticate with standards (OAuth2/OIDC, short-lived tokens). Authorize every request — the most common API flaw is broken object-level authorization (IDOR): checking the user is logged in but not that *this* user owns *that* resource. Validate all input against a schema. Rate limit and set quotas to prevent abuse and DoS. Use TLS everywhere and keep secrets out of URLs and logs. The OWASP API Top 10 specifically calls out authorization failures as the dominant API risk — authentication alone is not enough.

IDOR is the classic API bug: "Authenticated" ≠ "authorized for this object." Check ownership on every object access, or users read each other's data by changing an ID.

certs.eknathalabs.com

📚 **Official docs**

- RFC 6585: HTTP 429 Too Many Requests
- Microsoft: Cloud design pattern – Throttling

Q89 What is DevSecOps culturally — how do you embed security in dev teams?

AppSec

🎙 **Spoken answer · ~85 s**

DevSecOps culture means security is a shared responsibility of everyone who builds and runs software, integrated into daily workflows rather than a late gate owned by a separate team. How to embed it. Make the secure path the easy path: provide paved roads, such as hardened base images, service templates with scanning already wired into CI, shared libraries for authentication, and policy guardrails, so developers get security by default without extra effort. Automate checks in the pipeline with fast feedback and clear, actionable results, tuned to reduce noise. Embed people: security champions in each development team, who receive extra training and act as the local link to the security team; security engineers pairing with teams on design reviews and threat modelling instead of reviewing at the end. Train developers with hands-on secure coding practice relevant to their stack, capture-the-flag exercises, and learning from real incidents. Shared goals and metrics: track time to remediate, vulnerability backlog age, and coverage of controls, and include security in team objectives, not as a blocker metric. Blameless culture: treat vulnerabilities and incidents as learning opportunities, so people report problems early. Build trust by being helpful, explaining the why, offering solutions, and respecting developers' time. Provide clear policies and exception processes.

Likely follow-ups

- What does a security champion do?
- How do you measure DevSecOps culture?

```
✓ security as enabler, not gatekeeper (self-service guardrails)
```

```
✓ shift-left tooling devs actually use; fast, low-noise feedback
```

```
✓ security champions per team; blameless culture
```

DevSecOps is as much culture as tooling: making security a shared responsibility woven into development rather than a separate team that says no at the end. Practically: provide self-service guardrails and paved paths so the secure way is the easy way; integrate fast, low-false-positive tooling into existing workflows; embed security champions within product teams to spread expertise; and keep a blameless culture so people report issues instead of hiding them. The aim is to make security frictionless and owned by everyone, not a bottleneck owned by a few.

Enabler, not gate: Security that ships paved roads and fast feedback gets adopted; security that only blocks gets routed around. Culture determines which you are.

certs.eknathalabs.com

📚 **Official docs**

- OWASP DevSecOps Guideline

Q90 How do you manage security in a microservices architecture?

AppSec

🎙 **Spoken answer · ~87 s**

Microservices expand the attack surface: many services, many network paths, and many identities, so security must be systematic. Identity and authentication: give every service a strong identity, such as SPIFFE or service accounts, and use mTLS between services, often through a service mesh. Authorisation: enforce at the edge and also within services, because the internal network is not trusted; use token propagation with short-lived JWTs carrying user context, and policy engines like OPA or mesh authorisation policies for service-to-service rules based on identity. API gateway: centralise external authentication, rate limiting, input validation, and WAF. Network segmentation: default-deny network policies, so services can only reach what they need. Secrets: central management with workload identity and rotation, no shared credentials. Supply chain: scan and sign images, minimal base images, and dependency management across many repositories, with SBOMs. Runtime: hardened containers, non-root, read-only filesystems, and runtime detection. Data: each service owns its data, with access only through its API, and encryption in transit and at rest. Observability: centralised logs, distributed tracing, and security monitoring with correlation IDs to follow attacks across services. Governance: standardised templates and libraries, so security patterns are consistent; threat model service interactions; and contract testing. Automate everything, since manual security review does not scale to hundreds of services.

Likely follow-ups

- Why isn't the internal network trusted?
- How does a service mesh help with microservice security?

```
✓ zero-trust between services (mTLS, identity-based authz)
```

```
✓ per-service least-privilege identity & secrets
```

```
✓ each service validates its own input (don’t trust callers)
```

```
✓ centralized authn, distributed authz, mesh observability
```

Microservices multiply the attack surface — many services, much east-west traffic, many secrets. Apply zero-trust internally: services mutually authenticate (mTLS) and authorize based on cryptographic identity, not network location. Give each service its own minimal identity, permissions, and secrets so a single compromise is contained. Every service validates its own input rather than trusting upstream callers (defense in depth across the call graph). Use a service mesh for consistent mTLS, policy, and observability. The flat-trusted-network assumption is the trap to avoid.

certs.eknathalabs.com

Don’t trust the caller: Each service defends itself. A compromised upstream service shouldn’t mean automatic trust downstream — validate and authorize at every hop.

📚 **Start here (official docs)**

- Kubernetes security checklist

Q91 What is a WAF and what are its strengths and limits?

AppSec

🎙 **Spoken answer · ~88 s**

A Web Application Firewall inspects HTTP and HTTPS traffic to web applications and blocks malicious requests based on rules, sitting in front of the application as a reverse proxy, cloud service such as AWS WAF, Cloudflare, or Azure Front Door WAF, or an appliance. Typical protections: managed rule sets such as the OWASP Core Rule Set that detect SQL injection, XSS, path traversal, and known exploit patterns, rate limiting, bot management, IP reputation and geo-blocking, and custom rules. Strengths. Virtual patching: quickly block exploitation of a newly disclosed vulnerability, like Log4Shell, while the code fix is prepared. Defence in depth, blocking common automated attacks and noise before they reach the application. Visibility into attack traffic. DDoS and abusive bot mitigation at the edge. Protecting legacy applications that can't easily be fixed. Compliance, as PCI DSS recommends one. Limits. It's not a substitute for secure code: attackers can bypass rules with encoding tricks, payload obfuscation, or by exploiting logic the WAF doesn't understand. It can't detect business logic flaws or broken authorisation. False positives can block legitimate users, so tuning is needed, starting in detection mode. It sees only traffic that passes through it, so origins must not be reachable directly. Encrypted or API-heavy traffic requires proper TLS termination and schema-aware rules.

Likely follow-ups

- What is virtual patching?
- Why must the origin not be reachable directly?

```
WAF = Web Application Firewall (filters HTTP, layer 7)
```

```
blocks: common injection/XSS patterns, known exploit signatures
```

```
virtual patching: block an exploit before the code fix ships
```

A WAF inspects HTTP traffic and blocks malicious requests — injection attempts, known exploit signatures, suspicious patterns. Its key operational value is **virtual patching**: when a critical CVE drops, a WAF rule can block exploitation in minutes while you prepare the real code fix (this bought teams crucial time during Log4Shell). Limits: it’s a perimeter control that can be bypassed, generates false positives, and is no substitute for fixing the underlying vulnerability. Treat it as one defense-in-depth layer and a rapid-mitigation tool, not the actual fix.

certs.eknathalabs.com

WAF buys time, code fixes solve: Use it to neutralize active exploitation fast, then patch the real flaw. Relying on the WAF alone leaves the vulnerability live.

📚 **Official docs**

- OWASP: Web Application Firewall

Q92 How do you handle responsible disclosure and a bug bounty program?

AppSec

🎙 **Spoken answer · ~85 s**

Responsible disclosure is a process by which security researchers report vulnerabilities to the organisation privately and give it time to fix before publication. A mature programme starts with a clear vulnerability disclosure policy published on the website, with a security.txt file at /.well-known/security.txt, a dedicated contact address, scope of what's covered, safe harbour language promising not to pursue legal action against good-faith research, and expected response times. Process: triage reports quickly, acknowledging within a day or two; reproduce and assess severity with a standard scoring like CVSS plus business context; assign to the owning team with an SLA; fix, test, and deploy; communicate with the reporter throughout; and coordinate disclosure timing, often ninety days, crediting the researcher if they wish; issue advisories and CVEs where appropriate. A bug bounty adds financial rewards, typically run on platforms such as HackerOne or Bugcrowd, which supply researcher communities and triage help. Before launching a bounty: mature your internal vulnerability management, so you can handle the volume; define scope, out-of-scope items, and reward tiers; start private with invited researchers; ensure engineering capacity to fix; and prepare for duplicates and low-quality reports. Benefits: wider testing coverage and early detection. Risks: noise, cost, and expectations if responses are slow.

Likely follow-ups

- What goes in security.txt?
- When is a company ready for a bug bounty?

```
✓ published security.txt + clear reporting channel
```

```
✓ defined scope, safe-harbor for good-faith researchers
```

```
✓ triage SLA, severity-based rewards, track to remediation
```

A vulnerability disclosure program gives external researchers a safe, legitimate way to report flaws instead of selling or dropping them publicly. Provide a clear channel (a `security.txt`, a dedicated address), define scope and a safe-harbor policy so good-faith research isn't treated as an attack, and commit to triage SLAs. A bug bounty adds severity-based rewards to incentivize reporting. The payoff: you learn about vulnerabilities from friendly researchers before adversaries exploit them — but only invest in a bounty once your internal remediation process can keep up with the findings.

certs.eknathalabs.com

Friendly finders first: A disclosure program converts "researcher who found a bug" from a PR risk into an early-warning asset. Safe harbor is what makes them choose you over disclosure.

📚 **Official docs**

- security.txt (RFC 9116)

Q93 What are the phases of security incident response?

IR & Governance

🎙 **Spoken answer · ~85 s**

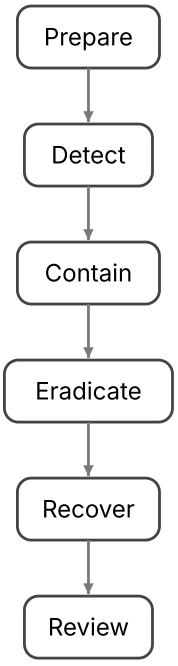
The widely used framework, from NIST SP 800-61, has four phases, often expanded to six. Preparation: build the capability before incidents happen, including an incident response plan and playbooks, defined roles and escalation paths, on-call rotations, logging and monitoring, tooling, contact lists, communication templates, and regular exercises such as tabletop drills. Detection and analysis: identify that an incident is happening through alerts, user reports, or threat intelligence; triage to determine whether it's real, scope, severity, and affected systems; gather evidence and build a timeline; and declare the incident with an incident commander. Containment, eradication, and recovery: contain to stop the spread, with short-term actions like isolating hosts or revoking credentials and long-term actions like segmentation, balanced against evidence preservation; eradicate the root cause by removing malware, closing the vulnerability, and resetting credentials; recover by restoring systems from known-good state, validating, and monitoring closely for recurrence. Post-incident activity: a blameless review covering what happened, what worked, gaps, and root causes; document the timeline; update detections, playbooks, and controls; track action items; and meet notification obligations to regulators or customers. Throughout, communicate with stakeholders, keep records, involve legal where needed, and preserve chain of custody for evidence. Practising the plan matters more than having it.

Likely follow-ups

- What happens in preparation?
- Why preserve evidence during containment?

certs.eknathlabs.com

🗺 **Incident response phases**




```
Preparation → Detection & Analysis → Containment →
```

```
Eradication → Recovery → Post-incident (lessons learned)
```

```
(NIST SP 800-61 lifecycle)
```

The NIST IR lifecycle: **Preparation** (runbooks, tooling, access, drills before anything happens), **Detection & Analysis** (identify and scope the incident), **Containment** (stop the spread — isolate hosts, revoke credentials), **Eradication** (remove the attacker's foothold), **Recovery** (restore to known-good and verify), and **Post-incident** (blameless review, fix root causes, improve detection). The phases often loop. The biggest determinant of outcome is preparation — the work done before the incident dwarfs improvisation during it.

Preparation wins incidents: The team with runbooks, pre-granted access, and prior drills contains in minutes; the unprepared team loses hours figuring out who can do what.

📚 **Official docs**

- Google SRE Book: Managing Incidents
- Google SRE Workbook: Incident Response
- NIST: Computer Security Incident Handling Guide (SP 800-61)

certs.eknathalabs.com

Q94 How do you contain a compromised cloud workload without destroying evidence?

IR & Governance

🎙 Spoken answer · ~85 s

The tension is between stopping the attacker quickly and keeping forensic evidence, which is often volatile. Approach for a cloud workload, such as an instance or container. First, preserve before destroying: capture volatile data if feasible, such as memory, running processes, and network connections, and take snapshots of disks and volumes, tagging them for investigation. Collect relevant logs: cloud audit logs, flow logs, and application logs, copied to a secure, separate location. Then isolate rather than terminate: change the instance's security group to a quarantine group with no ingress or egress, except for the forensic access; or on Kubernetes, apply a deny-all network policy, cordon the node, and label the pod so it's removed from services without deleting it. Detach it from load balancers and autoscaling groups so it isn't replaced, or protect it from termination. Revoke access: disable or rotate any credentials the workload could use, remove its role permissions or attach a deny policy, and invalidate active sessions, remembering that temporary credentials already issued may remain valid until expiry and can be revoked via session revocation. Don't power off or reboot unless necessary, as it destroys memory evidence. Then analyse copies from snapshots in an isolated forensic environment, not on the original.

Likely follow-ups

- Why isolate instead of terminate?
- What does a quarantine security group do?

```
1. isolate: tighten SG to deny all, detach from LB (don’t terminate)
```

```
2. snapshot disk + capture memory for forensics
```

```
3. revoke the workload’s credentials/role sessions
```

```
4. THEN investigate from the copy
```

The tension is containment vs. forensics. Don’t just terminate the instance — you destroy evidence and learn nothing about scope. Instead isolate it (security group to deny-all, remove from load balancer) so it can’t cause more harm or talk to C2, snapshot the disk and capture memory for analysis, and revoke its credentials/active role sessions (an attacker may have exfiltrated them). Then investigate the forensic copy. Preserve the trail because you need to know what the attacker accessed and whether they’re elsewhere too.

Isolate, don’t nuke: Terminating the box stops the bleeding but erases how they got in and what they took — and they may still hold credentials to come back.

📚 Official docs

- NIST: Computer Security Incident Handling Guide (SP 800-61)

certs.eknathalabs.com

Q95 What is a SIEM and what role does it play in detection?

IR & Governance

🎙 Spoken answer · ~89 s

A Security Information and Event Management system collects, normalises, correlates, stores, and analyses security-relevant logs and events from across the environment, giving a central place for detection, investigation, and compliance reporting. Sources include cloud audit logs, identity providers, endpoints, network devices, firewalls, Kubernetes audit logs, applications, and vulnerability scanners. Core functions: ingestion and parsing into a common schema; correlation rules that combine events across sources to spot patterns no single log shows, such as a login from an unusual country followed by a new access key and large data downloads; alerting with severity and enrichment from threat intelligence and asset context; search and investigation tools for analysts to pivot across data and build timelines; dashboards and reporting; long-term retention for compliance and forensics; and integration with SOAR for automated response and ticketing. Examples: Splunk, Microsoft Sentinel, Elastic Security, Google Chronicle, and Sumo Logic. Role in detection: it's the hub of the SOC's visibility, enabling detection of multi-stage attacks and insider threats. Challenges: data volume and cost, noisy rules and alert fatigue, parsing and onboarding effort, and gaps if log sources are missing. To get value: prioritise high-signal sources, map detections to MITRE ATT&CK, tune rules continuously, manage detections as code, measure coverage and mean time to detect, and make sure analysts have playbooks.

Likely follow-ups

- What is a correlation rule?
- How does SOAR relate to a SIEM?

```
SIEM = Security Information & Event Management
```

```
# aggregates logs from everywhere → correlate → detect → alert
```

```
tools: Splunk, Elastic, Sentinel, Chronicle
```

A SIEM centralizes logs and security events from across your environment (cloud, hosts, apps, network, identity) and correlates them to detect threats that no single source reveals. The power is correlation: a failed login here, a new IAM key there, and an unusual data transfer become one detected attack chain. It provides alerting, dashboards, and the searchable history needed for investigation and compliance. Its challenges: tuning to reduce alert fatigue, log volume/cost, and the fact that it only detects what your rules/detections look for — it needs ongoing detection engineering.

Correlation is the value: Individual logs are noise; a SIEM connects events across systems into a coherent "this is an attack" signal.

📚 Official docs

- NIST Cybersecurity Framework (measuring a security program)

certs.eknathalabs.com

Q96 What is threat detection engineering and the detection-as-code approach?

IR & Governance

🎙 Spoken answer · ~88 s

Threat detection engineering is the practice of systematically designing, building, testing, and maintaining detections that find malicious activity, treating detection as an engineering discipline rather than ad hoc rule writing. The process: start with threats relevant to your environment, from threat intelligence, past incidents, and frameworks such as MITRE ATT&CK, and identify the techniques adversaries use; check data availability, as you can't detect what you don't log; write detection logic with clear hypotheses; test against real or simulated attack data, using tools like Atomic Red Team and Stratus Red Team for cloud, and check false positive rates against production data; document each detection with its purpose, data sources, ATT&CK mapping, triage steps, and response guidance; deploy, monitor performance, and tune; and retire detections that no longer add value. Detection-as-code applies software engineering practices: detections written in portable formats such as Sigma, or in the platform's language, stored in Git, reviewed through pull requests, unit tested with sample logs, validated in CI, deployed through pipelines, and versioned. Benefits: quality and consistency, peer review, auditability, faster iteration, rollback, shared libraries, and portability across SIEMs. Measure coverage of ATT&CK techniques, alert precision, time to detect, and analyst feedback on usefulness. Prefer high-fidelity, behaviour-based detections over brittle indicators, and close the loop with incident learnings.

Likely follow-ups

- What is Sigma?
- How do you test a detection before deploying it?

```
# detections are version-controlled, tested code (Sigma rules)
```

```
# map coverage to MITRE ATT&CK techniques
```

```
test against attack simulations; measure false-positive rate
```

Detection engineering treats detections as a deliberate, iterative product rather than ad-hoc alerts. Detection-as-code version-controls rules (e.g. Sigma, a vendor-neutral format), tests them against simulated attacks and benign traffic to measure efficacy and false-positive rate, and maps coverage to the MITRE ATT&CK framework so you know which adversary techniques you can and can’t see. This brings software-engineering rigor — review, testing, CI — to detections, and turns "do we have coverage?" from a guess into a measurable, gap-driven program.

MITRE ATT&CK as a map: Mapping detections to ATT&CK techniques reveals your blind spots — the techniques an attacker could use that you’d never see.

📚 Official docs

- MITRE ATT&CK for Containers

certs.eknathalabs.com

Q97 What is zero trust as an architecture and its core principles?

IR & Governance

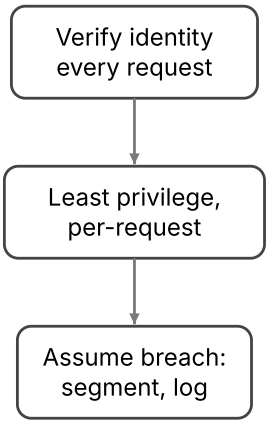
🎙 Spoken answer · ~88 s

Zero trust is a security architecture that removes implicit trust based on network location: no user, device, or service is trusted just because it is inside the corporate network; every access request is verified. The mantra is 'never trust, always verify', described in NIST SP 800-207. Core principles. Verify explicitly: authenticate and authorise every request using all available signals, such as strong identity with MFA, device health and posture, location, and behaviour. Least privilege access: grant minimal, just-in-time, just-enough permissions, scoped to resources, with short-lived credentials. Assume breach: design as though attackers are already inside, minimising blast radius through micro-segmentation, encrypting all traffic, and monitoring and logging everything for rapid detection. Treat identity as the new perimeter, with centralised identity providers and conditional access. Continuous evaluation, not one-time login: sessions are reassessed as risk changes. Protect resources individually: policy enforcement points in front of applications and data, replacing broad VPN access with identity-aware proxies, as in BeyondCorp and Zero Trust Network Access. For workloads: service identity, mTLS, and authorisation policies between services. Implementation is incremental: inventory assets and flows, strengthen identity and device management, segment networks, apply policy engines, and instrument telemetry. Challenges include legacy systems, complexity, user experience, and cultural change. It's a journey and a model, not a single product.

Likely follow-ups

- Why is identity the new perimeter?
- What does assume breach change in design?

🗺 Zero trust




```
"never trust, always verify"
```

```
✓ verify explicitly (identity, device, context per request)
```

```
✓ least-privilege access
```

certs.eknathalabs.com

```
✓ assume breach (segment, limit blast radius, log everything)
```

Zero trust discards the old "trusted internal network" model — there is no implicit trust based on network location. Every access request is authenticated, authorized, and validated based on identity, device posture, and context, regardless of where it originates. Core principles: verify explicitly (per request, not once at the perimeter), enforce least privilege, and assume breach (segment everything, minimize blast radius, log and monitor continuously). In practice it shows up as workload identity, mTLS, micro-segmentation, and identity-aware proxies replacing flat VPN-style network trust.

Location ≠ trust: "Inside the network" means nothing in zero trust. A request from a prod pod is verified the same as one from the internet.

📚 **Official docs**

- NIST Zero Trust Architecture (SP 800-207)

Q98 How do you measure the effectiveness of a security program (metrics)? IR & Governance

🎙 **Spoken answer · ~89 s**

Measuring a security programme means choosing metrics that show risk reduction and operational effectiveness, not just activity. Categories. Detection and response: mean time to detect, mean time to respond and contain, mean time to recover, and incident counts by severity and root cause; this shows how quickly you handle threats. Vulnerability management: time to remediate by severity compared to SLA, the age of the backlog, percentage of assets scanned, and exposure of internet-facing vulnerabilities; trends matter more than totals. Preventive control coverage: percentage of services with MFA, encryption, SSO, endpoint agents, signed images, and logging enabled; percentage of repositories with secret scanning and SAST. Posture: number of critical misconfigurations, policy compliance rate, and drift. Identity: percentage of privileged accounts with MFA, unused or excessive permissions, and age of credentials, and the number of long-lived keys. Secure development: findings caught in pipeline versus production, and escape rate. People and culture: training completion, phishing simulation results, security champion engagement, and reporting rates. Compliance: audit findings. Business-oriented: risk reduction, cost of incidents avoided, and security's effect on delivery speed. Good practices: tie metrics to goals and risk, make them actionable, automate collection, avoid vanity metrics and perverse incentives, report trends to leadership with context, and review regularly. Use frameworks such as NIST CSF to structure maturity.

Likely follow-ups

- Why do trends matter more than totals?
- What is a vanity metric in security?

```
MTTD / MTTR (detect / respond time)
```

```
mean time to remediate vulns (by severity)
```

certs.eknathalabs.com

```
% workloads meeting baseline; patch/coverage rates; escaped defects
```

Useful security metrics measure outcomes and reduce risk, not vanity counts. Mean time to detect/respond (MTTD/MTTR) gauges your detection and IR capability. Mean time to remediate vulnerabilities (segmented by severity) shows whether real risk is being closed. Coverage metrics — percentage of workloads meeting the hardening baseline, scan/policy coverage, patch compliance — show how broadly controls apply. "Escaped" findings (issues caught in prod that should've been caught earlier) measure shift-left effectiveness. Avoid pure activity metrics (number of alerts) that reward noise over risk reduction.

Measure risk reduced, not activity: "5,000 alerts" says nothing. "Critical vulns remediated in <7 days, 95% workload coverage" measures whether you're actually safer.

📚 **Official docs**

- NIST Cybersecurity Framework (measuring a security program)

Q99 What is the difference between vulnerability management and risk management?

IR & Governance

🎙 **Spoken answer · ~83 s**

Vulnerability management is an operational process: finding, prioritising, remediating, and verifying weaknesses in systems and software through scanning, patching, and configuration fixes. Its focus is on individual technical flaws, such as CVEs and misconfigurations, and its metrics are things like time to patch and coverage. Risk management is a broader business discipline that identifies, assesses, and decides how to treat risks to the organisation's objectives, whether or not they come from technical vulnerabilities: threats, vulnerabilities, assets, likelihood, and impact, including operational, legal, financial, and reputational risk. Options for treating a risk are to mitigate, transfer through insurance, avoid, or accept, with decisions made by accountable business owners within a defined risk appetite. The relationship: vulnerability management feeds risk management, but a vulnerability is not automatically a high risk. Risk depends on exploitability, exposure, such as whether the asset is internet-facing, the criticality of the affected asset and data, existing compensating controls, and threat activity, such as whether it's in CISA's Known Exploited Vulnerabilities list or has a high EPSS score. So mature programmes prioritise by risk, not only CVSS severity; patching ten thousand medium findings on low-value systems is less valuable than one reachable critical flaw on a key system.

Likely follow-ups

- What are EPSS and the KEV list?
- How does risk acceptance get documented?

```
vuln mgmt: find & fix technical flaws (CVEs, misconfigs)
```

```
risk mgmt: prioritize by business impact × likelihood × exposure
```

certs.eknathalabs.com

Vulnerability management is the technical process of discovering and remediating flaws. Risk management is the broader discipline of deciding *which* ones matter and in what order, based on business context: a critical CVE on an internet-facing system holding customer data is urgent; the same CVE on an isolated internal tool with no sensitive access may not be. Risk = impact × likelihood × exposure. You can't fix everything, so risk management directs limited remediation effort where it reduces the most actual business risk — CVSS score alone is not risk.

CVSS ≠ risk: A "critical" CVE on an unreachable, low-value asset may be lower risk than a "medium" on your crown-jewel system. Context decides priority.

📚 **Official docs**

- Trivy

Q100 **Scenario: secrets were committed to a public repo and the key was used. Walk through your response.** IR & Governance

🎙 **Spoken answer · ~83 s**

I'd handle this as a security incident with urgency, structured in phases. Immediately: confirm the exposure and revoke or disable the key first, before anything else, because the repository is public and credentials get scanned within minutes. Don't wait for the investigation; if production depends on it, rotate with a new credential and deploy it quickly. Make the repository private or remove the secret, but remember history, forks, and caches mean it's already compromised. Declare an incident, assign an incident commander, and open a channel. Scope: find what the key could do, and check audit logs for every use of that key, including source IPs, user agents, times, and actions, comparing to normal use. Since it was used, assume unauthorised access: look for new resources, instances in unusual regions, new IAM users, roles, or keys, policy changes, data downloads or snapshot sharing, disabled logging, and persistence such as backdoor functions or scheduled jobs. Contain and eradicate: delete attacker-created resources and credentials, revoke active sessions, rotate other secrets the key could read, and isolate affected workloads while preserving evidence. Assess impact: what data was accessed, and whether notification to customers or regulators is required, engaging legal. Recover and verify.

Likely follow-ups

- Why revoke before investigating?
- What persistence would you check for in the account?

```
1. CONTAIN: revoke/rotate the key NOW; disable the identity
```

```
2. SCOPE: audit logs — what did the attacker access/create?
```

```
3. HUNT: new IAM users/keys, resources, persistence, lateral move
```

```
4. ERADICATE & RECOVER: remove attacker artifacts, restore clean
```

ceris.eknathalabs.com

```
5. PREVENT: secret scanning, OIDC, history purge, postmortem
```

This integrates the whole guide. **Contain first** — revoke the key and disable the identity immediately; every minute it's live is more damage. **Scope** via audit logs (CloudTrail): what did the credential do, from where? **Hunt for persistence** — attackers create backdoor IAM users, access keys, and resources; assume they tried to entrench and move laterally. **Eradicate** those artifacts and **recover** to a known-good state. Then **prevent recurrence**: secret scanning in CI and pre-commit, replace static keys with OIDC/workload identity, purge history (knowing the key must still be rotated), and run a blameless postmortem on how it leaked and why it wasn't caught.

What earns the offer: Leading with containment over investigation, knowing a leaked key means hunting for attacker-created persistence (not just rotating), and closing with systemic prevention — it shows you think in incidents and root causes, not just one-off fixes.

📚 **Official docs**

- GitHub: Removing sensitive data from a repository

Q101 How do you use short-lived OIDC tokens in CI to eliminate stored cloud secrets?

Secrets Mgmt

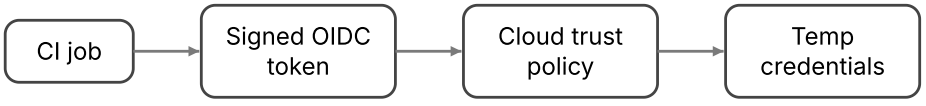
🎙 **Spoken answer · ~64 s**

Static cloud keys stored as CI secrets are a standing target: they leak through logs, forks and compromised runners, and they rarely get rotated. With OIDC federation, each pipeline job receives a short-lived signed token that states which repository, branch and workflow it belongs to. In the cloud I register the CI platform as a trusted identity provider and create a role whose trust policy accepts tokens only when claims match, for example the exact repository and the main branch or a protected deployment environment. The job exchanges the token for temporary credentials valid for minutes, so there is nothing long-lived to steal. I separate roles by purpose, with a read-only role for pull request builds and a deploy role for protected branches, and I scope permissions tightly. The common mistake is a loose subject condition, which lets other repositories in the organisation assume the role, so I test the policy with negative cases.

Likely follow-ups

- What claims would you trust?
- Why separate roles for pull requests?

🗺 **OIDC in CI**



certs.EKnathalabs.com

The CI platform issues a signed identity token per job; the cloud trusts that issuer and exchanges the token for temporary credentials. Trust conditions on repo, branch and environment limit who can assume the role.

```
aws sts assume-role-with-web-identity --role-arn $ROLE --web-identity-token $TOKEN --role-session-name ci
```

```
az login --federated-token $TOKEN --service-principal -u $APP -t $TENANT
```

```
gcloud iam workload-identity-pools providers describe ci --workload-identity-pool=pool --location=global
```

Interview tip: The subject claim in the trust policy is the security boundary. A wildcard there lets any repository assume the role.

📚 **Official docs**

- GitHub Actions: OpenID Connect
- OAuth 2.0 / OIDC overview (OpenID)

Q102 What is the Secrets Store CSI Driver and how does it differ from External Secrets Operator?

Secrets Mgmt

🎙 **Spoken answer · ~63 s**

Both bring secrets from a manager such as Vault or a cloud service into Kubernetes, but they work differently. The Secrets Store CSI driver uses a volume: when a pod starts, the driver authenticates with the pod's identity, fetches the listed secrets and mounts them as files in a memory-backed volume. Rotation can be handled by periodic refresh of the mount. By default the value never becomes a Kubernetes Secret, so it is not stored in etcd, although it can optionally be synced to one for environment variable use. External Secrets Operator instead runs a controller that reads from the external store and creates or updates ordinary Secret objects, which any workload can consume in the usual ways, and it handles refresh intervals and templating well. I choose CSI when minimising exposure in etcd matters or apps read files, and ESO when applications and tools expect native Secrets, such as Ingress TLS.

Likely follow-ups

- When do secrets land in etcd?
- How does rotation reach a running pod?

The CSI driver mounts secrets from a provider directly into pods as files at start time, optionally syncing to a Kubernetes Secret. ESO continuously syncs external secrets into native Secret objects. Pick by whether you want files or Secret objects.

```
kubectl get secretproviderclass -A
```

```
kubectl get externalsecret -A
```

certs.eknathalabs.com

```
volumes: - name: sec csi: driver: secrets-store.csi.k8s.io readOnly: true
```

Interview tip: CSI mounting can avoid storing the value in etcd at all, which is its main security advantage.

📚 Official docs

- External Secrets Operator
- Secrets Store CSI Driver

Q103 How do you handle secrets in Docker builds without baking them into image layers?

Secrets Mgmt

🎙 Spoken answer · ~62 s

Image layers are permanent, and anyone who can pull the image can read every layer, so a secret copied in during build is exposed even if a later step deletes it. Build arguments and environment variables are also recorded in the image history. The correct tool is BuildKit's secret mount: the secret is supplied at build time from a file or environment on the builder, appears at a path under run secrets only during the single instruction that mounts it, and is never written to a layer. For private Git dependencies, an SSH mount forwards the agent socket in the same transient way. I also use multi-stage builds, so credentials only exist in a builder stage that is discarded, and scan the final image and its history for secrets. The runtime secrets are not baked in at all; they are injected by the platform when the container starts.

Likely follow-ups

- Why doesn't deleting a file help?
- What is a multi-stage build's role here?

Use BuildKit secret mounts or SSH mounts, which expose the secret only during one RUN step and leave nothing in layers or history. Never use ARG or ENV for credentials, and use multi-stage builds.

```
docker build --secret id=npmrc,src=$HOME/.npmrc .
```

```
RUN --mount=type=secret,id=npmrc cp /run/secrets/npmrc ~/.npmrc && npm ci
```

```
docker history --no-trunc image:tag
```

```
RUN --mount=type=ssh git clone git@host:org/private.git
```

Interview tip: Deleting a file in a later layer does not remove it from the earlier layer. Anyone with the image can extract it.

📚 Official docs

- Docker documentation

certs.eknathalabs.com

Q104 How do you design a break-glass process for secrets and privileged access?

Secrets Mgmt

🎙 Spoken answer · ~60 s

Normal access controls sometimes block legitimate emergency work, for example when the identity provider is down or a production incident needs temporary elevated rights. Break-glass is the controlled exception. The credentials are stored offline or in a vault with split custody, and using them requires multi-person approval or at least a strong second factor. Access is time-limited, so it expires automatically, and requires a stated reason linked to an incident. Using the account triggers an immediate alert to the security team, sessions are recorded or logged in detail, and after the emergency the credentials are rotated and the usage is reviewed within a day or two. I test the procedure regularly, because an untested emergency path often fails when needed, and verify that the monitoring really fires. The accounts are excluded from daily use and from routine automation to keep the signal clean.

Likely follow-ups

- Why must use trigger an alert?
- How often would you test break-glass?

A documented emergency path with strong authentication, time-boxed elevated access, mandatory justification, alerting, session recording, and automatic credential rotation afterwards. Test it regularly so it works under pressure.

```
vault token create -policy=breakglass -ttl=30m
```

```
aws iam list-roles --query "Roles[?contains(RoleName,'breakglass')]"
```

```
aws cloudtrail lookup-events --lookup-attributes
AttributeKey=Username,AttributeValue=breakglass
```

Interview tip: Every break-glass use should page the security team and generate a review ticket. Silent use defeats the purpose.

📚 Start here (official docs)

- Kubernetes security checklist

certs.eknathalabs.com

Q105 What is secret zero and how do you solve the bootstrap problem?

Secrets Mgmt

🎙 Spoken answer · ~62 s

The bootstrap problem is that to get secrets from a vault, an application first needs a credential for the vault, and storing that credential securely raises the same question again. The answer is to make the platform vouch for the workload. A cloud instance can present signed metadata, a Kubernetes pod presents its projected service account token, and CI presents an OIDC token. The secrets system verifies the signature, checks the claims against a configured role, and issues a short-lived token or credential in return. No long-lived password is ever placed into the image or configuration. Trust then relies on the integrity of the platform identity and on tight role bindings, such as namespace and service account name. Where hardware is involved, attestation with a TPM can prove the machine's identity. I also limit the lifetime and use count of the first token so a leak has little value.

Likely follow-ups

- What identities can the platform assert?
- Why short-lived initial tokens?

Any system that fetches secrets needs an initial credential. Solve it with platform identity: cloud instance or workload identity, Kubernetes service account tokens, or attestation, so no static bootstrap secret is distributed.

```
vault write auth/kubernetes/role/app bound_service_account_names=app
bound_service_account_namespaces=prod policies=app ttl=15m
```

```
curl -H 'Metadata-Flavor: Google' http://metadata/computeMetadata/v1/instance/service-
accounts/default/token
```

Interview tip: Trusted platform attestation turns who the workload is into the credential, which removes the chicken-and-egg problem.

📚 Start here (official docs)

- Kubernetes security checklist

certs.eknathalabs.com

Q106 How do you handle certificate and key lifecycle with cert-manager and a private CA?

Secrets Mgmt

🎙 Spoken answer · ~58 s

Manual certificate handling leads to outages from expiry and to risk from long-lived private keys. cert-manager runs in the cluster, watches Certificate resources, requests certificates from a configured issuer, stores them as secrets and renews them before expiry. Issuers can be a public ACME service for ingress, Vault's PKI engine, or an internal certificate authority for service-to-service mTLS. I keep lifetimes short, days or weeks, so compromise has limited value and renewal becomes routine. The root CA key stays offline or in a hardware security module, and a protected intermediate signs workload certificates. I monitor the renewal status and set alerts on days to expiry, since webhook or quota problems can make renewal fail quietly. Applications must reload certificates without restart, or be restarted by a controller. Revocation is handled mostly by short lifetime rather than CRL distribution.

Likely follow-ups

- Why short-lived certificates?
- How do apps pick up renewed certs?

cert-manager automates issuance and renewal using Issuers backed by ACME, Vault or a private CA. Short-lived certs reduce exposure; monitor expiry, protect CA keys offline or in an HSM, and automate rotation end to end.

```
kubectl get certificate,certificaterequest -A
```

```
kubectl describe certificate api-tls -n prod
```

```
cmctl status certificate api-tls -n prod
```

```
cmctl renew api-tls -n prod
```

Interview tip: Alert on certificates nearing expiry even when renewal is automated, because silent renewal failures cause outages.

📚 Official docs

- cert-manager documentation

certs.eknathalabs.com

Q107 What is the OpenSSF Scorecard and how do you use it to evaluate dependencies?

Supply Chain

🎙 Spoken answer · ~63 s

Choosing a dependency means trusting its maintainers and their practices. The Scorecard project automates a set of checks on the repository: whether the default branch is protected, whether changes need review, whether dependencies and workflows are pinned, whether releases are signed, whether there is a security policy, how recently it was maintained, and whether fuzzing or static analysis is used. Each check gets a score and there is an aggregate. I use it in two places. When evaluating a new library, a very low score on review and branch protection tells me that a single compromised account could ship malicious code. And in continuous monitoring, I scan the dependencies I already use and track trends. It is a heuristic, so I combine it with other evidence such as the bus factor, release cadence, known advisories and how critical the component is. For critical dependencies I may fork, vendor or sponsor maintenance.

Likely follow-ups

- What would a low Pinned-Dependencies score mean?
- Does a high score guarantee safety?

Scorecard runs automated checks on open source projects, such as branch protection, code review, pinned dependencies, signed releases and vulnerability handling, producing a score. Use it as one signal when selecting and monitoring dependencies.

```
scorecard --repo=github.com/org/project
```

```
scorecard --repo=github.com/org/project --format=json | jq '.checks[] | {name,score}'
```

```
scorecard --checks=Pinned-Dependencies,Branch-Protection --repo=github.com/org/project
```

Interview tip: A low score is a prompt to investigate, not an automatic ban. Combine with maintenance activity and usage context.

📚 Official docs

- OpenSSF Scorecard

certs.eknathalabs.com

Q108 How do you pin GitHub Actions and other CI dependencies to resist tag hijacking?

Supply Chain

🎙 Spoken answer · ~63 s

A workflow step that uses an action by tag trusts whatever that tag points to at run time. If the action's repository is compromised, an attacker can move the tag to a malicious commit, and every pipeline using it runs the attacker's code with access to the job's secrets and token, which actually happened in real incidents. Pinning to a full commit hash makes the reference immutable. To keep pins current without manual effort, I use a dependency bot that proposes updates as pull requests with a diff and release notes, so a human reviews each change. I also restrict the organisation to an allowlist of approved actions, set the default token permissions to read-only and grant write scopes per job, avoid running untrusted pull request code with secrets, and prefer well-known maintainers or vendoring small actions. The same principle applies to container images referenced by digest and scripts downloaded in pipelines.

Likely follow-ups

- Why can tags be moved?
- How do you keep pins up to date?

Pin third-party actions to full commit SHAs rather than mutable tags or branches, review updates through automated PRs, restrict allowed actions, and use minimal token permissions.

```
uses: actions/checkout@b4ffde65f46336ab88eb53be808477a3936bae11
```

```
permissions: contents: read
```

```
gh api repos/org/repo/actions/permissions
```

```
npx pin-github-action .github/workflows/ci.yml
```

Interview tip: Tags can be moved by an attacker who compromises the action repository. A commit SHA is immutable.

📚 Official docs

- GitHub Actions documentation

certs.eknathalabs.com

Q109 How do you generate and consume an SBOM in a CI pipeline, and what formats exist?

Supply Chain

🎙 Spoken answer · ~61 s

A software bill of materials lists the components inside an artifact. In the pipeline I generate it right after building the image, using a scanner that inspects the packages in the final image and the language lockfiles, and output it in a standard format, either SPDX or CycloneDX. I then attach it to the image digest as a signed attestation, so consumers can verify it came from my pipeline, and also store it in a central inventory. The practical use comes later: when a new vulnerability such as a major library CVE is published, I query the stored SBOMs to find every affected service in minutes rather than scanning all repositories, and I can feed the same SBOMs to a vulnerability scanner on a schedule. Quality matters, so I check that the SBOM covers transitive dependencies and OS packages, and compare against what is actually deployed.

Likely follow-ups

- SPDX vs CycloneDX?
- Why attach the SBOM to the digest?

🗺 SBOM



Generate an SBOM per build with tools like Syft in SPDX or CycloneDX format, attach it to the image as an attestation, store it, and scan it continuously against new vulnerability data.

```
syft packages registry/app:1.4 -o cyclonedx-json > sbom.json
```

```
grype sbom:sbom.json --fail-on high
```

```
cosign attest --predicate sbom.json --type cyclonedx registry/app@sha256:abc
```

```
cosign verify-attestation --type cyclonedx registry/app@sha256:abc
```

Interview tip: The value is in re-scanning stored SBOMs when a new CVE drops, so you can answer 'where are we affected' in minutes.

📚 Official docs

- SPDX specification
- CycloneDX
- Syft (SBOM generator)

certs.eknathalabs.com

Q110 What are reproducible builds and why do they matter for supply-chain security?

Supply Chain

🎙 Spoken answer · ~60 s

The central trust question for a binary is whether it really came from the source code claimed. If builds are reproducible, two independent builders using the same source and toolchain produce identical bytes, so I can rebuild and compare hashes. A mismatch reveals a compromised build machine or a tampered artifact. Achieving it requires removing nondeterminism: embedding the commit time rather than the current time, sorting file lists, normalising paths and user IDs, pinning every dependency and the compiler version, and building in a clean, hermetic environment. Tools such as diffoscope show where two outputs differ. Even where fully bit-identical builds are impractical, hermetic and provenance-recording builds give much of the benefit. Reproducibility complements signing and provenance: signatures prove who built it, reproducibility lets others check that what they built was honest. Organisations often apply it first to their most critical or widely distributed components.

Likely follow-ups

- What breaks reproducibility?
- How does it complement signing?

A reproducible build yields bit-for-bit identical output from the same source, so independent parties can verify that a binary matches its claimed source and detect tampering in the build environment.

```
SOURCE_DATE_EPOCH=$(git log -1 --format=%ct) make build
```

```
sha256sum dist/app-linux-amd64
```

```
diffoscope build-a/app build-b/app
```

```
docker buildx build --provenance=true --sbom=true .
```

Interview tip: Common blockers are timestamps, file ordering, embedded paths and non-pinned dependencies.

📚 Official docs

- Google SRE Book: Release Engineering
- SLSA specification

certs.eknathalabs.com

Q111 How do you secure a private package registry and prevent typosquatting and malicious packages?

Supply Chain

🎙 **Spoken answer · ~58 s**

Developers and pipelines install code from the internet constantly, and attackers exploit that with typosquatted names, dependency confusion and compromised maintainers. I route all installs through an internal repository manager that proxies the public registry, so I have a single control point. There I can allow only approved or scanned packages, quarantine brand-new versions for a few days so that malicious releases are usually detected before they are consumed, and scan for known malware and vulnerabilities. Private package names are scoped to a namespace bound to the internal registry so a public package of the same name cannot win. Lockfiles with integrity hashes make installs deterministic and detect modification. I disable lifecycle install scripts by default and enable them case by case, and verify provenance and signatures where the ecosystem supports them. Developer education on checking package names also helps.

Likely follow-ups

- What is a cooldown period?
- Why disable install scripts?

Proxy public registries through an internal repository manager, allowlist packages, scope private namespaces, add cooldown periods and malware scanning, require lockfiles and integrity hashes, and block install scripts where possible.

```
npm config set ignore-scripts true
```

```
pip install --require-hashes -r requirements.txt
```

```
npm audit signatures
```

```
npm config set @acme:registry https://registry.internal/
```

Interview tip: Install scripts execute with developer or CI privileges at install time, so treat new packages like running unknown code.

📚 **Official docs**

- OpenSSF Scorecard

certs.eknathalabs.com

Q112 How do you assess and respond to a compromised maintainer or malicious release of a dependency you use?

Supply Chain

🎙 Spoken answer · ~60 s

The first step is scoping: which of our services and build jobs used the affected version, and when. SBOMs and lockfiles answer that quickly, and the registry proxy logs show who downloaded it. I then contain: block the version in the internal registry, pin or roll back to the last known good release, rebuild and redeploy affected services. Next I assume impact where the package executed, since a malicious install script or runtime payload could have stolen environment variables, tokens or SSH keys, so I rotate any credential available to those CI jobs or hosts and review audit logs for misuse. I look for indicators of compromise published by researchers, such as network connections or files. Finally I communicate to teams, record a timeline, and improve controls such as version cooldowns, pinning by hash, install script restrictions and egress filtering from build runners.

Likely follow-ups

- Why rotate CI secrets?
- How does the registry proxy help response?

Identify exposure through SBOMs and lockfiles, pin or roll back to a known good version, rotate any secrets exposed to affected builds, check for indicators of compromise, and block the version at the registry proxy.

```
npm ls malicious-pkg --all
```

```
grep -R 'malicious-pkg' */package-lock.json
```

```
git log --all -S'malicious-pkg@1.2.4' --oneline
```

```
npm view malicious-pkg time --json
```

Interview tip: If the bad version ran in CI, assume that job's secrets were exposed and rotate them.

📚 Official docs

- NIST: Computer Security Incident Handling Guide (SP 800-61)

certs.eknathalabs.com

Q113 What is in-toto and how does it add verifiable steps to a software supply chain?

Supply Chain

🎙 Spoken answer · ~59 s

Signing the final artifact only proves who signed it, not that the right process produced it. In-toto addresses this by describing the whole pipeline. The project owner defines a layout: the steps that must occur, such as code review, build, test and scan, which functionaries are authorised to perform each step, and the expected relationships between inputs and outputs. When each step runs, the tool records the materials consumed and products created with their hashes and signs this as link metadata. Before deployment, a verifier checks the layout's signature, confirms every required step has links signed by authorised keys, and that the products of one step match the materials of the next, so nothing was swapped between stages. The attestation concept in SLSA provenance and sigstore builds on the same idea. It makes bypassing a control, like skipping tests, cryptographically visible.

Likely follow-ups

- How is this different from signing the image?
- What does the layout define?

in-toto defines a layout of required steps and who may perform them, and each step emits signed link metadata. A verifier checks that every step happened, by the right party, with matching materials and products.

```
in-toto-run --step-name build --key builder.key --materials src/ --products dist/ -- make
```

```
in-toto-verify --layout root.layout --layout-keys owner.pub
```

```
cosign attest --type slsaprovenance --predicate prov.json $IMAGE
```

Interview tip: Think of it as a signed checklist across the pipeline, so a skipped scan or an unauthorised build step is detectable.

📚 Official docs

- in-toto

certs.eknathalabs.com

Q114 How do you vet container base images and maintain an internal golden image pipeline?

Supply Chain

🎙 Spoken answer · ~58 s

Every application image inherits the risk of its base, so I standardise a small set of approved bases. A platform team builds these golden images from minimal parents, such as distroless or a slim distribution, applies updates, removes shells and package managers where possible, adds only needed CA certificates and a non-root user, then scans and signs the result. They are rebuilt on a fixed schedule, for example weekly, and immediately when a critical vulnerability appears in a base package, then published with immutable digests. Application teams reference the base by digest through an automated update bot, so they get a pull request when a new base appears. An admission policy or registry rule ensures only images derived from approved bases run in production. Metrics track the age of bases in use, which shows teams lagging behind patches.

Likely follow-ups

- Why reference by digest?
- How do you enforce use of golden images?

Choose minimal, maintained bases, build golden images centrally with patching, scanning and signing, publish by digest, and rebuild on a schedule and on CVE triggers. Teams must use the approved images.

```
docker buildx imagetools inspect registry/golden/python:3.12
```

```
trivy image --severity HIGH,CRITICAL registry/golden/python:3.12
```

```
cosign sign registry/golden/python@sha256:abc
```

```
crane digest registry/golden/python:3.12
```

Interview tip: Distroless or minimal images shrink the attack surface and the noise from scanners.

📚 Official docs

- Packer documentation

certs.eknathalabs.com

Q115 How do you verify provenance of third-party binaries and tools downloaded in pipelines?

Supply Chain

🎙 Spoken answer · ~60 s

Pipelines often download CLI tools and scripts, and a curl-to-shell pattern executes whatever the server returns that day. I pin exact versions and verify what I download. A checksum is the minimum, but if it comes from the same server as the binary an attacker who controls the server controls both, so I prefer cryptographic signatures verified against a key or identity I trust, such as a project's signing key or a sigstore identity. For the tools my builds depend on heavily, I mirror them into an internal artifact repository after verification, so pipelines pull the vetted copy and are not affected by upstream outages or tampering. Package manager installs with signature checks are better than ad hoc downloads. In build images I bake the verified tools into the image rather than downloading at run time. Egress filtering on runners limits unexpected downloads.

Likely follow-ups

- Why is a same-origin checksum weak?
- What does mirroring give you?

Pin versions, verify checksums and signatures from a trusted source, prefer package managers with verification, mirror approved tools internally, and avoid piping curl to bash.

```
curl -fsSLO https://host/tool_1.2.3_linux_amd64.tar.gz && sha256sum -c tool.sha256
```

```
cosign verify-blob --certificate-identity=release@org --certificate-oidc-
issuer=https://token.actions.githubusercontent.com --signature tool.sig tool.tar.gz
```

```
gpg --verify tool.tar.gz.asc tool.tar.gz
```

Interview tip: A checksum fetched from the same place as the binary only proves integrity in transit, not authenticity. Verify a signature.

📚 Official docs

- GitHub: Artifact attestations (build provenance)

certs.eknathalabs.com

Q116 How do you use Pod Security Standards with namespace labels and exemptions in practice?

K8s / Container

🎙 Spoken answer · ~60 s

Pod Security Admission is the built-in replacement for pod security policies. It applies one of three standards per namespace through labels: privileged, baseline, which blocks known escalations, and restricted, which follows current hardening practice such as non-root, dropped capabilities and a seccomp profile. Each namespace can have separate modes: enforce rejects violating pods, audit records violations in the audit log, and warn returns warnings to the user. My rollout is gradual. I first add audit and warn labels at the target level, and use the server-side dry-run to see which existing workloads would fail. Teams fix manifests, then I enable enforce, starting with restricted for application namespaces and baseline or privileged only for system components that truly need it. Exemptions in the admission configuration are kept minimal and documented. For finer rules than the three levels, I add policy engines such as Kyverno.

Likely follow-ups

- What are the three modes?
- How do you test before enforcing?

Label namespaces with enforce, audit and warn levels (privileged, baseline, restricted). Start with warn and audit to find violations, fix workloads, then enforce. Exempt only specific system namespaces or users.

```
kubectl label ns prod pod-security.kubernetes.io/enforce=restricted pod-
security.kubernetes.io/warn=restricted
```

```
kubectl label --dry-run=server --overwrite ns --all pod-
security.kubernetes.io/enforce=baseline
```

```
kubectl get ns --show-labels
```

Interview tip: The server-side dry-run label shows which existing pods would violate a level before you enforce it.

📚 Official docs

- Kubernetes: Pod Security Standards

certs.eknathalabs.com

Q117 What is a securityContext and which settings give the best hardening for a typical application pod?

K8s / Container

🎙 Spoken answer · ~56 s

The securityContext defines the privileges the container process has. A strong baseline for ordinary applications is: run as a non-root numeric user, so the process cannot write to root-owned locations or bind privileged ports; disable privilege escalation so setuid binaries cannot raise privileges; drop all Linux capabilities and add back only specific ones if truly required; make the root filesystem read-only and provide emptyDir volumes only for paths that must be writable; and apply the runtime default seccomp profile to filter dangerous system calls. Also avoid privileged mode, host networking, host PID and hostPath mounts. These settings limit what an attacker can do after exploiting the application, and they reduce container escape opportunities. Some images assume root or writable paths, so I fix them at build time. Admission policies enforce the baseline across all teams.

Likely follow-ups

- Why a numeric UID?
- What does allowPrivilegeEscalation block?

runAsNonRoot, a numeric UID, readOnlyRootFilesystem, allowPrivilegeEscalation false, drop ALL capabilities, seccompProfile RuntimeDefault, and no hostPath or host namespaces. Mount writable emptyDirs only where needed.

```
securityContext: runAsNonRoot: true runAsUser: 10001 allowPrivilegeEscalation: false
readOnlyRootFilesystem: true capabilities: { drop: ["ALL"] } seccompProfile: { type:
RuntimeDefault }
```

```
kubectl auth can-i --list --as=system:serviceaccount:prod:app
```

```
kubectl exec pod -- id
```

Interview tip: Test with the restricted Pod Security level. If your manifest passes it, you have the main hardening in place.

📚 Official docs

- Kubernetes: Pod Security Standards
- Kubernetes: Security context

certs.eknathalabs.com

Q118 How do you audit and reduce excessive Kubernetes RBAC permissions?

K8s / Container

🎙 Spoken answer · ~56 s

Over-permissive RBAC is one of the most common Kubernetes risks. I start with an inventory: list cluster role bindings, particularly any that grant cluster-admin, wildcard resources or verbs, and bindings to broad groups. For each service account I ask what it can actually do, using the can-i listing and who-can plugins. Certain verbs are dangerous even when they look benign: read access to secrets, the ability to create pods which can mount anything, exec into pods, impersonation, and the rights to modify roles or bindings, which enables escalation. To right-size, I compare granted permissions against the API calls actually seen in audit logs, and generate narrower namespaced roles. Human access goes through the identity provider with groups, and break-glass admin is time-limited. I review regularly and add policy that blocks wildcard roles from being created.

Likely follow-ups

- Which verbs enable escalation?
- How do audit logs help right-size roles?

Review bindings with tools such as kubectl-who-can, rbac-tool and audit logs, remove cluster-admin and wildcard grants, prefer namespaced Roles, avoid access to secrets and exec, and generate least-privilege roles from observed usage.

```
kubectl get clusterrolebindings -o json | jq '.items[] | select(.roleRef.name=="cluster-admin") | .subjects'
```

```
kubectl auth can-i --list --as=system:serviceaccount:prod:app -n prod
```

```
kubectl who-can get secrets -n prod
```

```
rbac-tool lookup -e '.*admin.*'
```

Interview tip: Access to create pods, read secrets, or exec into pods is effectively privilege escalation, so treat those verbs as sensitive.

📚 Official docs

- Kubernetes: RBAC
- Linux audit documentation (auditd)

certs.eknathalabs.com

Q119 How do you secure the container runtime with gVisor, Kata Containers or user namespaces for untrusted workloads?

K8s / Container

🎙 Spoken answer · ~58 s

Standard containers share the host kernel, so a kernel vulnerability can allow an escape. For untrusted or multi-tenant workloads, such as running customer code or build jobs, I add isolation. gVisor implements a user-space kernel that intercepts and handles most system calls, greatly reducing the attack surface exposed to the host kernel. Kata Containers runs each pod inside a lightweight virtual machine with its own kernel, giving hardware-virtualisation isolation. Both are selected per pod through a RuntimeClass, so I can sandbox only the risky workloads. User namespaces are another layer: root inside the container maps to an unprivileged user on the host, so even a breakout has limited rights. The costs are overhead, startup time and incomplete syscall or device support, so I benchmark and test compatibility, and combine sandboxing with network policy, minimal permissions and node isolation.

Likely follow-ups

- How do you select a sandbox for a pod?
- What are the compatibility costs?

Sandboxed runtimes add a boundary between workload and host kernel: gVisor intercepts syscalls in user space, Kata runs pods in lightweight VMs, user namespaces map root in the container to an unprivileged host user. Use via RuntimeClass.

```
kubectl get runtimeclass
```

```
spec: runtimeClassName: gvisor
```

```
spec: hostUsers: false
```

```
kubectl run test --image=alpine --overrides='{"spec":{"runtimeClassName":"kata"}}' -- sleep 3600
```

Interview tip: Sandboxing trades some performance and compatibility for a stronger isolation boundary. Apply it where tenants are untrusted.

📚 Official docs

- Kubernetes: Container runtimes
- Kubernetes: User namespaces
- Kata Containers

certs.eknathalabs.com

Q120 How do you protect Kubernetes nodes with hardened OS images, minimal access and kubelet settings?

K8s / Container

🎙 Spoken answer · ~58 s

Worker nodes run my workloads and hold credentials, so a compromised node is a serious event. I use minimal, purpose-built node operating systems with an immutable root filesystem and automatic replacement rather than in-place patching, so drift and manual changes are eliminated. SSH access is disabled in favour of audited session tooling, and nodes sit in private subnets. The kubelet is configured with anonymous authentication off, authorization mode set to webhook so the API server decides, the read-only port disabled, and rotation of its certificates enabled. I benchmark against the CIS guidelines with tools like kube-bench. The cloud metadata endpoint is protected so pods cannot obtain the node's credentials, and nodes have only the minimum cloud permissions. Runtime detection watches for shells, unexpected network connections and file changes. Patching means rolling out a new image and draining old nodes.

Likely follow-ups

- Why replace nodes instead of patching?
- What does kube-bench check?

Use minimal immutable node OS images, disable SSH, restrict kubelet anonymous access and read-only port, enable authorization webhook mode, apply patches by replacing nodes, and monitor with runtime tools.

```
kubectl get --raw /api/v1/nodes/NODE/proxy/configz | jq '.kubeletconfig.authentication'
```

```
kube-bench run --targets node
```

```
curl -k https://NODE:10250/pods
```

```
kubectl drain NODE --ignore-daemonsets
```

Interview tip: Test that the kubelet port rejects unauthenticated requests. An open kubelet API means remote command execution on pods.

📚 Official docs

- Kubernetes: kubelet

certs.eknathalabs.com

Q121 How do you secure etcd, including encryption at rest, access control and backups?

K8s / Container

🎙 Spoken answer · ~60 s

etcd stores all cluster state, including Secrets, so access to it is equivalent to full cluster control. Only the API server should talk to it, over mutually authenticated TLS, with network rules that block everything else, and its peer traffic encrypted too. Kubernetes Secrets are stored base64 encoded, which is not protection, so I enable encryption at rest in the API server, ideally with an envelope scheme where a key management service holds the master key, so the data on disk and in snapshots is ciphertext. Backups are encrypted, stored with restricted access, tested for restore, and retention is limited. I audit who accesses etcd and the API, and rotate keys periodically. In managed services like EKS, GKE and AKS the provider operates etcd, but I still enable the available secrets encryption option with my own key to meet compliance and keep control.

Likely follow-ups

- Is base64 encryption?
- What is envelope encryption for secrets?

Restrict etcd to the API server with mTLS, isolate network access, enable encryption of Secrets at rest using a KMS provider, encrypt and access-control backups, and monitor access. Managed control planes handle much of this.

```
ETCDCTL_API=3 etcdctl --cacert ca.crt --cert client.crt --key client.key endpoint health
```

```
kubectl get secret db -o yaml
```

```
apiVersion: apiserver.config.k8s.io/v1 kind: EncryptionConfiguration resources: - resources: [secrets] providers: [kms: {name: kms, endpoint: unix:///kms.sock}]
```

```
etcdctl snapshot save backup.db
```

Interview tip: Base64 in a Secret is not encryption. Without encryption at rest, anyone with etcd or backup access reads every secret.

📚 Official docs

- Kubernetes: Encrypting Secret data at rest
- Kubernetes: Operating etcd clusters
- etcd: Disaster recovery

certs.eknathalabs.com

Q122 What is Kubernetes audit logging, and how do you design an audit policy that is useful without being overwhelming?

K8s / Container

🎙 Spoken answer · ~62 s

Audit logging answers who did what to which resource and when, which is essential for incident investigation and compliance. Each API request generates events through stages, and the audit policy decides how much to record per rule. The levels range from none, through metadata only, to the request body, to request and response bodies. Recording everything at the highest level would be huge and could capture sensitive data, so I design the policy deliberately. I drop high-volume noise like health checks and routine get and watch calls from system components, log secrets and config maps at metadata level only, and record full request bodies for sensitive changes such as RBAC modifications, pod creation, exec and port-forward, and admission webhook changes. The logs are shipped off the node to a SIEM with retention, and detection rules alert on suspicious patterns like exec into production pods or role binding changes.

Likely follow-ups

- Why not log secrets at RequestResponse?
- Which events deserve alerts?

Audit logs record API requests by who, what and when. A policy sets levels (None, Metadata, Request, RequestResponse) by resource. Log secrets at Metadata only, log writes in detail, drop noisy reads, and ship logs to a SIEM.

```
--audit-policy-file=/etc/kubernetes/audit-policy.yaml --audit-log-path=/var/log/audit.log
```

```
- level: Metadata resources: [{group: "", resources: ["secrets","configmaps"]}]
```

```
- level: RequestResponse verbs: ["create","patch","delete"] resources: [{group:
rbac.authorization.k8s.io}]
```

```
jq 'select(.verb=="create" and .objectRef.resource=="pods/exec")' audit.log
```

Interview tip: Never log request bodies for secrets, or the audit log itself becomes a store of secrets.

📚 Official docs

- Linux audit documentation (auditd)

certs.eknathalabs.com

Q123 How do you secure ingress and API gateways, including TLS, WAF integration and rate limiting?

K8s / Container

🎙 Spoken answer · ~58 s

Ingress is the front door of the cluster, so hardening it has high value. At the transport layer I terminate TLS with current protocol versions and strong ciphers, automate certificates, redirect HTTP to HTTPS and send HSTS. In front of or alongside the controller, a web application firewall filters common attacks, and I configure rate limits and connection limits per client to blunt brute force and denial of service. The gateway can enforce authentication, such as validating JWTs or client certificates, before traffic reaches services. Admin or internal paths are not exposed publicly. The controller itself runs with minimal permissions, since it often can read secrets across namespaces, in a dedicated namespace with network policy, and it is patched promptly because vulnerabilities in popular controllers are actively exploited. Configuration snippets from users are restricted via admission policy to avoid injection.

Likely follow-ups

- Why is the ingress controller high risk?
- Where should auth be enforced?

Terminate TLS with modern settings and managed certificates, enforce HTTPS and HSTS, add a WAF, rate-limit and authenticate at the edge, restrict admin paths, and keep ingress controllers patched and isolated.

```
nginx.ingress.kubernetes.io/ssl-redirect: "true"
```

```
nginx.ingress.kubernetes.io/limit-rps: "20"
```

```
kubectl get ingress -A -o wide
```

```
testssl.sh https://api.example.com
```

Interview tip: The ingress controller is internet-facing and highly privileged, so treat it as critical, patch quickly, and limit its RBAC.

📚 Official docs

- Kubernetes: Ingress
- RFC 6585: HTTP 429 Too Many Requests
- Amazon API Gateway

certs.eknathalabs.com

Q124 How do you handle image pull secrets and registry authentication securely in Kubernetes?

K8s / Container

🎙 Spoken answer · ~57 s

Pulling private images requires credentials, and static registry passwords stored as Kubernetes secrets are long-lived and widely readable. On cloud platforms I avoid them: the node or workload identity is granted read-only pull permission on the registry, and the kubelet obtains short-lived tokens through a credential provider. Where a static credential is unavoidable, for a third-party registry, I create a dedicated read-only robot account scoped to specific repositories, store it through the secrets management flow, and rotate it. Rather than adding the pull secret to every pod spec, I attach it to the service account in each namespace, so it is applied consistently. I use pull-through caches to limit exposure to public registries and rate limits. Namespaces have RBAC that prevents unrelated teams reading the secret, and policy forces images to come from approved registries only.

Likely follow-ups

- Why avoid static registry passwords?
- What is a pull-through cache?

Use workload or node identity for cloud registries to avoid static pull secrets, scope credentials read-only per registry, rotate them, and attach pull secrets to service accounts rather than each pod.

```
kubectl create secret docker-registry regcred --docker-server=registry --docker-username=u
--docker-password=p
```

```
kubectl patch sa default -p '{"imagePullSecrets":[{"name":"regcred"}]}'
```

```
aws ecr get-login-password | head -c 10
```

Interview tip: With cloud registries, node or workload IAM roles remove the need to store pull credentials in the cluster at all.

📚 Official docs

- Kubernetes: Good practices for Secrets
- Kubernetes: Images (pull policy, registries)

certs.eknathalabs.com

Q125 How do you write and test Rego policies with conftest and OPA unit tests? Policy-as-Code

🎙 Spoken answer · ~64 s

Rego is declarative, so I think in terms of rules that produce violation messages when conditions hold. I write each policy in its own file with a clear message that tells the developer how to fix the problem, not only what is wrong. For example, a rule that denies a deployment when the pod security context does not set run as non-root. I test policies like code: unit tests with the OPA test command feed sample inputs and assert that violations appear exactly when expected, including the passing case and edge cases such as missing fields, because an undefined value in Rego makes a rule silently not fire. In pipelines, conftest evaluates policies against Kubernetes manifests, Helm output or Terraform plan JSON, failing the build on violations. Policies live in a versioned repository with review, release tags and a documented exception mechanism, and I measure how often rules block changes to tune them.

Likely follow-ups

- Why test the allowed path?
- How do you document exceptions?

Write policies as deny rules over structured input, test them with opa test using fixtures for allowed and denied cases, and run conftest in CI against manifests or Terraform plan JSON. Include messages that explain the fix.

```
conftest test deployment.yaml -p policy/
```

```
opa test policy/ -v
```

```
opa eval -i input.json -d policy/ 'data.main.deny'
```

```
deny[msg] { input.kind == "Deployment"; not
input.spec.template.spec.securityContext.runAsNonRoot; msg := "pods must run as non-root" }
```

Interview tip: Test both the denied and allowed paths, and include edge cases like missing fields, because Rego treats undefined as false.

📚 Official docs

- Conftest
- Open Policy Agent documentation

certs.eknathalabs.com

Q126 What is Kyverno's verifyImages and how does it enforce signed images? Policy-as-Code

🎙 Spoken answer · ~58 s

Image signing is only useful if something checks it at deploy time. Kyverno's verifyImages rule does that inside the admission webhook. For images matching a registry pattern, it verifies the signature using either a static public key or keyless verification, where the policy states which identity provider issuer and which subject, such as a particular repository's release workflow, are acceptable. It can also check attached attestations, such as a vulnerability scan result or provenance, and require specific fields. A helpful side effect is that the rule can rewrite mutable tags to the immutable digest that was verified, preventing a swap between verification and pull. I roll the policy out in audit mode first to see what would fail, then enforce. Exceptions for system images are explicit, and the key or identity configuration is managed through GitOps and rotated carefully.

Likely follow-ups

- Why verify identity, not just a signature?
- What does rewriting to a digest prevent?

verifyImages is a Kyverno rule type that checks image signatures or attestations at admission, using cosign keys or keyless identities, and can mutate tags to digests. Unverified images are rejected.

```
kubectl get clusterpolicy
```

```
verifyImages: - imageReferences: ["registry.example.com/*"] attestors: - entries: -
keyless: { issuer: "https://token.actions.githubusercontent.com", subject:
"https://github.com/acme/*" }
```

```
cosign verify --certificate-identity-regexp 'https://github.com/acme/.*'
registry.example.com/app@sha256:abc
```

Interview tip: Verify against an identity (who built it), not just that some signature exists. Any attacker can sign with their own key.

📚 Official docs

- Sigstore documentation
- Kyverno documentation

certs.eknathalabs.com

Q127 How do you use ValidatingAdmissionPolicy and CEL as a native alternative to webhook-based policy engines?

Policy-as-Code

🎙 Spoken answer · ~60 s

Webhook-based policy engines add moving parts: a service that must be available, with latency and failure policy decisions. Kubernetes now offers validating admission policies, where rules are written as expressions in the common expression language and evaluated inside the API server itself. A policy resource defines the matching resources and the expressions that must be true, with messages for failure, and a separate binding says which namespaces or objects it applies to and whether to deny, warn or audit. Because evaluation is in-process there is no network hop, no outage risk from a webhook, and strong performance. It works well for straightforward constraints such as replica limits, required labels, forbidding hostPath or privileged containers. For richer needs such as mutation, generation, external data lookups, image verification or cross-resource logic, a policy engine like Kyverno or Gatekeeper remains useful, and they can be combined.

Likely follow-ups

- What are the limits of CEL?
- What is a policy binding?

ValidatingAdmissionPolicy lets the API server evaluate CEL expressions in-process, so no external webhook is needed. It is fast, reliable and simple for common checks, with bindings selecting where to apply.

```
kubectl get validatingadmissionpolicy,validatingadmissionpolicybinding
```

```
validations: - expression: "object.spec.replicas <= 10" message: "replicas must be 10 or fewer"
```

```
kubectl apply --dry-run=server -f deployment.yaml
```

Interview tip: No webhook means no availability dependency and no extra latency, but CEL has less power than Rego for complex logic.

📚 Official docs

- Kubernetes: ValidatingAdmissionPolicy

certs.eknathalabs.com

Q128 How do you build a policy exception process that is auditable and time-bound?

Policy-as-Code

🎙 Spoken answer · ~58 s

Rigid policy without exceptions gets bypassed, while unmanaged exceptions erode security. I make exceptions a first-class, reviewable object. A team opens a request describing the workload, the rule, the reason, the compensating controls and the owner. If approved, the exception is committed as code in a dedicated repository, using the policy engine's exception resource, scoped as narrowly as possible, for example one deployment in one namespace rather than a whole cluster. Each has an expiry date and a ticket reference. Automation lists active exceptions on a dashboard, notifies owners before expiry, and either removes the exception or opens a review when time is up. Metrics show the number of exceptions per policy, which tells me when a rule is unrealistic and should be changed. Because everything is in Git, the audit trail shows who approved what and when.

Likely follow-ups

- Why must exceptions expire?
- What do exception counts tell you?

Exceptions are requested as code, linked to a ticket and owner, approved by security, scoped narrowly, and given an expiry. Dashboards track active exceptions, and automation removes or flags expired ones.

```
kubectl get policyexception -A
```

```
apiVersion: kyverno.io/v2 kind: PolicyException metadata: { name: legacy-root, annotations:
{ expires: "2026-12-31", ticket: "SEC-1042" } }
```

```
kubectl get policyexception -A -o json | jq '.items[] |
select(.metadata.annotations.expires < "2026-10-21")'
```

Interview tip: An exception without an expiry becomes permanent policy. Make the default end date short.

📚 Official docs

- Kyverno: Policies (validate, mutate, generate)
- Kubernetes: Policies overview

certs.eknathalabs.com

Q129 How do you scan Helm charts and Kubernetes manifests in CI with kube-linter, kubesec, Trivy and Checkov?

Policy-as-Code

🎙 Spoken answer · ~59 s

Static analysis of manifests catches risky configuration before it reaches a cluster. For Helm, I render the chart with the real values file for each environment, because settings that matter, like security contexts or image tags, are often set through values. The rendered YAML goes through several tools: a linter that looks for best practice violations such as missing resource limits, probes or the latest tag, a misconfiguration scanner that flags privileged containers, host namespaces, and dangerous capabilities, and optionally a policy check with the same Rego or Kyverno rules used at admission, so developers see failures in the pull request instead of at deploy time. I set severity thresholds: high and critical findings fail the build, lower ones are reported. Findings link to documentation on how to fix, and suppressions require justification in code. Periodic scans of live clusters catch drift.

Likely follow-ups

- Why render with production values?
- Why align CI rules with admission rules?

Render charts to manifests, then run linters and scanners for misconfigurations such as privileged pods, missing limits and latest tags. Fail on high severity, report the rest, and keep rules consistent with admission policy.

```
helm template release ./chart -f values-prod.yaml > rendered.yaml
```

```
kube-linter lint rendered.yaml
```

```
trivy config rendered.yaml --severity HIGH,CRITICAL --exit-code 1
```

```
checkov -f rendered.yaml --framework kubernetes
```

Interview tip: Scan the rendered output with production values, since misconfigurations often come from values files, not the chart defaults.

📚 Official docs

- Helm: Chart best practices
- Trivy
- Checkov

certs.eknathalabs.com

Q130 What is the difference between preventive, detective and corrective controls in policy as code?

Policy-as-Code

🎙 Spoken answer · ~57 s

These describe where in time a control acts. Preventive controls stop non-compliant changes from happening: admission policies that reject a privileged pod, CI checks that fail a build, or organisation-level guardrails that deny creating a public bucket. Detective controls observe the state afterwards and report deviations, such as posture management scanning cloud accounts, config rules evaluating resources, drift detection, and audit-mode policies. They are essential because not every path is covered by prevention, and because old resources predate the rules. Corrective controls respond: an automation that removes a public ACL, quarantines a pod, or opens a ticket with an SLA for the owning team. A strong program layers them. I start with detection to learn the baseline, move high-confidence rules to prevention, and automate correction for well-understood cases while keeping humans in the loop for risky ones.

Likely follow-ups

- Why do you need detective controls too?
- What can safely be auto-remediated?

Preventive controls block bad changes (admission, CI gates, SCPs), detective controls find issues after the fact (CSPM, audits, drift detection), corrective controls fix automatically or via remediation workflow. Mature programs layer all three.

```
aws organizations list-policies --filter SERVICE_CONTROL_POLICY
```

```
kubectl get constraints
```

```
aws configservice describe-compliance-by-config-rule
```

```
kubectl get clusterpolicy -o jsonpath='{.items[*].spec.validationFailureAction}'
```

Interview tip: Preventive controls are cheapest per incident, but detective ones are needed for what you could not or did not prevent.

📚 Official docs

- Open Policy Agent documentation

certs.eknathalabs.com

Q131 How do you enforce tagging, ownership and cost-allocation standards through policy?

Policy-as-Code

🎙 Spoken answer · ~60 s

Consistent metadata makes security and finance operations work: when a scanner finds an exposed bucket, the owner tag tells me who to contact, and cost reports depend on allocation tags. I define a small mandatory set, such as owner team, environment, service and cost centre, with allowed values. Enforcement happens at several layers. In pull requests, plan checks written in Rego fail if a resource lacks required tags. In the cloud, tag policies and organisation guardrails can reject creation of untagged resources or standardise formats, and modules inject default tags so developers rarely forget. For existing resources, scheduled queries find gaps and report them by team. Some tags can be inherited from the account or resource group. The result is that every finding and every dollar maps to an accountable owner, and dashboards show compliance percentage, which creates healthy pressure to fix it.

Likely follow-ups

- Why do tags matter for security?
- How do modules help consistency?

Define required tags, enforce on creation via IaC policy checks and cloud tag policies or SCPs, detect gaps with scanning, and tie tags to ownership for alerts and cost reports. Inherit tags automatically where possible.

```
aws organizations create-policy --type TAG_POLICY --name required-tags --content
file://tags.json
```

```
az policy assignment create --name require-owner --policy 'Require a tag on resources'
```

```
conftest test plan.json -p policy/tags.rego
```

```
gcloud asset search-all-resources --query='-labels:owner'
```

Interview tip: Tags let a security finding find its owner. An unowned resource is an unfixed finding.

📚 Start here (official docs)

- Kubernetes security checklist

certs.eknathalabs.com

Q132 How do you use AWS Organizations SCPs, Azure Policy and GCP organisation policies as preventive guardrails?

Cloud Security

🎙 Spoken answer · ~56 s

Identity policies grant permissions to principals, but a single misconfigured admin can still do anything within an account. Organisation-level guardrails sit above and define what is impossible regardless of IAM. In AWS, service control policies attached to organisational units deny actions such as disabling CloudTrail, leaving the organisation, using unapproved regions, or creating unencrypted volumes. Azure Policy assigns rules at management group or subscription level, to deny, audit or automatically remediate resources. Google organisation policies apply constraints like disabling service account key creation or restricting public IPs. I apply them through a landing zone design with environments separated into different OUs, start in audit or in a sandbox, then enforce, with a documented exception process. They don't grant access, only limit it, so effective permission is the intersection of the guardrail and the identity policy.

Likely follow-ups

- Do SCPs grant permissions?
- How do you roll out a new deny safely?

They set boundaries above individual accounts or projects: deny dangerous actions, restrict regions and services, require encryption. They limit what even admins can do inside, and complement IAM rather than replace it.

```
aws organizations list-policies --filter SERVICE_CONTROL_POLICY
```

```
az policy assignment list --scope /providers/Microsoft.Management/managementGroups/prod
```

```
gcloud resource-manager org-policies list --organization=ORG_ID
```

```
{"Effect":"Deny","Action":"*","Resource":"*","Condition":{"StringNotEquals":
{"aws:RequestedRegion":["eu-west-1"]}}}
```

Interview tip: Guardrails set the maximum permissions, they don't grant any. Test them in a sandbox OU first because a bad deny can break pipelines.

📚 Official docs

- AWS Organizations: Service control policies
- AWS Organizations: What is AWS Organizations?
- AWS Control Tower

certs.eknathalabs.com

Q133 How do you secure serverless and managed container platforms (Lambda, Cloud Run, Fargate) differently from VMs?

Cloud Security

🎙 Spoken answer · ~61 s

With serverless and managed containers, the provider handles patching of the operating system and runtime isolation, so my responsibility moves up the stack. I focus on identity: each function or service gets its own role with only the permissions it needs, because one compromised function with broad rights becomes a pivot to the account. Input is the main attack surface, since events from queues, buckets and APIs may carry attacker-controlled data, so I validate and treat all event fields as untrusted to prevent injection. Dependencies are scanned, since the package contents are my code. Secrets come from a secrets manager at runtime rather than environment variables in the template. I limit network egress with private connectivity and restrict outbound access, set reasonable timeouts and concurrency limits to contain abuse and cost, enable structured logging and tracing, and use code signing or image verification for deployment integrity.

Likely follow-ups

- Why per-function roles?
- What is the attack surface of an event-driven function?

The provider secures the host, so focus shifts to code, dependencies, per-function IAM roles, secrets handling, event input validation, network egress controls and logging. Use least privilege per function and short timeouts.

```
aws lambda get-function --function-name fn --query 'Configuration.Role'
```

```
aws iam simulate-principal-policy --policy-source-arn $ROLE --action-names s3:DeleteBucket
```

```
gcloud run services describe svc --format='value(spec.template.spec.serviceAccountName)'
```

Interview tip: One shared over-privileged role for all functions is the classic serverless mistake. Give each function its own scoped role.

📚 Official docs

- Cloud Run
- AWS Lambda Developer Guide
- Amazon ECS Developer Guide

certs.eknathalabs.com

Q134 How do you use IAM Access Analyzer, Policy Sentinel-style tools or CIEM to find unused and external access?

Cloud Security

🎙 Spoken answer · ~56 s

Over time, cloud permissions accumulate far beyond what workloads use. Analyzer tools help in two ways. External access analysis reasons about resource policies and tells me which buckets, roles, keys or queues can be reached from outside my account or organisation, which finds accidental sharing. Unused access analysis compares granted permissions with actual activity from audit logs and reports roles, keys and permissions that haven't been used for, say, ninety days. Policy generation goes further, producing a least-privilege policy from observed behaviour that I review and apply. Cloud infrastructure entitlement management products extend this across multiple clouds. I run these continuously, send findings to owners with severity and SLA, and automate cleanup where safe, such as disabling long-unused access keys. Reviews combine tooling output with human judgement, since rarely used break-glass permissions should remain.

Likely follow-ups

- What does external access analysis find?
- How do you handle rarely used break-glass roles?

Analyzers find resources shared outside the account or organisation and permissions unused over a period, and generate least-privilege policies from CloudTrail activity. Run continuously and route findings to owners.

```
aws accessanalyzer list-findings --analyzer-arn $ARN --filter '{"status":{"eq":
["ACTIVE"]}}'
```

```
aws accessanalyzer start-policy-generation --policy-generation-details principalArn=$ROLE
```

```
aws iam generate-service-last-accessed-details --arn $ROLE
```

```
gcloud recommender recommendations list --recommender=google.iam.policy.Recommender
```

Interview tip: Unused permissions and unused roles are free attack surface. Remove them on a schedule.

📚 Official docs

- AWS IAM: Policy evaluation logic
- Conftest
- Redis: Sentinel

certs.eknathalabs.com

Q135 How do you design and secure cross-account access in AWS using roles and external IDs?

Cloud Security

🎙 Spoken answer · ~60 s

Instead of sharing credentials between accounts, I create a role in the target account whose trust policy names which principals in other accounts may assume it. Callers request temporary credentials and their activity is logged against the session. To tighten the trust, I specify exact roles rather than a whole account root where possible, add conditions requiring multi-factor authentication for human access, and restrict to principals in my own organisation using the organisation ID condition key. When granting access to a third-party vendor, an external ID adds protection against the confused deputy problem: the vendor, who serves many customers, must present the unique ID I gave them, so another customer cannot trick the vendor's system into acting on my account. Permissions on the role are least privilege, session duration is short, and CloudTrail events from role assumption are monitored for unusual sources or times.

Likely follow-ups

- What is the confused deputy problem?
- Why short session durations?

Use IAM roles assumed via sts:AssumeRole, with trust policies naming exact principals, conditions such as MFA or source organisation, and external IDs for third parties to prevent confused deputy attacks. Avoid long-lived user keys.

```
aws sts assume-role --role-arn arn:aws:iam::222233334444:role/audit --role-session-name me
```

```
{"Principal":
{"AWS":"arn:aws:iam::111122223333:root"},"Action":"sts:AssumeRole","Condition":
{"StringEquals":{"sts:ExternalId":"abc123"}}}
```

```
aws iam get-role --role-name audit --query Role.AssumeRolePolicyDocument
```

Interview tip: The external ID stops a third party from being tricked into using their access to your account on someone else's behalf.

📚 Official docs

- AWS IAM: Roles

certs.eknathalabs.com

Q136 How do you secure DNS and prevent subdomain takeover?

Cloud Security

🎙 Spoken answer · ~59 s

A subdomain takeover happens when a DNS record still points to a cloud service resource that has been deleted, such as a storage website, a PaaS app or a CDN endpoint, and an attacker claims the same name on that service. They then serve content on my trusted domain, enabling phishing, cookie theft or malware delivery. Prevention is lifecycle discipline: create and remove DNS records in the same infrastructure as code that manages the resource, so deleting one deletes the other. Detection is continuous: enumerate all records, resolve them, and flag CNAMEs that point to services returning a not-found or unclaimed error. Many clouds support domain verification, so a claim requires proving ownership of the domain. I also protect the registrar account with strong authentication and a registry lock, enable DNSSEC where appropriate, and limit who can change zones with audited access.

Likely follow-ups

- How does an attacker claim the resource?
- Why manage DNS in the same IaC?

Inventory DNS records, remove dangling CNAMEs pointing to deprovisioned cloud resources, monitor for them continuously, use DNSSEC and registrar locks, and tie DNS changes to resource lifecycle in IaC.

```
dig +short CNAME old.example.com
```

```
curl -sI https://old.example.com | head -3
```

```
subjack -w subdomains.txt -t 50 -ssl
```

```
aws route53 list-resource-record-sets --hosted-zone-id Z123 --query "ResourceRecordSets[?Type=='CNAME']"
```

Interview tip: Delete the DNS record in the same change that deletes the cloud resource it points to.

📚 Start here (official docs)

- Kubernetes security checklist

certs.eknathalabs.com

Q137 How do you build a cloud incident response capability with forensic snapshots, isolation and log preservation?

Cloud Security

🎙 Spoken answer · ~59 s

Good incident response in the cloud depends on preparation. Logs from audit trails, network flows and application sources go to a separate, write-protected account with retention, so an attacker in a workload account cannot erase them. I keep runbooks and automation ready for common cases. For a suspect instance, the first move is containment by replacing its security groups with a quarantine group that blocks traffic but leaves it running, and revoking or disabling any credentials it holds, rather than deleting it. Then I capture evidence: snapshot the volumes, take a memory image if tooling exists, and copy the data into a locked forensic account with object lock, recording hashes and who handled what for chain of custody. Analysis happens on copies in an isolated environment. Afterwards I eradicate, rebuild from clean images, rotate credentials, and write a post-incident review with improvements.

Likely follow-ups

- Why not terminate the instance?
- Why a separate logging account?

Prepare in advance: centralised immutable logs, a dedicated forensics account, runbooks, and automation to isolate (quarantine security group, disable keys), snapshot disks and memory, and preserve evidence with chain of custody.

```
aws ec2 modify-instance-attribute --instance-id i-0abc --groups sg-quarantine
```

```
aws ec2 create-snapshot --volume-id vol-0abc --description 'IR case 1042'
```

```
aws iam update-access-key --access-key-id AKIA... --status Inactive --user-name suspect
```

```
aws s3api put-object-lock-configuration --bucket ir-evidence
```

Interview tip: Isolate, don't terminate. Terminating a compromised instance destroys memory and volatile evidence.

📚 Official docs

- Kubernetes: Security context
- Linux man-pages: capabilities(7)
- Google SRE Book: Managing Incidents

certs.eknathalabs.com

Q138 How do you handle authentication and session security with OAuth2, OIDC and JWT correctly? AppSec

🎙 Spoken answer · ~63 s

OAuth2 handles delegated authorisation and OpenID Connect adds identity on top. For browser and mobile apps I use the authorization code flow with PKCE, which prevents interception of the code, and avoid the implicit flow. When my API receives a JWT access token, I must validate it fully: verify the signature against the issuer's published keys, check the issuer and audience claims match my service, check expiry and not-before times, and confirm required scopes. I pin the allowed signing algorithms to avoid algorithm confusion attacks, including the none algorithm. Access tokens are short-lived, and refresh tokens rotate with reuse detection so a stolen refresh token is noticed. Tokens are stored in secure, httpOnly cookies or platform secure storage rather than local storage where scripts can read them. JWT payloads are only encoded, not encrypted, so I do not put sensitive data in them, and I have a revocation strategy for compromised sessions.

Likely follow-ups

- Why PKCE?
- What is algorithm confusion?

Use authorization code flow with PKCE, validate issuer, audience, signature and expiry, keep tokens short-lived with refresh rotation, store them safely, and never put sensitive data in unencrypted JWTs.

```
curl -s https://issuer/.well-known/openid-configuration | jq .jwks_uri
```

```
jwt decode $TOKEN
```

```
code_challenge=$(echo -n $VERIFIER | openssl dgst -sha256 -binary | base64 | tr '+/' '-_' |
tr -d '=')
```

Interview tip: Reject tokens with alg none and pin accepted algorithms. Never trust the header to choose verification behaviour.

📚 Official docs

- GitHub Actions: OpenID Connect
- OWASP Cheat Sheet Series
- OAuth 2.0 / OIDC overview (OpenID)

certs.eknathalabs.com

Q139 How do you secure file uploads and prevent SSRF, path traversal and deserialization vulnerabilities?

AppSec

🎙 Spoken answer · ~60 s

These are three separate classes with a shared root cause: trusting input that the attacker controls. For file uploads I check the real content type rather than the extension or header, limit size, generate a random server-side name, store files in object storage or a directory outside the web root with execution disabled, and scan for malware. Path traversal is prevented by never concatenating user input into file paths; I resolve the final path and verify it stays within the intended base directory. Server-side request forgery occurs when the app fetches a user-supplied URL, so I use an allowlist of hosts, resolve DNS and block private, loopback and link-local addresses including the cloud metadata address, disable redirects or revalidate them, and enforce metadata service v2 style protections. For deserialization I avoid native object formats for untrusted data and use simple data formats with schema validation.

Likely follow-ups

- Why block the metadata address?
- What is safe to deserialize?

For uploads: validate type by content, limit size, rename files, store outside the web root with no execute permission. For SSRF: allowlist destinations and block internal ranges. Avoid unsafe deserialization and canonicalise paths.

```
file --mime-type upload.bin
```

```
realpath --relative-base=/srv/uploads "$path"
```

```
curl http://169.254.169.254/latest/meta-data/   # must be blocked from app pods
```

```
grep -R 'pickle.loads\|ObjectInputStream' src/
```

Interview tip: Never trust the filename or Content-Type header from a client. Both are attacker-controlled.

📚 Official docs

- OWASP Top Ten

certs.eknathalabs.com

Q140 What is software composition analysis and how do you prioritise findings beyond CVSS scores?

AppSec

🎙 Spoken answer · ~58 s

Software composition analysis compares the libraries in my code against vulnerability databases. Raw output can be thousands of findings, many irrelevant, so prioritisation is the real work. CVSS describes severity in theory but not likelihood. I add several signals: is a public exploit available and is it being exploited in the wild, as shown by the known exploited list and the exploit prediction score; is the vulnerable function actually reachable from my code, which reachability analysis can estimate; is the service internet-facing or internal; what data does it handle; and is a fixed version available. A high CVSS in a test-only dependency matters less than a medium one in a public authentication service. I set SLAs by combined risk, automate fix pull requests, suppress findings with documented not-affected statements using VEX, and track time to remediate as a metric.

Likely follow-ups

- What is EPSS?
- What is reachability analysis?

SCA finds known vulnerabilities in dependencies. Prioritise by reachability, exploit availability (EPSS, KEV), exposure of the service, data sensitivity and fix availability, not CVSS alone.

```
trivy fs --scanners vuln .
```

```
osv-scanner -r .
```

```
grype dir:. --only-fixed
```

```
curl -s https://api.first.org/data/v1/epss?cve=CVE-2021-44228
```

Interview tip: CISA's known exploited vulnerabilities list is a strong signal that a CVE is being used in attacks right now.

📚 Official docs

- Trivy

certs.eknathalabs.com

Q141 How do you perform threat modeling with STRIDE on a new feature, and make it lightweight for agile teams?

AppSec

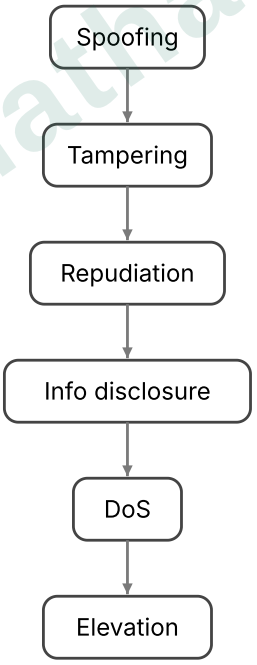
🎙 Spoken answer · ~59 s

Threat modeling is structured thinking about what can go wrong before code is written. For a new feature I sketch a simple data flow diagram showing actors, services, data stores and flows, and draw the trust boundaries, such as internet to service or service to database. Then for each element and flow I walk through STRIDE: can someone spoof an identity, tamper with data, deny having done something, disclose information, disrupt availability, or elevate privileges. Each plausible threat gets a mitigation, an accepted-risk decision, or a follow-up ticket, with an owner. To keep it agile, I time-box sessions to under an hour with developers present, reuse templates and previous models, trigger it only for changes that alter data flows or trust boundaries, and store the model in the repository next to the code. The output feeds security test cases and acceptance criteria.

Likely follow-ups

- What is a trust boundary?
- When do you trigger a new model?

🗺 STRIDE



Draw a data flow diagram, mark trust boundaries, apply STRIDE (spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege) to each element, record mitigations and track them as tickets. Keep sessions short and recurring.

```
pytm --dfd threat_model.py | dot -Tpng -o dfd.png
```

```
threatdragon # open model file
```

certs.eknathalabs.com

```
gh issue create --title 'Mitigate: JWT replay on /refresh' --label threat-model
```

Interview tip: Focus on trust boundaries. The interesting threats are where data crosses from one trust level to another.

📚 **Official docs**

- MITRE ATT&CK for Containers

Q142 How do you secure GraphQL and gRPC APIs differently from REST?

AppSec

🎙 **Spoken answer · ~57 s**

The principles stay the same, but the attack surface shifts. GraphQL exposes one endpoint where clients shape queries, so a malicious client can craft deeply nested or expensive queries, or send many aliased operations in one request to bypass rate limits. I enforce limits on depth, complexity and batch size, set timeouts, and disable or restrict introspection in production. Authorisation cannot rely on URL paths, so resolvers check access for each object and field. Errors should not leak internals. For gRPC, which typically runs between services, I require mutual TLS, authenticate callers with tokens or workload identity, authorise per method, set maximum message sizes and deadlines, and turn off server reflection in production. For both, I still validate every input against the schema, apply rate limiting at the gateway, log sensible metadata and test with API-aware fuzzers.

Likely follow-ups

- How can aliasing bypass rate limits?
- Why disable reflection?

GraphQL needs query depth and complexity limits, introspection control, per-field authorisation and batching limits. gRPC needs mTLS, per-method authz, message size limits and reflection control. Both still need validation and rate limiting.

```
query { a { b { c { d } } } }   # depth attack
```

```
graphql-armor --max-depth 7 --max-cost 1000
```

```
grpcurl -plaintext host:50051 list
```

```
grpcurl -cacert ca.pem host:443 describe svc.Service
```

Interview tip: GraphQL's single endpoint hides authorisation gaps. Check permissions per field and per object, not per URL.

📚 **Official docs**

- gRPC documentation

certs.eknathalabs.com

Q143 What are secure coding guardrails for AI-assisted or third-party generated code in the pipeline?

AppSec

🎙 **Spoken answer · ~59 s**

Code from an assistant, a snippet from the internet or a vendored sample can contain vulnerabilities, outdated APIs, hard-coded secrets, or licence obligations, and it can look convincing. The guardrail is that nothing bypasses the normal pipeline. It goes through pull request review by an engineer who actually understands the change, automated tests that exercise behaviour, static analysis for insecure patterns, dependency and licence scanning, and secret detection before commit and in CI. Developers must not paste confidential data or credentials into external tools, so I set policy and provide approved, access-controlled tooling. Any new dependency suggested must be vetted, since invented or typosquatted package names are a real risk. Where tools act with access to repositories or shells, I limit their permissions, run them in sandboxes, and log their actions. Metrics on defects and findings help calibrate the review level.

Likely follow-ups

- Why vet suggested dependencies?
- What must reviewers verify?

Treat generated or copied code as untrusted: run the same SAST, SCA, secret scanning and review as human code, require tests, restrict secrets exposure to tools, and check licences and provenance.

```
semgrep --config auto --error src/
```

```
gitleaks protect --staged
```

```
trivy fs --scanners vuln,secret,license .
```

```
git log --format='%an %s' --since=1.week | grep -i 'generated'
```

Interview tip: The review burden shifts to verifying behaviour. Require tests and a human who understands the change before merge.

📚 **Official docs**

- AWS Control Tower
- OWASP Cheat Sheet Series

certs.eknathalabs.com

Q144 How do you implement rate limiting, bot protection and abuse prevention for public endpoints?

AppSec

🎙 Spoken answer · ~58 s

Public endpoints get abused by credential stuffing, scraping, enumeration and resource exhaustion. I use several layers. At the edge, a CDN or web application firewall provides coarse per-IP limits, managed bot detection and DDoS absorption. In the application or gateway, I apply finer limits keyed by authenticated user, API key or tenant, using algorithms like token bucket that allow short bursts but cap sustained rates, and return 429 with a retry hint. Sensitive flows such as login, password reset and sign-up get stricter limits, progressive delays, lockout with care to avoid enabling denial of service against real users, and challenges like CAPTCHA when risk is high. I add detection of anomalies: many failed logins across accounts, enumeration patterns, unusual user agents. Monitoring dashboards and alerts show spikes, and I review limits regularly against real traffic to avoid blocking customers.

Likely follow-ups

- Why not limit only by IP?
- What are the risks of account lockout?

Apply layered limits at the edge and the application per IP, per user and per token, with token bucket algorithms, progressive challenges, anomaly detection, and returning 429 with Retry-After. Protect sensitive flows such as login and signup more tightly.

```
limit_req_zone $binary_remote_addr zone=login:10m rate=5r/m;
```

```
curl -i https://api.example.com/login   # expect 429 + Retry-After
```

```
aws wafv2 list-rule-groups --scope REGIONAL
```

```
kubectl get envoyfilter -A
```

Interview tip: Limit by identity (account, API key) as well as IP, since attackers rotate IPs and many legitimate users share one.

📚 Official docs

- RFC 6585: HTTP 429 Too Many Requests
- Google SRE Book: Handling Overload
- Microsoft: Cloud design pattern – Throttling

certs.eknathalabs.com

Q145 How do you map security controls to compliance frameworks like SOC 2, ISO 27001 and PCI DSS without duplicating effort?

IR & Governance

🎙 Spoken answer · ~60 s

Frameworks overlap heavily: access control, change management, logging, encryption, vulnerability management and incident response appear in all of them in different words. So I define one internal control set, written in plain language, and map each control to the specific clauses of each framework in a matrix. Engineering implements the control once, for example required reviews for production changes, and the same evidence satisfies several requirements. The efficient part is automation of evidence: configuration queries, policy-as-code results, pull request approvals, vulnerability scan reports and access review exports are collected on a schedule and stored, so an audit samples live records rather than relying on screenshots gathered in a rush. Continuous control monitoring shows when a control drifts, and owners get alerts. Compliance is the minimum baseline; the real objective is actual risk reduction, and I avoid box-ticking controls that provide no security value.

Likely follow-ups

- What evidence is strongest for an auditor?
- Why is compliance not equal to security?

Build one control library, map each control to multiple framework requirements, automate evidence collection from systems, and run continuous control monitoring. Audit once, satisfy many.

```
aws configservice get-compliance-summary-by-config-rule
```

```
inspec exec cis-baseline -t aws://
```

```
steampipe query "select * from aws_s3_bucket where not versioning_enabled"
```

```
git log --since=90.days --format='%h %an %s' -- policy/
```

Interview tip: Evidence from Git history, tickets and system configuration is stronger and cheaper than screenshots collected before the audit.

📚 Official docs

- NIST Cybersecurity Framework

certs.eknathalabs.com

Q146 How do you run effective access reviews and manage the joiner-mover-leaver lifecycle?

IR & Governance

🎙 Spoken answer · ~57 s

Access should follow roles that follow people's jobs. Joiners receive baseline access automatically through groups linked to the identity provider and HR record, with additional access requested and approved. Movers lose permissions of their old role as they gain the new one, which is where privilege creep usually starts. Leavers are deprovisioned immediately and completely, including SaaS accounts, API keys and cloud roles, ideally triggered by the HR event. Periodic access reviews ask managers or resource owners to confirm each person still needs their access; for privileged access I review monthly or quarterly, and for standard access less often. Review packets must be clear and prefilled with last-used data to help reviewers decide. Decisions and removals are recorded as audit evidence. I reduce workload by using time-limited, just-in-time elevation and by auto-expiring access that has been inactive.

Likely follow-ups

- What causes privilege creep?
- How does just-in-time access help?

Automate provisioning from the HR system, use role-based groups, review privileged access frequently and general access periodically, remove access on moves and exits immediately, and keep evidence of reviewers' decisions.

```
aws iam generate-credential-report && aws iam get-credential-report
```

```
aws iam list-users --query 'Users[?PasswordLastUsed<`2026-07-01`].UserName'
```

```
gh api orgs/acme/members --paginate -q '.[].login'
```

```
gcloud asset search-iam-policies --scope=organizations/ID --query='policy:user:*'
```

Interview tip: Access that nobody reviews tends to grow forever. Make inactive access expire by default.

📚 Start here (official docs)

- Kubernetes security checklist

certs.eknathalabs.com

Q147 How do you design a vulnerability management SLA policy and handle exceptions?

IR & Governance

🎙 **Spoken answer · ~58 s**

A policy turns scattered patching into a managed process. I define target remediation times based on risk: critical vulnerabilities on internet-facing systems or those with known exploitation get a very short window, such as a few days; high severity gets a couple of weeks; medium and low are handled within regular maintenance cycles. Exposure and exploit information can adjust the deadline up or down. Findings flow automatically into the teams' backlogs with an owner. If a fix cannot be applied in time, the team requests an exception that states the reason, the compensating controls such as network isolation or a virtual patch, the risk owner who accepts it, and a firm expiry for re-evaluation. Reporting covers overdue counts, mean time to remediate and SLA compliance per team, reviewed regularly with engineering leadership, with escalation paths for repeated breaches.

Likely follow-ups

- How do compensating controls work?
- What metrics show program health?

Set remediation deadlines by severity and exposure (for example critical internet-facing in days, low in quarter), measure compliance, require risk-accepted exceptions with owner, compensating controls and expiry, and escalate breaches.

```
trivy image --severity CRITICAL --format json app:1.4 | jq '.Results[].Vulnerabilities | length'
```

```
gh api repos/org/repo/dependabot/alerts -q '.[] | select(.state=="open") | .security_advisory.severity' | sort | uniq -c
```

```
jq '.[] | select(.due < now)' findings.json
```

Interview tip: Track mean time to remediate and the percentage within SLA per team. Trends show whether the program is working.

📚 **Official docs**

- Google SRE Book: Service Level Objectives

certs.eknathalabs.com

Q148 How do you run a security champions program and embed security into engineering teams at scale?

IR & Governance

🎙 Spoken answer · ~60 s

A central security team cannot review everything, so I extend its reach through champions: one interested engineer in each product team who acts as the local point of contact. I give them structured training, access to tools and early information on new risks, regular community sessions, and time in their workload to do the job, and I recognise the contribution in performance conversations. Champions run threat modeling sessions, help triage scanner findings and review risky changes. Equally important is making the secure way the easy way: paved road templates, shared pipeline components with scanning built in, vetted libraries for authentication and encryption, and clear short guidance. I measure outcomes such as vulnerabilities found early, time to fix, and training participation, and share them transparently. The culture goal is that security is viewed as a quality attribute owned by engineering, with security acting as enabler.

Likely follow-ups

- What do champions need to succeed?
- What is a paved road?

Recruit volunteer engineers per team, give them training, tooling and a direct line to security, align incentives, share metrics, and make secure paths the easy default through paved roads and templates.

```
gh repo create org/secure-service-template --template
```

```
gh api orgs/acme/teams/security-champions/members -q '.[].login'
```

```
semgrep ci --config p/owasp-top-ten
```

Interview tip: Champions amplify the security team, but only if they get time allocated and recognition for the role.

📚 Official docs

- OWASP DevSecOps Guideline

certs.eknathalabs.com

Q149 What is a security posture dashboard and which metrics should it show to executives and engineers?

IR & Governance

🎙 Spoken answer · ~62 s

Different audiences need different views from the same data. For executives I show a small set of trends tied to risk: open critical and exploitable vulnerabilities and how fast they are closed, percentage of services covered by key controls such as scanning, MFA and logging, incident counts and detection and response times, and audit and compliance status. For engineering teams the dashboard is operational: findings grouped by service and owner with severity, age, SLA status and a link to the fix, so they can act without hunting. I avoid vanity metrics like total scans run or raw vulnerability count, which can rise simply because coverage improved, and prefer risk-weighted measures, mean time to remediate and the ratio of issues caught before production. Data comes from the scanners and cloud posture tools via a common pipeline, with consistent ownership tags, and each metric has a defined meaning and a target.

Likely follow-ups

- Why avoid raw vulnerability counts?
- What data feeds the dashboard?

Executives need risk trend, time to remediate critical issues, coverage, incident metrics and compliance status. Engineers need actionable per-service findings, SLA status and ownership. Avoid vanity counts.

```
jq '[.findings[]|select(.severity=="CRITICAL")]|length' findings.json
```

```
promql: sum by (team) (vuln_open{severity="critical"})
```

```
promql: histogram_quantile(0.5, sum(rate(vuln_remediation_seconds_bucket[30d])) by (le))
```

Interview tip: A single big number such as total vulnerabilities rewards the wrong behaviour. Show risk-weighted and trend-based metrics.

📚 Official docs

- Kubernetes: Security checklist
- Grafana documentation

certs.eknathalabs.com

Q150 A developer pastes a production database password into a public GitHub issue. What do you do in the first hour?

Secrets Mgmt

🎙 Spoken answer · ~62 s

I treat the password as compromised from the moment it was posted. Step one is rotation: change the database credential and update the consumers, using the secret manager so the rollout is quick and safe. In parallel I ask the platform to delete or redact the comment and its edit history, and request cache purges, but I do not rely on that since automated scrapers collect public content within minutes. Then I investigate exposure: database logs and cloud audit logs for connections from unknown addresses since the time of posting, unusual queries or exports, and whether the account had network reachability from the internet at all. If misuse is found, it becomes a full incident with containment. I notify the data owner and, depending on data and regulations, privacy or legal teams. Afterwards I add secret scanning and push protection, correct the process, and coach the developer without blame.

Likely follow-ups

- Why doesn't deleting the comment fix it?
- What logs do you check?

Rotate the credential immediately, remove or redact the content (knowing it is already scraped), check database logs for use since exposure, invalidate sessions, and then fix the process with scanning and training.

```
aws secretsmanager rotate-secret --secret-id prod/db
```

```
psql -c "select usename, client_addr, backend_start from pg_stat_activity"
```

```
gh api -X DELETE repos/org/repo/issues/comments/123
```

```
aws cloudtrail lookup-events --start-time 2026-10-20T00:00:00Z
```

Interview tip: Redaction does not help. Public content is scraped within minutes, so rotation is the only real remediation.

📚 Start here (official docs)

- Kubernetes security checklist

certs.eknathalabs.com

Q151 A scanner flags a critical CVE in a base image used by 80 services. How do you drive remediation at scale?

Supply Chain

🎙 Spoken answer · ~63 s

First I validate the finding: is the vulnerable component actually present and reachable in how our services use it, is there a known exploit, and which services are internet-facing. That sets urgency. Next I fix it once at the source: the platform team rebuilds the golden base image with the patched package, scans and signs it, and publishes a new digest. Then I use automation to propagate the change: a dependency bot opens pull requests in each repository bumping the base digest, and CI rebuilds, tests and deploys. I prioritise the exposed and most critical services, track progress on a dashboard listing which services still run the old digest, and chase owners of stragglers. To prevent indefinite lag, a policy blocks deployment of images built on bases older than a defined age after a deadline. If patching is delayed, I add compensating controls such as WAF rules or network restrictions.

Likely follow-ups

- How do you find all services using the old base?
- What are compensating controls?

Confirm exploitability and exposure, rebuild the golden image with the patch, let a bot open PRs across services, prioritise internet-facing ones, track progress on a dashboard, and use admission policy to block old digests after the deadline.

```
syft registry/golden/python:3.12 -o json | jq '.artifacts[]|select(.name=="libfoo")'
```

```
grype registry/golden/python:3.12 --only-fixed
```

```
gh search prs --owner acme 'bump base image' --state open
```

```
kubectl get pods -A -o jsonpath='{..image}' | tr ' ' '\n' | sort | uniq -c | grep old-digest
```

Interview tip: Central base image plus automated update PRs turns an 80-repo problem into one patch and one rollout.

📚 Official docs

- Trivy

certs.eknathalabs.com

Q152 Falco alerts that a shell was spawned in a production pod and an outbound connection to an unknown IP was made. Walk through your response.

K8s / Container

🎙 **Spoken answer · ~61 s**

A shell in a production container combined with an unexpected outbound connection is a strong indicator of compromise, so I act on it. First I contain without destroying evidence: apply a quarantine network policy that blocks the pod's ingress and egress, keep it running, and cordon the node if I suspect the host. I collect evidence: logs, the process list, network connections, the filesystem diff and, if possible, a memory image, and preserve the alert context. Then I scope the incident: what service account token and secrets the pod had, what cloud role it could assume, and whether those credentials were used elsewhere, using audit logs. I identify the entry point, such as a vulnerable endpoint or a leaked credential. Eradication means replacing the workload from a clean image, rotating all exposed secrets and tokens, and patching the weakness. Finally I tune detections and write a review.

Likely follow-ups

- Why not delete the pod immediately?
- How do you scope lateral movement?

Treat it as likely compromise. Preserve evidence, isolate the pod with network policy rather than deleting, capture logs and filesystem, identify the entry point, check lateral movement and credentials, then rebuild and rotate.

```
kubectl label pod web-7d9 quarantine=true --overwrite
```

```
kubectl apply -f networkpolicy-quarantine.yaml
```

```
kubectl logs web-7d9 --previous > pod.log
```

```
kubectl debug web-7d9 -it --image=busybox --target=app
```

Interview tip: Cordon the node and snapshot before deleting anything, so forensics remain possible.

📚 **Official docs**

- Falco documentation

certs.eknathalabs.com

Q153 GuardDuty or a similar service reports crypto-mining on an EC2 instance. What is your process?

Cloud Security

🎙 **Spoken answer · ~63 s**

I start by confirming the finding with network flow data, process information or CPU graphs, to avoid acting on a false alarm, but I do not delay containment for a likely true positive. I isolate the instance by switching to a quarantine security group, and revoke or rotate the credentials of its instance role, since a miner often arrives together with stolen credentials. I snapshot the volumes for forensics and keep the instance until evidence is captured. To find the root cause, I review how the host was reachable and what was running: an exposed management interface, a vulnerable web application, weak SSH credentials or leaked access keys. Then I hunt for scope using audit logs: new users, keys, roles, instances launched in unused regions, and changes to security settings. After cleanup I rebuild from a clean image, patch the entry point, enable guardrails like region restrictions, and set budget alerts.

Likely follow-ups

- Why check other regions?
- How can billing alerts help detection?

Verify the finding, isolate the instance, preserve snapshots, identify how it got in (exposed service, leaked key, vulnerable app), revoke credentials, hunt for other affected resources and persistence, then rebuild.

```
aws guardduty get-findings --detector-id $D --finding-ids $F
```

```
aws ec2 modify-instance-attribute --instance-id i-0abc --groups sg-quarantine
```

```
aws cloudtrail lookup-events --lookup-attributes
AttributeKey=ResourceName,AttributeValue=i-0abc
```

```
aws ec2 describe-instances --filters Name=iam-instance-profile.arn,Values=*role*
```

Interview tip: Check for new IAM users, access keys, and additional instances in other regions. Attackers often persist and scale out.

📚 **Official docs**

- AWS GuardDuty

certs.eknathalabs.com

Q154 You enforce a new admission policy and it blocks a critical production deployment during an incident. What do you do?

Policy-as-Code

🎙 Spoken answer · ~61 s

Service restoration comes first. A documented break-glass path should exist so that on-call engineers can switch the offending policy to audit mode, or add a narrowly scoped, time-limited exception, with approval recorded and an alert sent to the security team. I use it, get the deployment through, and confirm the incident is resolved. Then I analyse the cause: did the workload violate the rule legitimately, such as an old image running as root, or was the policy wrong or too broad? I fix whichever it is, either by remediating the manifest or by correcting the policy, and remove the temporary exception. In the retrospective I ask why my rollout missed it: audit mode should have surfaced this workload, perhaps the policy was enforced early or the reports were not reviewed. Improvements include better dry-run reports, staged rollout by namespace, clear communication, and drills of the emergency procedure.

Likely follow-ups

- What makes a safe break-glass path?
- How should policies roll out?

Restore service first using a documented, audited bypass or exception, then fix the root cause. Afterwards review why audit mode did not catch it and improve rollout, communication and escape hatches.

```
kubectl annotate ns prod policies.kyverno.io/exclude=true --overwrite
```

```
kubectl get clusterpolicy -o custom-
columns=NAME:.metadata.name,ACTION:.spec.validationFailureAction
```

```
kubectl patch clusterpolicy require-nonroot --type merge -p '{"spec":
{"validationFailureAction":"Audit"}}'
```

Interview tip: Pre-agree the emergency path. During an incident, nobody should be inventing how to switch a policy to audit mode.

📚 Official docs

- Conftest

certs.eknathalabs.com

Q155 An auditor asks for evidence that all production changes were reviewed and approved in the last quarter. How do you produce it?

IR & Governance

🎙 Spoken answer · ~58 s

I answer with data from systems of record, not screenshots. First I show the control design: the branch protection rules that require approval from someone other than the author, passing required checks, and no direct pushes, along with the audit log proving the settings were in place throughout the quarter. Then I demonstrate operation: from the deployment system, list every production deployment in the period, link each to the commit and merged pull request, and show approvals, reviewer identity and check results. A script flags anomalies such as merges without review, administrator bypasses and emergency changes. For those I present the emergency procedure and the retrospective approval record. An auditor will usually sample, so I can pull full evidence for any chosen change quickly. Transparency about exceptions builds credibility, and any gaps become tracked improvements, such as removing bypass rights.

Likely follow-ups

- How do you handle emergency changes?
- Why prefer system records to screenshots?

Query Git and the deployment system: list production deploys, map each to a merged PR with approvals and passing checks, flag exceptions such as emergency changes with retroactive review, and provide the protection settings as evidence.

```
gh pr list --state merged --base main --search 'merged:2026-07-01..2026-09-30' --json number,reviews,mergedBy --limit 500
```

```
gh api repos/org/repo/branches/main/protection
```

```
git log --first-parent --merges main --since=2026-07-01 --until=2026-09-30 --format='%h %s'
```

```
jq '[.[]|select(.reviews|length==0)]' prs.json
```

Interview tip: Automate the query so evidence takes minutes, and treat unreviewed changes as findings to explain, not hide.

📚 Start here (official docs)

- Kubernetes security checklist

certs.eknathalabs.com